



WYDZIAŁ ELEKTRONIKI,
TELEKOMUNIKACJI
I INFORMATYKI

Imię i nazwisko studenta: Jakub Ławicki

Nr albumu: 189788

Poziom kształcenia: studia drugiego stopnia

Forma studiów: stacjonarne

Kierunek studiów: Elektronika i telekomunikacja

Specjalność: Systemy wbudowane czasu rzeczywistego

PRACA DYPLOMOWA MAGISTERSKA

Tytuł pracy w języku polskim: Badanie wpływu różnych metod optymalizacji kodu na wydajność systemu wbudowanego opartego na systemie Linux.

Tytuł pracy w języku angielskim: Investigation of the impact of various code optimization methods on the performance of a Linux-based embedded system.

Opiekun pracy: dr hab. inż. Jacek Marszał

STRESZCZENIE

Kompilator jest niezbędnym narzędziem przy tworzeniu programów komputerowych, które efektywnie wykorzystują zasoby sprzętowe systemu. Bywa on jednak traktowany jedynie jako tłumacz kodu, z pominięciem jego możliwości optymalizacyjnych, choć sposób tłumaczenia kodu źródłowego na instrukcje maszynowe wpływa bezpośrednio na wydajność programu.

Zagadnienie to ma szczególne znaczenie w systemach wbudowanych, w których poprawa wydajności oznacza często niższy koszt i mniejszy pobór energii. W prezentowanej pracy zaprojektowano system wbudowany oparty na platformie Raspberry Pi 4B z systemem Linux, przeznaczony do badania możliwości optymalizacyjnych kompilatorów GCC i Clang. Jako algorytm testowy wybrano szybką transformatę Fouriera, dla której zaimplementowano w języku C jedenaście wariantów, zarówno sekwencyjnych jak i wielowątkowych.

W pracy zawarto również opis systemów operacyjnych w zastosowaniach wbudowanych, podstaw transformaty Fouriera oraz budowy kompilatora i stosowanych przez niego optymalizacji. Badania objęły jedenaście zestawów flag kompilacji i trzy rozmiary danych wejściowych, a oprócz czasu wykonania analizowano liczniki sprzętowe, rozmiar kodu wynikowego oraz dokładność obliczeń.

Największy zysk przyniosło odejście od domyślnego poziomu `-O0`, które skróciło czas wykonania od 2,18 do 5,11 raza, podczas gdy dalszy dobór flag dawał zwykle około 12%. Dla dużych zbiorów danych o czasie wykonania decydował przede wszystkim wzorzec dostępu do pamięci, dlatego wybór schematu obliczeń okazał się równie istotny jak poziom optymalizacji. Ręczna wektoryzacja z użyciem instrukcji NEON oraz zrównoleglenie obliczeń przynosiły wyraźne korzyści tylko w określonych zakresach rozmiaru danych. Kompilator nie skorygował przy tym błędów numerycznych wynikających z konstrukcji algorytmu. Uzyskane wyniki wskazują, że optymalizację programu należy zaczynać od doboru algorytmu i organizacji danych, a ustawienia kompilacji traktować jako ich uzupełnienie.

Słowa kluczowe: optymalizacja kompilacji, kompilatory GCC i Clang, szybka transformata Fouriera, systemy wbudowane, system operacyjny Linux

Dziedzina nauki i techniki zgodna z OECD Nauki inżynierijskie i techniczne, Elektrotechnika, elektronika i inżynieria informatyczna,

ABSTRACT

The compiler is an essential tool in the development of computer programs that make efficient use of the system's hardware resources. It is often treated, however, as a mere code translator, with its optimization capabilities left aside, even though the way source code is translated into machine instructions directly affects program performance.

This issue is of particular importance in embedded systems, where improved performance often means lower cost and reduced power consumption. This thesis presents an embedded system based on the Raspberry Pi 4B platform running Linux, designed to study the optimization capabilities of the GCC and Clang compilers. The fast Fourier transform was chosen as the test algorithm, for which eleven variants were implemented in C, both sequential and multithreaded.

The thesis also covers operating systems in embedded applications, the fundamentals of the Fourier transform, and the structure of a compiler together with the optimizations it applies. The study covered eleven sets of compilation flags and three input data sizes; in addition to execution time, hardware counters, output code size, and computational accuracy were analysed.

The largest gain came from moving away from the default `-O0` level, which reduced execution time by a factor of 2.18 to 5.11, whereas further flag selection typically yielded around 12%. For large data sets, execution time was determined mainly by the memory access pattern, which made the choice of computational scheme as important as the optimization level. Manual vectorization using NEON instructions and parallelization of the computation brought clear benefits only within specific ranges of data size. The compiler did not correct numerical errors resulting from the construction of the algorithm. The results indicate that program optimization should begin with the choice of algorithm and data organization, with compilation settings treated as a complement to them.

Keywords: compilation optimization, GCC and Clang compilers, fast Fourier transform, embedded systems, Linux operating system

OECD consistent field of science and technology classification: Engineering and technology, Electrical engineering, Electronic engineering, Information engineering,

SPIS TREŚCI

Spis treści	5
1 Wstęp i cel pracy	8
1.1 Cel pracy	8
1.2 Zawartość rozdziałów	9
2 Architektura systemów operacyjnych w zastosowaniach wbudowanych	10
2.1 Rola systemów operacyjnych w systemach wbudowanych	10
2.1.1 Model warstwowy systemów operacyjnych	11
2.2 Kluczowe abstrakcje i mechanizmy stosowane w systemach operacyjnych	12
2.2.1 Tryby pracy procesora	13
2.2.2 Procesy	13
2.2.3 Wątki	13
2.2.4 Szeregowanie zadań	14
2.2.5 Pamięć wirtualna	14
2.3 Systemy wbudowane czasu rzeczywistego wraz z ich podziałem	14
2.4 Architektura struktur jądra systemów operacyjnych	16
2.5 Studium przypadku systemu operacyjnego Linux	17
2.5.1 Procesy, wątki w systemie Linux	18
2.5.2 Szeregowanie i synchronizacja w systemie Linux	18
2.5.3 Zarządzanie pamięcią	19
3 Algorytm przetwarzania danych i techniki optymalizacji programów komputerowych	20
3.1 Opis i analiza Szybkiej Transformaty Fouriera	20
3.1.1 Podstawy matematyczne algorytmu FFT	20
3.1.2 Szybka transformata Fouriera	22
3.2 Opis procesu kompilacji programu komputerowego	26
3.2.1 Procesory języka programowania	26
3.2.2 Szczegółowy opis procesu kompilacji	26
3.2.3 Wewnętrzna struktura kompilatora	28
3.3 Przegląd najpopularniejszych kompilatorów języka C	29
3.3.1 Kompilator GCC	29
3.3.2 Kompilator Clang	29
3.4 Zaawansowane techniki optymalizacji kodu i flagi kompilacji	30
3.4.1 Przekształcenia optymalizujące	31
3.4.2 Hierarchia pamięci i pamięć podręczna	32
3.4.3 Podstawowe flagi optymalizacyjne	32
4 Projekt systemu testowego i metodyka przeprowadzonych badań	35
4.1 Opis platformy sprzętowej	35
4.1.1 Jednostka NEON	36
4.2 Zaimplementowane warianty algorytmu FFT	37

4.2.1	Wspólne założenia implementacyjne	38
4.2.2	Użyte biblioteki	38
4.2.3	Badane wersje pojedynczo wątkowe	39
4.2.4	Badane wersje wielowątkowe	42
4.3	Założenia projektowe i architektura aplikacji	44
4.3.1	Użyty system operacyjny i jego konfiguracja	44
4.3.2	Architektura programu testowego	45
4.3.3	Programowe narzędzia pomiarowe	46
4.3.4	Weryfikacja poprawności obliczeń z biblioteka zewnętrzna	46
4.4	Metodyka przeprowadzonych pomiarów	48
4.4.1	Metryki używane w badaniach	49
5	Analiza wyników przeprowadzonych badań	51
5.1	Powtarzalność pomiarów i próg istotności różnic	51
5.2	Analiza wpływu optymalizacji programu sekwencyjnego	52
5.2.1	Wpływ poziomów optymalizacji kompilatora	52
5.2.2	Wpływ celowych modyfikacji kodu programu, w tym wybór algorytmu	53
5.2.3	Porównanie kompilatorów GCC i Clang	54
5.2.4	Wpływ kompilatora na błędy numeryczne wybranych algorytmów	55
5.2.5	Wpływ wybranych flag na wektoryzację	56
5.2.6	Wpływ rozmiaru danych wejściowych	57
5.3	Analiza wpływu optymalizacji programu wielowątkowego	59
5.3.1	Wpływ flag kompilacji na warianty równoległe	60
5.3.2	Skalowanie z liczbą wątków i wpływ rozmiaru danych	60
5.3.3	Wpływ sposobu podziału pracy i schematu algorytmu	61
6	Podsumowanie i wnioski	63
6.1	Wnioski końcowe	63
6.2	Kierunek dalszych badań	65
	Spis rysunków	66
	Spis tabel	67

LISTA SYMBOLI

N	- liczba próbek transformaty
n	- indeks próbki w dziedzinie czasu
k	- indeks prążka widma
$x(n)$	- n -ta próbka sygnału dyskretnego
$X(k)$	- k -ty prążek widma dyskretnego
W_N	- czynnik obrotu transformaty o długości N
p, q	- pozycje dwóch elementów przetwarzanych w operacji motylkowej
s	- numer etapu algorytmu FFT
$t_{1/2}$	- mediana czasu wykonania transformaty
Q_1, Q_3	- pierwszy i trzeci kwartył czasu wykonania
t_{CPU}	- sumaryczny czas procesora
t_{wall}	- czas ściany, czyli rzeczywisty czas trwania obliczeń
R	- stosunek czasu procesora do czasu ściany
I	- liczba wykonanych instrukcji
C	- liczba cykli procesora
m	- współczynnik chybień danego poziomu pamięci podręcznej
r, a	- liczba chybień i liczba odwołań do danego poziomu pamięci podręcznej
v	- udział instrukcji wektorowych w kodzie wynikowym
$n_{\text{SIMD}}, n_{\text{skal}}$	- liczba instrukcji wektorowych i skalarnych
X_k, X_k^{ref}	- k -ty prążek widma badanego wariantu i widma referencyjnego

LISTA SKRÓTÓW

AoS	- układ złożony danych (ang. <i>Array of Structures</i>)
DFT	- dyskretna transformacja Fouriera (ang. <i>Discrete Fourier Transform</i>)
DIF	- decymacja w dziedzinie częstotliwości (ang. <i>Decimation in Frequency</i>)
DIT	- decymacja w dziedzinie czasu (ang. <i>Decimation in Time</i>)
FFT	- szybka transformacja Fouriera (ang. <i>Fast Fourier Transform</i>)
FFTW	- biblioteka szybkiej transformacji Fouriera (ang. <i>Fastest Fourier Transform in the West</i>)
GCC	- zestaw kompilatorów projektu GNU (ang. <i>GNU Compiler Collection</i>)
IPC	- liczba instrukcji wykonanych w jednym cyklu (ang. <i>Instructions Per Cycle</i>)
IQR	- rozstęp międzykwartylowy (ang. <i>Interquartile Range</i>)
LLVM	- infrastruktura budowy kompilatorów (ang. <i>Low Level Virtual Machine</i>)
LTO	- optymalizacja na etapie konsolidacji (ang. <i>Link Time Optimization</i>)
MMU	- jednostka zarządzania pamięcią (ang. <i>Memory Management Unit</i>)
NEON	- jednostka wektorowa architektury ARM (ang. <i>Advanced SIMD</i>)
POSIX	- przenośny interfejs systemów operacyjnych (ang. <i>Portable Operating System Interface</i>)
RMSE	- pierwiastek błędu średniokwadratowego (ang. <i>Root Mean Square Error</i>)
RTOS	- system operacyjny czasu rzeczywistego (ang. <i>Real-Time Operating System</i>)
SIMD	- pojedynczy strumień instrukcji, wiele strumieni danych (ang. <i>Single Instruction, Multiple Data</i>)
SNR	- stosunek sygnału do szumu (ang. <i>Signal-to-Noise Ratio</i>)
SoA	- układ rozdzielony danych (ang. <i>Structure of Arrays</i>)
SoC	- system w układzie scalonym (ang. <i>System on Chip</i>)

1. WSTĘP I CEL PRACY

Kompilator jest narzędziem, z którego inżynier oprogramowania korzysta przy każdym uruchomieniu procesu budowania, a mimo to rzadko bywa przedmiotem jego uwagi. W praktyce traktuje się go jak czarną skrzynkę: na wejściu znajduje się kod źródłowy, na wyjściu plik wykonywalny, a to, co dzieje się pomiędzy, pozostaje poza zainteresowaniem programisty. Poziom optymalizacji ustawia się zwykle raz, przy konfigurowaniu projektu, i nie wraca się do niego przez cały cykl życia oprogramowania. Gdy pojawia się potrzeba przyspieszenia programu, modyfikowana jest przede wszystkim logika algorytmu, znacznie rzadziej rozważa się wpływ, jaki na wynik końcowy ma sam proces translacji kodu na instrukcje maszynowe.

Pominięcie to jest o tyle zaskakujące, że znaczna część oprogramowania działającego bezpośrednio na sprzęcie powstaje w językach kompilowanych, takich jak język C [1]. Sposób, w jaki kompilator tłumaczy słowa kluczowe tego języka na instrukcje konkretnej architektury, decyduje o liczbie wykonanych operacji, o wykorzystaniu rejestrów oraz o tym, czy program skorzysta z jednostek wektorowych procesora. Każdy z tych czynników wpływa bezpośrednio na czas wykonania, a jednak żaden z nich nie wymaga zmiany choćby jednej linii kodu źródłowego.

Znaczenie tego zagadnienia rośnie w systemach wbudowanych, ponieważ dysponują one ograniczoną mocą obliczeniową, niewielką pamięcią operacyjną i pracują często w rozwiązaniach wymagających oszczędzania energii. W ich przypadku poprawa wydajności uzyskana bez wymiany sprzętu przekłada się wprost na niższy koszt systemu, mniejszy pobór energii oraz większy zapas mocy obliczeniowej na pozostałe zadania. Jeżeli zmiana ustawień kompilacji pozwala skrócić czas wykonania krytycznej procedury, jest to korzyść osiągnięta bez żadnych nakładów.

Wpływu ustawień kompilacji na czas wykonania nie da się jednak oszacować bez pomiaru. Kompilator podejmuje decyzje na podstawie struktury konkretnego kodu, zestawu dostępnych instrukcji procesora oraz przyjętego modelu kosztu. Dodatkowo część flag się wyklucza. Do udzielenia odpowiedzi potrzebne jest zatem stanowisko pomiarowe, na którym można uruchomić ten sam kod w wielu wariantach kompilacji i porównać określone metryki optymalizacji.

1.1. Cel pracy

Celem niniejszej pracy jest projekt systemu wbudowanego przeznaczonego do badania możliwości optymalizacyjnych kompilatora oraz do porównania różnych metod poprawy wydajności programu. Jako algorytm przetwarzania danych wybrano szybką transformatę Fouriera, algorytm o ustalonej i dobrze opisanej strukturze, szeroko wykorzystywany w cyfrowym przetwarzaniu sygnałów, a zarazem na tyle wymagający obliczeniowo, że różnice w jakości wygenerowanego kodu są w jego przypadku mierzalne.

Na system składają się dwie części. Część sprzętową stanowi Raspberry Pi 4B, pracująca pod kontrolą systemu operacyjnego Linux. Część programowa obejmuje jedenaście wariantów implementacji transformaty Fouriera, różniących się od siebie podejściem do realizacji Dyskretnej transformaty Fouriera oraz program pomiarowy, który uruchamia je w kontrolowanych warunkach i rejestruje czas wykonania wraz z odczytami niezbędnych informacji o innych metrykach optymalizacji.

Zakres pracy obejmuje trzy grupy zagadnień. Pierwszą z nich jest zbadanie, jaki wpływ na wydajność ma sama zmiana flag optymalizacji, przy niezmiennym kodzie źródłowym. Drugą

stanowi porównanie tego efektu z korzyściami wynikającymi z celowych modyfikacji programu, takich jak zmiana układu danych wejściowych, jawne użycie instrukcji wektorowych czy wybór odmiennego schematu obliczeń. Trzecią jest sprawdzenie, w jakim stopniu wnioski te zachowują ważność przy zmianie rozmiaru przetwarzanych danych oraz po zrównolegleniu obliczeń na wiele rdzeni procesora.

1.2. Zawartość rozdziałów

Ogólną strukturę niniejszej pracy można przedstawić w następujący sposób:

- **Rozdział 2: Architektura systemów operacyjnych w zastosowaniach wbudowanych.**
Omawia rolę systemu operacyjnego w systemie wbudowanym, stosowane w nim abstrakcje i mechanizmy, podział systemów czasu rzeczywistego oraz architektury jądra. Zawiera studium przypadku systemu Linux, użytego jako platforma programowa w niniejszej pracy.
- **Rozdział 3: Algorytm przetwarzania danych i techniki optymalizacji programów komputerowych.**
Przedstawia podstawy matematyczne dyskretnej i szybkiej transformacji Fouriera, przebieg procesu kompilacji oraz wewnętrzną strukturę kompilatora. Zawiera przegląd kompilatorów GCC i Clang, opis przekształceń optymalizujących oraz odpowiadających im flag kompilacji.
- **Rozdział 4: Projekt systemu testowego i metodyka przeprowadzonych badań.**
Opisuje platformę sprzętową, jedenaście zaimplementowanych wariantów transformaty Fouriera oraz architekturę programu pomiarowego. Zawiera badane zestawy flag kompilacji, przyjętą metodykę pomiarów i definicje stosowanych metryk.
- **Rozdział 5: Analiza wyników przeprowadzonych badań.**
Zawiera ocenę powtarzalności pomiarów wraz z progiem istotności różnic, a następnie analizę wyników osobno dla wariantów sekwencyjnych i współbieżnych. Poszczególne podrozdziały omawiają wpływ poziomów optymalizacji, celowych modyfikacji kodu, wyboru kompilatora, rozmiaru danych wejściowych oraz liczby wątków.
- **Rozdział 6: Podsumowanie i wnioski.**
Zawiera wnioski końcowe wynikające z przeprowadzonych badań oraz propozycje kierunków dalszych prac.

2. ARCHITEKTURA SYSTEMÓW OPERACYJNYCH W ZASTOSOWANIACH WBUDOWANYCH

Projektowanie złożonych systemów komputerowych, zwłaszcza takich jak rozwiązania bazujące na implementacji algorytmów przetwarzania sygnałów w rygorze czasu rzeczywistego przy jednocześnie ograniczonych zasobach sprzętowych, jest zadaniem, które wymaga wielu przemyślnych decyzji przy doborze architektury tworzonego systemu.

Wybór odpowiedniej platformy programowej jest jedną z kluczowych decyzji projektowych. Projektant systemu wbudowanego staje wówczas często przed wyborem między podejściem bezsystemowym, zapewniającym pełną kontrolę nad sprzętem, a zastosowaniem systemu operacyjnego, który wprowadza warstwę abstrakcji kosztem częściowej utraty determinizmu czasowego oraz zwiększenia zużycia fizycznych zasobów systemu [1]. Decyzja ta ma bezpośredni wpływ na poprawność systemu, nie tylko z uwagi na osiąganą wydajność obliczeniową, ale również poprzez bezpieczeństwo i stabilność systemu. W celu zapewnienia wysokiej jakości projektowanego systemu, inżynier oprogramowania powinien posiadać podstawową wiedzę na temat działania systemów operacyjnych. Znajomość ich mechanizmów pozwala projektować systemy komputerowe charakteryzujące się wysoką wydajnością i niezawodnością.

Celem prezentowanego rozdziału jest usystematyzowanie wiedzy niezbędnej do zrozumienia, w jaki sposób architektura stosowanego systemu operacyjnego wpływa na wydajność uruchamianych na nim programów. Rozważania rozpoczynają się od przedstawienia warstwowego modelu systemów operacyjnych oraz kluczowych mechanizmów jądra, takich jak tryby pracy procesora, procesy, wątki, szeregowanie zadań, pamięć wirtualna czy synchronizacja współbieżnych operacji. Na tej podstawie omówiony zostaje podział systemów wbudowanych czasu rzeczywistego, a także dwa podstawowe modele budowy jądra: architektura monolityczna oraz mikrojądrowa. Ostatnia część rozdziału zawiera studium przypadku systemu operacyjnego Linux, w którym w skrócony sposób starano się przedstawić niezbędne szczegóły implementacji mechanizmów systemowych.

2.1. Rola systemów operacyjnych w systemach wbudowanych

Podczas projektowania systemu wbudowanego, włączając w to system wbudowany czasu rzeczywistego, projektant staje przed wyborem jednego z dwóch podstawowych podejść do organizacji i kontroli całego środowiska. Pierwszym z nich jest **podejście bezsystemowe** (ang. *bare-metal*), w którym całe oprogramowanie funkcjonuje bez żadnej warstwy pośredniej, mając bezpośredni i wyłączny dostęp do fizycznych układów procesora oraz peryferiów. Choć takie podejście zapewnia pełną kontrolę nad sprzętem i minimalne opóźnienia, znacznie ogranicza skalowalność systemu. Każda zmiana komponentu lub rozbudowa funkcjonalności może wymagać istotnych modyfikacji w strukturze kodu.

Alternatywą jest zastosowanie **systemu operacyjnego**, który możemy zdefiniować jako podstawowe oprogramowanie pośredniczące między sprzętem komputerowym a użytkownikiem, którego głównym zadaniem jest zarządzanie i przydzielanie zasobów sprzętowych oraz tworzenie środowiska pozwalającego na wykonywanie programów zaimplementowanych przez jego użytkownika [2].

Jak wynika z przytoczonej definicji, stosowanie systemu operacyjnego pomaga rozwiązy-

wać niektóre problemy napotykane przy stosowaniu podejścia bezsystemowego. Dzieje się tak poprzez całościowe odizolowanie warstwy programowej od fizycznych struktur sprzętu. System operacyjny działa jako nadrzędny pośrednik, przejmując na siebie zadanie nadzorowania środowiska wykonawczego i separując logikę działania od konkretnej platformy sprzętowej.

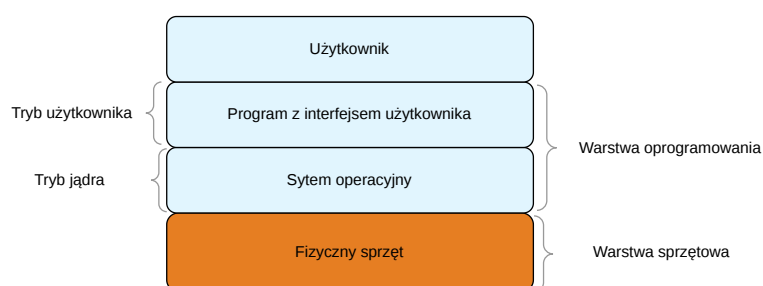
Aby w pełni zrozumieć, jak system operacyjny realizuje te zadania w środowisku wbudowanym, konieczne jest przyjrzenie się jego wewnętrznej architekturze. W dalszych sekcjach zostaną szczegółowo omówione kluczowe mechanizmy oraz podstawowa struktura ogólnych systemów operacyjnych.

2.1.1. Model warstwowy systemów operacyjnych

W inżynierii oprogramowania istotną rolę odgrywa koncepcja **abstrakcji**, polegająca na celowym ukryciu szczegółów implementacji danej funkcjonalności oprogramowania, przed jej potencjalnym użytkownikiem [1]. Podejście to opiera się na założeniu, że należy dostarczyć gotową funkcjonalność wraz z odpowiednią dokumentacją opisującą sposób jej wykorzystania, zamiast wymagać od użytkownika zrozumienia wewnętrznych mechanizmów działania wytworzonego modułu.

Taka praktyka w sposób znaczący podnosi wygodę wytwarzania nowych elementów systemu, umożliwia zrównoleglenie wytwarzania oprogramowania oraz pozwala na większą specjalizację jej producentów. W celu realizacji komunikacji między kolejnymi modułami systemu stosuje się takie narzędzia jak **interfejsy programistyczne** (ang. *Application Programming Interface*, skrót **API**), czyli zbiór ściśle określonych reguł i sposobów komunikacji, które umożliwiają komunikację między różnymi warstwami oprogramowania. Narzędzie to jest niezbędne, aby w sposób przewidywalny i bezpieczny przekazywać dane lub sterowanie, między różnymi elementami systemu.

Pojęcie abstrakcji jest kluczowe dla zrozumienia architektury współczesnych systemów operacyjnych. W literaturze spotyka się różne podejścia do klasyfikacji poziomów systemu komputerowego [1, 2], jednak ich cechą wspólną jest wyraźne odseparowanie zasobów fizycznych od uruchamianego na nich oprogramowania. Na potrzeby niniejszej pracy przyjęto strukturę opartą na propozycji przedstawionej w [2], którą w graficzny sposób zaprezentowano na rysunku 2.1.



Rysunek 2.1: Warstwowy model systemu komputerowego

Jak pokazano na schemacie, najniższy poziom stanowi warstwa sprzętowa, do której zalicza się wszystkie fizyczne komponenty urządzenia, takie jak **jednostka centralna** (ang. *Central Processing Unit*, CPU), **pamięć** oraz **urządzenia wejścia-wyjścia** (ang. *Input/Output Devices*, I/O). Ponad nią znajduje się warstwa oprogramowania, na którą składa się zarówno system operacyjny, jak i programy użytkownika. Choć oba te elementy należą do tej samej warstwy, funkcjonują one na innych zasadach. Odróżnia je praca w różnych trybach procesora, różniących

się poziomem uprawnień oraz zakresem dostępu do fizycznych urządzeń. Tryby te nazywają się odpowiednio **trybem jądra** (ang. *kernel mode*), dla systemu operacyjnego, oraz **trybem użytkownika** (ang. *user mode*), dla aplikacji użytkownika. Szczegółowe omówienie przytoczonych trybów oraz mechanizmów ich przełączania zostanie przedstawione w dalszych częściach tego rozdziału.

Warto przy tym podkreślić, że w przytoczonej pozycji literatury [2] jako integralny komponent struktury systemu komputerowego uwzględnia się jego użytkownika. Zabieg ten ma na celu jednoznaczne wskazanie, że przy projektowaniu dowolnego systemu komputerowego należy zawsze pamiętać o jego ostatecznym przeznaczeniu oraz o użytkowniku końcowym.

W modelu warstwowym architektury komputerowej system operacyjny pełni szczególną rolę, zarządzając granicą między aplikacjami a warstwą fizyczną. W celu zapewnienia poprawnego i przewidywalnego działania systemu wprowadza poziomy abstrakcji, które są niezbędne z kilku istotnych powodów:

1. **Konieczność zapewnienia bezpieczeństwa** - Współczesne systemy operacyjne charakteryzują się ogromną złożonością, a ich kod źródłowy sięga kilkudziesięciu milionów linii [1]. Izolacja poszczególnych poziomów wprowadza wyraźną hierarchię uprawnień. Dzięki temu błąd powstały w kodzie aplikacyjnym pozostaje odizolowany i nie wpływa bezpośrednio na stabilność pozostałych funkcjonalności systemowych ani na pracę samego sprzętu [2].
2. **Zapewnienie przenoszalności oprogramowania** - Zastosowanie warstw pośrednich sprawia, że oprogramowanie nie jest bezpośrednio uzależnione od konkretnych urządzeń fizycznych. Pozwala to na uruchamianie tych samych programów na różnorodnych platformach sprzętowych, umożliwiając przy tym niezależny rozwój warstwy aplikacyjnej i sprzętowej [2].
3. **Umożliwienie korzystania z narzędzi wysokiego poziomu** - Ukrycie technicznych szczegółów pracy ze sterownikami lub rejestrarami pozwala na tworzenie kodu w językach wysokiego poziomu. Programista może skupić się na logice i strukturze zgodnej ze sposobem myślenia człowieka, zamiast na bezpośredniej, niskopoziomowej obsłudze układów elektronicznych [2].

2.2. Kluczowe abstrakcje i mechanizmy stosowane w systemach operacyjnych

Pomimo dużej różnorodności współczesnych systemów operacyjnych, ich architektura opiera się na spójnym zestawie uniwersalnych abstrakcji i mechanizmów. Od strony koncepcyjnej rozwiązania te wprowadzają ład w złożonym środowisku sprzętowym, przybliżając sposób funkcjonowania komputera do logiki myślenia i pracy człowieka.

Koncepcje opisane w prezentowanej sekcji pozwalają na stworzenie przejrzystego i uporządkowanego modelu zarządzania zasobami w systemie operacyjnym. Zrozumienie tych bazowych koncepcji jest kluczowe dla analizy wydajności, ponieważ to one determinują sposób, w jaki program aplikacji, stworzonej przez inżyniera oprogramowania, jest uruchamiany i wykonywany na fizycznym sprzęcie.

Na potrzeby niniejszej pracy konieczne jest przedstawienie następujących pojęć i mechanizmów:

- Tryby pracy procesora
- Proces
- Wątek

- Szeregowanie zadań
- Pamięć wirtualna

2.2.1. Tryby pracy procesora

Współczesne komputery realizują ochronę zasobów poprzez tryby pracy procesora [2], z których kluczowe to **tryb użytkownika** (ang. user mode) oraz **tryb jądra** (ang. kernel mode). Aplikacje użytkownika funkcjonują w trybie użytkownika, nieuprzywilejowanym, co uniemożliwia im między innymi, bezpośredni dostęp do urządzeń peryferyjnych oraz fizycznej pamięci.

Wszelkie operacje wymagające kontroli nad sprzętem realizowane są za pośrednictwem **wywołań systemowych** [2] (ang. system calls), które przełączają procesor w uprzywilejowany tryb jądra.

2.2.2. Procesy

Proces (ang. process) można rozumieć jako wykonujący się program komputerowy [2]. Jego kluczową cechą jest przydzielona, odizolowana od innych procesów wirtualna **przestrzeń adresowa**. Jej wprowadzenie ma zagwarantować bezpieczeństwo - błąd w obrębie jednego procesu nie wpływa bezpośrednio na stabilność procesów systemowych i innych procesów użytkownika.

Współczesne systemy komputerowe składają się z licznych komponentów i aplikacji, co sprawia, że na wspólnych, fizycznych zasobach sprzętowych funkcjonuje jednocześnie wiele procesów. Ze względu na to, że pojedynczy rdzeń procesora może w danej chwili realizować tylko jedno zadanie, system operacyjny stosuje przełączanie kontekstu, cyklicznie przydzielając czas procesora poszczególnym procesom [1]. Ponieważ każdy proces operuje w odrębnej przestrzeni adresowej, zmiana ta wymaga przeładowania pamięci operacyjnej. W efekcie przełączanie kontekstu między procesami jest operacją stosunkowo czasochłonną i wiąże się z zauważalnym opóźnieniem, które można nazywać też **narzutem systemu operacyjnego** [1].

2.2.3. Wątki

Wątek (ang. thread) stanowi wyodrębnioną jednostkę wykonawczą (ciąg instrukcji) uruchamianą na procesorze, funkcjonującą w obrębie konkretnego procesu [1].

W ramach jednego procesu może działać wiele wątków wykonujących się we wspólnej przestrzeni adresowej. Wątki należące do tego samego procesu współdzielą ze sobą zasoby systemowe (m.in. przestrzeń adresową, zmienne globalne czy otwarte pliki), jednak każdy z nich posiada własny, prywatny kontekst wykonania [1], na który składają się:

- **Licznik instrukcji** (ang. Program Counter),
- **Zestaw rejestrów procesora**,
- **Prywatny stos** (przechowujący zmienne lokalne oraz historię wywołań funkcji),
- **Stan wątku** (np. uruchomiony, zablokowany, gotowy).

Dzięki współdzieleniu przestrzeni adresowej wątki generują znacznie mniejszy narzut czasowy niż procesy, przy przełączaniu, wymagając jednak stosowania odpowiednich mechanizmów synchronizacji.

Zarządzanie wątkami może być realizowane w przestrzeni użytkownika lub bezpośrednio w jądrze systemu. W celu zapewnienia przenoszalności oprogramowania wielowątkowego między różnymi środowiskami operacyjnymi stworzony został standard **POSIX Threads** (znany jako **Pthreads**, zdefiniowany w normie IEEE 1003.1c) [1]. Definiuje on uniwersalne interfejsy programistyczne przeznaczone m.in. do tworzenia, kończenia oraz synchronizacji pracy wątków.

2.2.4. Szeregowanie zadań

Szeregowanie zadań odpowiada za zarządzanie przydziałem czasu procesora. Pojedynczy rdzeń CPU nie jest w stanie wykonywać wielu zadań jednocześnie, dlatego **planista** (ang. scheduler) ustala kolejność oraz czas uruchamiania wątków i procesów na podstawie określonych algorytmów [2]. Mechanizm ten zapewnia sprawiedliwy dostęp do zasobów i płynność działania systemu, jednak częsta zmiana wykonywanych zadań generuje bezpośredni narzut czasowy związany z przełączaniem kontekstu.

2.2.5. Pamięć wirtualna

Pamięć wirtualna sprawia, że procesy nie operują bezpośrednio na fizycznej pamięci operacyjnej, lecz na wirtualnej przestrzeni adresowej przydzielanej i mapowanej na sprzęt przez **jednostkę zarządzania pamięcią (MMU, ang. Memory Management Unit)** [1]. Należy w tym miejscu zaznaczyć, że jednym z warunków korzystania z systemów operacyjnych jest obecność takiej jednostki w sprzęcie, na którym chcemy ten system uruchomić.

W kontekście pamięci wirtualnej kluczowym mechanizmem jest **stronicowanie** [1]. Polega ono na podziale wirtualnej przestrzeni adresowej programu na bloki o stałym rozmiarze, zwane stronami (ang. *pages*), które są dynamicznie odwzorowywane na fizyczną pamięć operacyjną. Operacje tę wykonuje jednostka MMU. Mechanizm zarządzania pamięcią wirtualną został opisany, na przykładzie systemu Linux w sekcji 2.5.3

2.3. Systemy wbudowane czasu rzeczywistego wraz z ich podziałem

Systemem wbudowanym nazywamy dedykowany system komputerowy, który stanowi integralną część większego urządzenia lub maszyny i jest zaprojektowany do wykonywania ściśle określonego zadania [1]. W przeciwieństwie do komputerów ogólnego przeznaczenia (takich jak komputery osobiste czy smartfony), na których użytkownik może uruchamiać dowolne aplikacje, system wbudowany ma z góry zdefiniowaną funkcję i nie jest przeznaczony do późniejszych, swobodnych modyfikacji przez użytkownika końcowego [2].

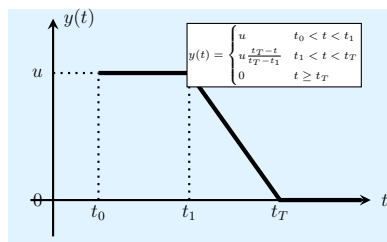
Szczególną grupą w obrębie systemów wbudowanych są **systemy wbudowane czasu rzeczywistego**. Są to takie systemy, których poprawność wyniku przetwarzania nie zależy tylko od poprawności logicznej odpowiedzi, ale też od czasu przetwarzania, poprzedzającego odpowiedź systemu [3]. W przypadku niewypracowania odpowiedzi w zadanym czasie zdarzenie to jest traktowane jako błąd systemu, co w skrajnych przypadkach może prowadzić do awarii lub uszkodzenia fizycznego sprzętu.

Systemy czasu rzeczywistego są powszechnie stosowane w zastosowaniach krytycznych, takich jak przemysł zbrojeniowy, aparatura medyczna czy branża motoryzacyjna, co wymusza wprowadzenie ścisłych kryteriów oceny ich niezawodności. W celu jednoznacznego zdefiniowania wymagań dotyczących determinizmu czasowego, w literaturze wprowadza się klasyfikacje systemów ze względu na rygor zachowania narzuconych terminów.

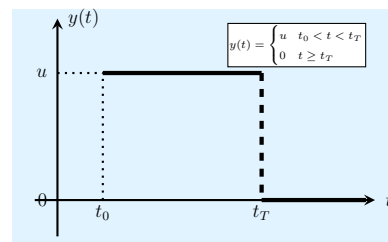
W literaturze [1] systemy czasu rzeczywistego dzieli się na:

- **Systemy twarde (Hard real-time systems)**
- **Systemy miękkie (Soft real-time systems)**
- **Systemy stabilne (Firm real-time systems)**

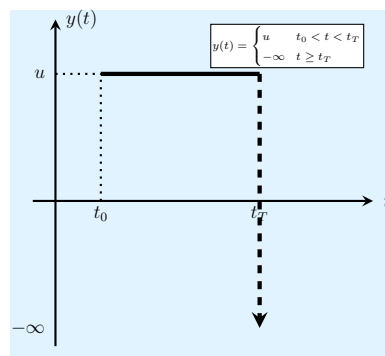
W celu lepszego zobrazowania podziału systemów czasu rzeczywistego, na rysunku rys. 2.2 przedstawiono wykresy funkcji przydatności odpowiedzi $y(t)$ od czasu jej wypracowania przez system [1].



(a) Miękki system czasu rzeczywistego



(b) Solidny system czasu rzeczywistego



(c) Twardy system czasu rzeczywistego

Rysunek 2.2: Zestawienie przebiegów funkcji przydatności odpowiedzi $y(t)$ w dziedzinie czasu [1].

Przed przystąpieniem do analizy poszczególnych wykresów, konieczne jest zdefiniowanie parametrów oraz skrótów stanowiących zmienne opisujące te funkcje:

- t – oś czasu, reprezentująca moment dostarczenia odpowiedzi przez system,
- $y(t)$ – funkcja przydatności (wartości) odpowiedzi dla obsługiwanego procesu,
- u – maksymalna, nominalna przydatność prawidłowo wypracowanego wyniku,
- t_0 – moment pojawienia się zdarzenia lub żądania wypracowania odpowiedzi przez system,
- t_1 – czas graniczny, do którego odpowiedź zachowuje pełną wartość (w systemach miękkich),
- t_T – ostateczny limit czasowy (ang. *deadline*), po przekroczeniu którego odpowiedź nie ma już pełnej wartości.

Analizując przebiegi funkcji przydatności dla poszczególnych klas systemów, można precyzyjnie scharakteryzować ich zachowanie.

Najbardziej restrykcyjne kryteria czasowe są charakterystyczne dla twardych systemów czasu rzeczywistego [1], co zobrazowano na rysunku 2.2c. Ich stosowanie gwarantuje bezwzględnie terminowe wypełnienie zadań krytycznych przed upływem limitu t_T . Przekroczenie czasu przetwarzania powoduje natychmiastowy spadek funkcji przydatności do wartości $-\infty$. W przypadku rozwiązań spóźniona odpowiedź jest nie tylko całkowicie bezużyteczna, ale jej pojawienie się po wyznaczonym terminie jest najczęściej równoznaczne z wystąpieniem krytycznego błędu, który może doprowadzić do poważnej awarii lub fizycznego uszkodzenia całego systemu [2].

Kolejną z omawianych grup stanowią miękkie systemy czasu rzeczywistego, których zachowanie zilustrowano na rysunku 2.2a. Systemy te cechują się mniejszym rygorem w kwestii terminowości, co oznacza, że zadania są wykonywane tak szybko, jak to możliwe, a ewentualne opóźnienia nie niosą za sobą skutków katastrofalnych [1]. Jeśli odpowiedź zostanie dostarczona po czasie t_1 , ale przed ostatecznym limitem t_T , jej przydatność maleje, powodując jedynie spadek jej jakości. Przy przetwarzaniu przekraczającym czas t_T przydatność odpowiedzi wynosi zero.

Solidne systemy czasu rzeczywistego stanowią pewnego rodzaju kompromis pomiędzy twardymi a miękkimi systemami czasu rzeczywistego. Zachowanie funkcji przydatności systemów solidnych przedstawiono na rysunku 2.2b. Każda odpowiedź dostarczona w przedziale od t_0 do t_T zachowuje stałą, pełną wartość u . W momencie przekroczenia terminu granicznego t_T przydatność wyniku natychmiastowo i skokowo spada do zera. Choć fakt spóźnienia powoduje całkowitą nieprzydatność rozwiązania, to sytuacja ta wciąż nie generuje bezpośredniego zagrożenia dla stabilności czy integralności samego systemu.

Zdefiniowane powyżej rygory czasowe określają teoretyczne oczekiwania wobec systemu czasu rzeczywistego. W praktyce realizacja tych założeń sprowadza się do oceny zasadności stosowania **systemu operacyjnego czasu rzeczywistego** w zderzeniu z bezpośrednim programowaniem sprzętu. Decyzja ta bezpośrednio determinuje strukturę i elastyczność budowanego środowiska.

2.4. Architektura struktur jądra systemów operacyjnych

Systemy operacyjne można klasyfikować według wielu kryteriów, z których jednym z najczęściej spotykanym jest podział, ze względu na ich docelowe przeznaczenie. Przy takim podziale możemy wyróżnić systemy operacyjne generalnego przeznaczenia, systemy operacyjne rozproszone, systemy operacyjne komputerów mainframe, systemy operacyjne serwerów oraz wiele innych [1].

Jednakże, w kontekście analizy wydajności kluczowy jest podział ze względu na strukturę systemu, która określa architekturę jego jądra. Na tym etapie jądro systemu możemy zdefiniować jako jedyny program, który działa w pamięci komputera nieprzerwanie przez cały czas od jego uruchomienia, pełniąc funkcję pomostu komunikacyjnego między uruchomionymi aplikacjami, a fizycznym sprzętem [2].

Konstrukcja jądra decyduje o sposobie komunikacji między usługami systemowymi, a warstwą aplikacyjną, co bezpośrednio wpływa na narzuty czasowe oraz efektywność optymalizacji kodu programów. Z uwagi na to, że przedmiotem badań w niniejszej pracy jest system Linux, analiza skupi się głównie na architekturze monolitycznej. Dla celów porównawczych zostanie ona zestawiona z inną szeroko spotykaną architekturą w systemach wbudowanych - strukturą mikrojądra.

W architekturze monolitycznej (ang. *monolithic kernel*) wszystkie główne zadania sys-

temu operacyjnego są połączone w jeden rozbudowany program komputerowy [1]. Zarządzanie pamięcią, programami oraz wszystkimi sterownikami urządzeń odbywa się we wspólnej przestrzeni. Takie podejście sprawia, że jądro zawiera w sobie ogromną liczbę elementów, co mocno komplikuje proces jego tworzenia oraz późniejszego szukania błędów. Największą zaletą architektury monolitycznej jest jednak wysoka prędkość przetwarzania informacji - z uwagi na to, że jądro jest w całości kompilowane do jednego programu, poszczególne funkcjonalności mogą z siebie nawzajem korzystać, co znacząco przyspiesza działanie systemu. Przy architekturze monolitycznej aplikacje użytkownika są uruchamiane jako osobne procesy w chronionej pamięci. Dzięki temu jądro nie jest narażone na błędy w programach użytkownika, a różne aplikacje nie mogą nawzajem bezpośrednio mieć dostępu zasobów im nie przydzielonych. Główną wadą systemów monolitycznych pozostaje jednak słaba odporność na awarie samego jądra; jeśli błąd pojawi się w jego obrębie, na przykład w jednym ze sterowników, może dojść do awarii całego systemu.

Architektura mikrojądra (ang. *microkernel*) opiera się na zasadzie minimalizacji kodu wykonywanego z najwyższymi uprawnieniami procesora [2]. Kompletny system operacyjny składa się w tym przypadku z uproszczonego jądra oraz zestawu współpracujących ze sobą procesów systemowych. Samo mikrojądro realizuje jedynie fundamentalne zadania, takie jak zarządzanie wątkami oraz obsługa komunikacji międzyprocesowej [1]. Wszystkie pozostałe usługi, wliczając w to sterowniki urządzeń i systemy plików, są odseparowane od jądra i funkcjonują jako niezależne programy w przestrzeni użytkownika.

Główną zaletą takiego podejścia jest wysoka stabilność środowiska - awaria oprogramowania lub sterownika nie narusza integralności samego jądra. System potrafi zrestartować uszkodzony proces bez zaburzania pracy całego systemu. Wadą tego rozwiązania jest jednak niższa wydajność, wynikająca z narzutu czasowego na ciągłą konieczność komunikacji międzyprocesowej oraz częste przełączanie kontekstu procesora.

2.5. Studium przypadku systemu operacyjnego Linux

System operacyjny Linux wywodzi się z tradycji architektonicznej rodziny systemów "Unix", czerpiąc z niej kluczowe założenia projektowe, takie jak wielozadaniowość, wielodostępność oraz podział przestrzeni pamięci [1]. Jego rozwój rozpoczął się w 1991 roku z inicjatywy Linusa Torvaldsa, a dziś stanowi on jeden z najważniejszych i najczęściej wykorzystywanych projektów tworzonych w modelu otwartego oprogramowania (ang. *open source*) [1]. Dzięki wsparciu globalnej społeczności Linux stał się najczęściej wykorzystywanym systemem komputerowym w wielu zastosowaniach, począwszy od infrastruktury serwerowej przez systemy wbudowane, aż po chmury obliczeniowe.

Z perspektywy użytkownika klasycznym i wysoce elastycznym sposobem komunikacji z systemem Linux jest powłoka (ang. *shell*), w systemach Linux reprezentowana najczęściej przez program *bash* [1]. Z technicznego punktu widzenia jest to zwykły program działający w przestrzeni użytkownika, który pełni rolę interpretera poleceń [1]. Powłoka pozwala na uruchamianie innych programów wraz z odpowiednimi parametrami. Do jej kluczowych funkcjonalności należy elastyczne przekierowywanie standardowych strumieni wejścia i wyjścia oraz łączenie pojedynczych programów w potoki (ang. *pipelines*), co pozwala na przekazywanie wyników między różnymi procesami [1]. Ponadto powłoka umożliwia asynchroniczne uruchamianie zadań w tle oraz tworzenie skryptów, które dzięki obsłudze zmiennych i instrukcji sterujących (np. pętli czy instrukcji warunkowych) pozwalają na automatyzację złożonych operacji systemowych [2].

Kluczowym elementem systemu jest jądro o strukturze monolitycznej [4], które odpowiada

za bezpośrednią kontrolę nad najważniejszymi zasobami systemu. Z uwagi na tę architekturę oraz konieczność zapewnienia bezpieczeństwa, aplikacje działające w przestrzeni użytkownika nie mają bezpośredniego dostępu do krytycznych zasobów systemu. Wszelka interakcja z zasobami zarządzanymi przez system musi odbywać się za pośrednictwem zdefiniowanego interfejsu wywołań systemowych, który w systemie Linux jest zgodny ze standardem **POSIX** [1]. Każde takie wywołanie wymusza sprzętowe przełączenie procesora do trybu jądra. Operacja ta wymaga zapisania stanu rejestrów oraz zmiany kontekstu wykonania, co wiąże się z dodatkowym narzutem czasowym.

2.5.1. Procesy, wątki w systemie Linux

W przeciwieństwie do wielu innych systemów operacyjnych, Linux nie wprowadza ścisłego, wewnętrznego rozróżnienia między procesami, a wątkami [1]. Każdy kontekst wykonania, niezależnie od tego, czy jest to jednowątkowy proces, czy jeden z wielu wątków wewnątrz procesu, jest reprezentowany w jądrze jako zadanie (ang. *task*) za pomocą struktury danych `task_struct` [4].

Klasyczne tworzenie nowego procesu odbywa się poprzez wywołanie systemowe `fork`. Aby uniknąć kosztownego i nieefektywnego kopiowania całej pamięci, Linux stosuje optymalizację kopiowania przy zapisie (ang. *copy on write*) [4]. Początkowo proces potomny otrzymuje własne tablice stron, jednak wskazują one na fizyczne strony pamięci procesu macierzystego z atrybutem tylko do odczytu. Nowa kopia strony jest alokowana w pamięci RAM dopiero w momencie, gdy któryś z procesów spróbuje dokonać na niej operacji zapisu [1].

Z kolei fundamentalną różnicą w architekturze systemu Linux, w porównaniu do klasycznych systemów UNIX, jest obsługa wątków. Wprowadzono w nim wywołanie systemowe `clone`, które zaciera tradycyjną różnicę między procesami a wątkami [1], umożliwiając tworzenie nowego wątku z bardzo podobnymi cechami do klasycznego procesu.

Pozwala ono zminimalizować narzut systemowy poprzez precyzyjne sterowanie tym, co jest kopiowane, a co współdzielone. Wywołanie to przyjmuje jako argument mapę bitową, w której poszczególne flagi (np. `CLONE_FILES`, `CLONE_SIGHAND`) decydują o współdzieleniu deskryptorów plików czy tablicy obsługi sygnałów [1].

2.5.2. Szeregowanie i synchronizacja w systemie Linux

System operacyjny Linux posiada przemyślany i wysoce elastyczny mechanizm szeregowania zadań oraz synchronizacji, który obsługuje zarówno standardowe programy użytkowe, jak i zadania czasu rzeczywistego [1].

W kontekście planowania czasu procesora, jądro wyróżnia trzy główne klasy wątków: zadania czasu rzeczywistego typu FIFO, zadania czasu rzeczywistego z rotacją (ang. *round-robin*) oraz zadania standardowe [1]. Wątki czasu rzeczywistego otrzymują zawsze najwyższe priorytety (w skali od 0 do 99). Mimo swojej nazwy nie dają one ścisłych gwarancji dotrzymania terminów (ang. *deadlines*), a jedynie zapewniają bezwzględne pierwszeństwo wykonania przed zwykłymi aplikacjami, [1]. Jest to bardzo istotna cecha systemu Linux, która pokazuje, że nie jest on odpowiedni (w klasycznej, nieprzerobionej wersji systemu) do bezpośredniego wykorzystania w twardych systemach czasu rzeczywistego [4]. Wątki typu FIFO działają aż do momentu dobrowolnego oddania procesora lub wyłączenia przez zadanie o wyższym priorytecie, podczas gdy wątkom szeregowanym cyklicznie przydzielane są dodatkowo określone kwanty czasu [1].

Z kolei standardowe zadania o niższych priorytetach (od 100 do 139) są z reguły obsługiwane przez algorytm *Completely Fair Scheduler* (CFS) [1], który optymalizuje i sprawiedliwie

rozdziela czas procesora z wykorzystaniem struktury drzewa czerwono-czarnego [2].

Uzupełnieniem efektywnego zarządzania zadaniami są zróżnicowane mechanizmy synchronizacji, które zastąpiły historyczną, nieefektywną globalną blokadę jądra. Aby zminimalizować ryzyko konfliktów o dostęp do zasobów, Linux udostępnia pełen przekrój rozwiązań: od operacji atomowych i barier pamięci, poprzez blokady wirujące (ang. *spinlocks*) [4] - optymalne, gdy wątek nie powinien być usypiany - aż po wysokopoziomowe muteksy i semaforey dla zadań mogących bezpiecznie przejść w stan oczekiwania [1].

2.5.3. Zarządzanie pamięcią

W systemie Linux pamięć jest stronicowana, w związku z czym podstawową jednostką zarządzania pamięcią fizyczną jest struktura o nazwie *page* [4]. Z uwagi na ograniczenia sprzętowe niektórych architektur, pamięć fizyczna została logicznie podzielona na strefy (ang. *zones*). Należą do nich między innymi *ZONE_DMA*, przeznaczona na operacje bezpośredniego dostępu do pamięci, *ZONE_NORMAL*, zawierająca standardowe strony, oraz *ZONE_HIGHMEM* [1].

Każdy proces uruchomiony w systemie operuje na własnej wirtualnej przestrzeni adresowej, która logicznie dzieli się na trzy główne segmenty: kod, dane oraz stos [1]. Segment kodu przechowuje instrukcje programu i domyślnie posiada atrybut umożliwiający jedynie jego odczyt, co pozwala na bezpieczne współdzielenie go między wieloma jednoczesnymi uruchomieniami tego samego programu [2]. Segment danych przechowuje zmienne globalne i statyczne, przy czym dla zmiennych niezainicjowanych Linux stosuje optymalizację: mapuje je na jedną fizyczną, wyzerowaną stronę, dopóki proces nie spróbuje dokonać modyfikacji, co wyzwala mechanizm kopiowania przy zapisie (ang. *copy-on-write*) [1]. Ponadto jądro oferuje mechanizm mapowania plików w pamięci, co umożliwia procesom bezpośredni dostęp do danych plikowych lub współdzielonych bibliotek z pominięciem standardowych wywołań systemowych [1].

Do zarządzania przydziałem fizycznych stron pamięci Linux wykorzystuje **algorytm bliźniaków** (ang. *buddy algorithm*) [1]. Choć mechanizm ten jest w większości przypadków wysoce wydajny, może prowadzić do zjawiska fragmentacji wewnętrznej w przypadku konieczności alokacji struktur o rozmiarach znacznie mniejszych niż domyślne wielkości strony. Aby zminimalizować ten problem i zredukować narzut systemowy, system wdraża mechanizm dyspozytora płytowego (ang. *slab allocator*) [4], który dysponuje pamięcią poprzez algorytm bliźniaków, aby później podzielić stronę na mniejsze fragmenty i umożliwić jej zarządzanie [1].

Na poziomie interfejsu programistycznego jądra, alokacja pamięci odbywa się za pomocą kilku wyspecjalizowanych funkcji do zarządzania pamięcią. Warto tutaj wspomnieć o tych najpowszechniejszych, czyli *kmalloc*, gwarantująca przydział obszaru ciągłego zarówno w przestrzeni wirtualnej, jak i fizycznej [4] oraz *vmalloc* wykorzystywana do alokacji większych struktur, niewymagających ciągłości fizycznej. Ta druga jednak wiąże się z większym narzutem wydajnościowym wynikającym z konieczności modyfikacji tablic stron [4].

3. ALGORYTM PRZETWARZANIA DANYCH I TECHNIKI OPTIMALIZACJI PROGRAMÓW KOMPUTEROWYCH

Przy konstrukcji systemu przetwarzania danych szczególne znaczenie ma dobór odpowiedniego algorytmu przetwarzania oraz sposobu jego implementacji. Jest to zasadniczo główne zadanie na początku pracy, ponieważ to właśnie algorytm przetwarzania najczęściej pozwala rozwiązać zadany systemowi problem. Dopiero po podjęciu tej decyzji architekt systemu może koncentrować się na wyborze platformy programowej czy sprzętowej, które będą miały wpływ na wydajność systemu wbudowanego. Należy jednak pamiętać, że ten sam algorytm, zapisany z wykorzystaniem tego samego języka programowania, może działać z różną szybkością w zależności od tego, jak został przetłumaczony na kod maszynowy konkretnego procesora.

Na potrzeby niniejszej pracy jako przedmiot badań wybrano **szybką transformatę Fouriera (skrót: FFT)**. Algorytm ten stanowi podstawowe narzędzie cyfrowej analizy sygnałów, a jego znaczenie dla rozwoju techniki obliczeniowej trudno przecenić — w zestawieniu opublikowanym w pozycji [5] umieszczono go wśród dziesięciu algorytmów o największym wpływie na naukę i inżynierię w XX wieku. O jego wyborze zadecydowała również charakterystyka obliczeniowa: FFT wykonuje wielokrotnie te same, proste operacje arytmetyczne na dużych zbiorach danych, przez co jest wdzięcznym obiektem badań nad wpływem optymalizacji na czas wykonania programu.

Każdy program komputerowy, również implementacja FFT, składa się nie tylko z idei algorytmu, tak zrozumiałej człowiekowi, ale też z niskopoziomowych operacji na sprzęcie, które są bezpośrednio zapisane w kodzie maszynowym, powstającym w wyniku przekształcenia programu użytkownika. Translację tę wykonują translatory języka programowania, a w przypadku niniejszej pracy, ze względu na użycie **języka programowania C** – kompilator. To, jakie instrukcje maszynowe znajdują się w programie wynikowym, zależy nie tylko od zapisu w kodzie źródłowym, ale również od zastosowanych narzędzi oraz przekazanych im poleceń. Zrozumienie tych mechanizmów jest niezbędne, aby świadomie wpływać na wydajność tworzonego oprogramowania.

Celem niniejszego rozdziału jest przedstawienie wiedzy z dwóch obszarów. Pierwsza część zawiera opis matematycznych podstaw dyskretnej transformaty Fouriera oraz analizę algorytmu FFT, wraz z omówieniem jego kolejnych etapów i wariantów. Druga część poświęcona jest procesowi kompilacji: przedstawiono w niej narzędzia biorące udział w tworzeniu programu wykonywalnego, wewnętrzną budowę kompilatora oraz przegląd rozwiązań najczęściej stosowanych w systemach opartych na jądrze Linux. Rozdział zamyka omówienie technik optymalizacji kodu wraz z flagami kompilacji, które pozwalają programiście na używanie tych technik.

3.1. Opis i analiza Szybkiej Transformaty Fouriera

3.1.1. Podstawy matematyczne algorytmu FFT

Przekształcenie Fouriera (ang. *Fourier Transform*) dla sygnałów ciągłych służy do wyznaczania widma częstotliwościowego $X(j\omega)$ sygnału czasu ciągłego $x(t)$ [6]. Wzory przekształcenia prostego i odwrotnego definiuje się następująco [6]:

$$X(j\omega) = \int_{-\infty}^{\infty} x(t) e^{-j\omega t} dt, \quad x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(j\omega) e^{j\omega t} d\omega \quad (3.1)$$

gdzie $\omega = 2\pi f$ oznacza pulsację, a f częstotliwość wyrażoną w hercach.

Aby całka określająca $X(j\omega)$ była zbieżna, sygnał $x(t)$ musi spełniać tzw. **warunki Dirichleta**: być bezwzględnie całkowny, mieć skończoną liczbę ekstremów oraz skończoną liczbę punktów nieciągłości w dowolnym skończonym przedziale [6].

Matematyczne podstawy ciągłej transformaty Fouriera są jednak nie do końca przydatne w świecie cyfrowego przetwarzania sygnałów, właśnie z uwagi na fakt, że sygnały, na których pracują maszyny cyfrowe, są dyskretne, a nie ciągłe. W tym celu definiuje się dyskretną transformatę Fouriera, która powstaje w wyniku dyskretyzacji szeregu Fouriera dla sygnału okresowego o okresie N [6].

W systemach cyfrowych przetwarzany sygnał dostępny jest w postaci skończonego ciągu próbek $x(n)$, $n = 0, 1, \dots, N - 1$, pobranych z częstotliwością próbkowania f_{pr} . Powyższy wzór na ciągłą transformatę Fouriera nie da się więc bezpośrednio zastosować. Zamiast niego widmo sygnału wyznacza się za pomocą **dyskretnej transformacji Fouriera** (DFT, ang. *Discrete Fourier Transform*). Matematyczne przekształcenia dla prostej i odwrotnej DFT definiuje się następująco [6]:

$$X(k) = \frac{1}{N} \sum_{n=0}^{N-1} x(n) e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \dots, N - 1 \quad (3.2)$$

$$x(n) = \sum_{k=0}^{N-1} X(k) e^{j\frac{2\pi}{N}kn}, \quad n = 0, 1, \dots, N - 1 \quad (3.3)$$

gdzie k oznacza numer składowej widma, dla której wyliczana jest aktualnie wartość transformaty. Kolejnym składowym odpowiada pulsacja $\Omega_k = 2\pi k/N$.

Wygodnym uproszczeniem powyższych zapisów jest wprowadzenie tzw. **czynnika obrotu** (ang. *twiddle factor*) [7]:

$$W_N = e^{-j\frac{2\pi}{N}} \quad (3.4)$$

co pozwala przedstawić parę wzorów DFT w zwartej postaci [6]:

$$X(k) = \frac{1}{N} \sum_{n=0}^{N-1} x(n) W_N^{kn}, \quad k = 0, 1, \dots, N - 1 \quad (3.5)$$

$$x(n) = \sum_{k=0}^{N-1} X(k) W_N^{-kn}, \quad n = 0, 1, \dots, N - 1 \quad (3.6)$$

W inżynierii oprogramowania do porównywania wydajności algorytmów powszechnie wykorzystuje się **notację dużego O** (ang. *Big O notation*), opisującą tempo wzrostu liczby wykonywanych operacji wraz ze długości wektora przetwarzanych danych wejściowych N . Zapis $O(g(N))$ oznacza, że liczba wykonywanych operacji rośnie nie szybciej niż funkcja $g(N)$ pomnożona przez pewną stałą, dla odpowiednio dużych N [7].

Innymi słowy, notacja ta pomija czynniki stałe i wyrazy niższego rzędu, skupiając się na tym, jak rośnie liczba operacji wraz ze wzrostem rozmiaru danych wejściowych [7].

Dla przykładu, bezpośrednie obliczenie wzoru (3.2) dla każdego z N prążków widma wymaga N mnożeń zespolonych, co dla całego wektora $X(k)$ wymaga wykonania zasadniczo N^2 mnożeń. W takim wypadku złożoność algorytmu DFT można określić jako $O(N^2)$. Przy ciągłym przetwarzaniu długich sygnałów złożoność ta stanowi istotne ograniczenie praktyczne. Jej redukcja jest możliwa dzięki zastosowaniu **algorytmu szybkiej transformacji Fouriera** (FFT, ang. *Fast Fourier Transform*), który pozwala znacznie obniżyć koszt obliczeniowy.

3.1.2. Szybka transformata Fouriera

Redukcja złożoności obliczeniowej DFT jest możliwa dzięki zastosowaniu algorytmów określanych mianem szybkich transformacji Fouriera (FFT). Można w tym wypadku mówić o pewnym zbiorze rozwiązań, bazujących na tym samym koncepcie. Po raz pierwszy algorytm ten zaproponowano w latach sześćdziesiątych XX wieku w pracy [8]. Rodzina algorytmów FFT pozwala na znaczące obniżenie złożoności obliczeniowej, przy zachowaniu takiej samej poprawności obliczeń jak w przypadku klasycznego algorytmu DFT. FFT jest więc wydajnym sposobem na obliczenie DFT — jego złożoność obliczeniowa spada do rzędu $O(N \log_2 N)$ [6].

W pracy Cooleya-Tukeya [8] zaproponowano nowe podejście do obliczania dyskretnej transformaty Fouriera, polegające na rozłożeniu długiego sygnału, o N próbkach, na kilka mniejszych grup. Następnie policzono transformatę osobno dla każdej z nich, by w ostatnim kroku połączyć wyniki w widmo całego sygnału.

Autorzy zauważyli również, że wybór N będącego potęgą liczby 2 lub 4 jest szczególnie wygodny dla komputerów działających na liczbach binarnych. Co więcej, cały algorytm można było wykonać na tej samej tablicy danych, bez potrzeby rezerwowania dodatkowej pamięci na wyniki pośrednie [8].

Zaproponowana metoda stała się inspiracją dla tworzenia kolejnych wariantów szybkiej transformaty Fouriera [6]:

- **Radix-2**,
- **radix-4**,
- **split-radix**,
- **algorytm Gooda-Thomasa**,
- **transformacja świergotowa (ang. chirp-Z)**.

Wszystkie realizują to samo przekształcenie, czyli dyskretną transformatę Fouriera, różnią się natomiast wewnętrznymi mechanizmami, takimi jak podejście do podziału danych czy długości pojedynczego bloku przetwarzanego sygnału [6]. Szczegółowy opis wariantów wykorzystanych w niniejszej pracy, wraz z ich implementacjami, przedstawiono w Sekcja 4.2.

Do dalszej analizy teoretycznej posłuży popularny wariant FFT oparty na **podstawie 2** (ang. *radix-2*), czyli podziale na dwa podzbiory, jako najprostszy do intuicyjnego wyjaśnienia. Na algorytm w tej postaci składa się kilka podstawowych zagadnień, omówionych kolejno w dalszej części podrozdziału: sposób podziału danych, uporządkowanie próbek przed rozpoczęciem obliczeń oraz elementarna operacja arytmetyczna, powtarzana na każdym etapie.

Podział wektora wejściowego może być przeprowadzony na dwa podstawowe sposoby. W wariacie z **decymacją w czasie** (DIT, ang. *Decimation in Time*) dzieli się próbki sygnału wejściowego na te o indeksach parzystych i nieparzystych, wykonuje DFT osobno na każdym zbiorze, a następnie odtwarza widmo całego sygnału z widm cząstkowych [6]. W wariacie z **decymacją w częstotliwości** (DIF, ang. *Decimation in Frequency*) postępuje się odwrotnie — dzieli się nie próbki wejściowe, lecz prążki widma wyjściowego [6].

Konsekwencją podziału jest to, że dane trafiają do obliczeń w innej kolejności niż ta pierwotna w sygnale wejściowym, więc na którymś etapie algorytmu należy je uporządkować. W algorytmach DIT robi się to na wejściu, przed właściwymi obliczeniami, i wynik otrzymuje się w kolej-

ności naturalnej [9]. W DIF jest odwrotnie — porządkowany jest dopiero wynik, na co zwrócono uwagę już w oryginalnej pracy [8].

Samo przestawianie można wykonać na kilka różnych sposobów, z czego podstawowy polega na wielokrotnym dzieleniu próbek na parzyste i nieparzyste. Jest to jednak rozwiązanie nieefektywne, ponieważ dane sortowane są wielokrotnie [6]. Okazuje się, że docelową pozycję każdej próbki da się wyznaczyć z użyciem operacji **permutacji bitowo-odwróconej** (ang. *bit-reversal permutation*) [7]. Metoda ta ustawia w odwrotnej kolejności bity indeksu próbki wektora wejściowego [6], co pozwala dość niskim kosztem obliczeniowym poukładać wektor wejściowy w kolejności wymaganej przez algorytm DIT. Warto dodać, że wiele procesorów sygnałowych udostępnia sprzętowy mechanizm adresowania z odwróceniem bitów, co dodatkowo przyspiesza ten etap [6].

Wynik tej operacji dla sygnału o długości $N = 16$ zestawiono w tab. 3.1 [6]. Warto zauważyć, że w celu ponownego ustawienia próbek w pierwotnej kolejności wystarczy raz jeszcze wykonać tę samą operację. W prezentowanej tabeli jako n oznaczono indeks początkowy, a jako n' indeks po operacji bit-reversal.

Tabela 3.1: Permutacja bitowo-odwrócona indeksów próbek dla $N = 16$

n	zapis dwójkowy	bity odwrócone	n'	n	zapis dwójkowy	bity odwrócone	n'
0	0000	0000	0	8	1000	0001	1
1	0001	1000	8	9	1001	1001	9
2	0010	0100	4	10	1010	0101	5
3	0011	1100	12	11	1011	1101	13
4	0100	0010	2	12	1100	0011	3
5	0101	1010	10	13	1101	1011	11
6	0110	0110	6	14	1110	0111	7
7	0111	1110	14	15	1111	1111	15

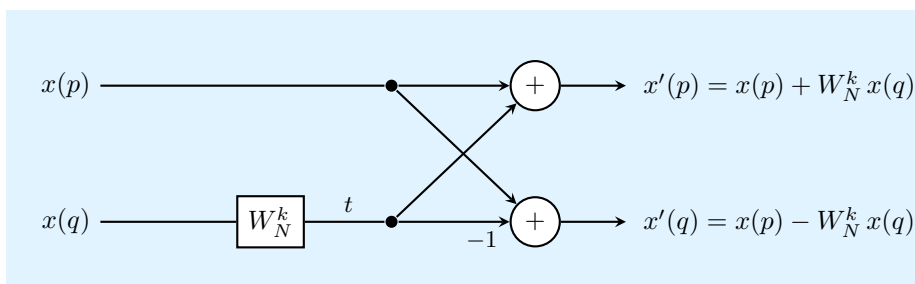
Niezależnie od przyjętego wariantu uporządkowania danych, kolejną elementarną operacją wykonywaną na każdym etapie algorytmu FFT, bazujących na założeniach zaprezentowanych w [8], jest tak zwana **operacja motylkowa** (ang. *butterfly*) [6]. Operacja ta pobiera parę liczb zespolonych $x(p)$ oraz $x(q)$, mnoży drugą z nich przez odpowiedni czynnik obrotu (3.4), a następnie wyznacza sumę i różnicę otrzymanych w ten sposób wartości [7]:

$$x'(p) = x(p) + W_N^k x(q), \quad x'(q) = x(p) - W_N^k x(q) \quad (3.7)$$

gdzie p i q oznaczają pozycje dwóch przetwarzanych elementów w tablicy danych, oddalone od siebie o połowę długości bieżącego etapu, a wykładnik k — numer operacji motylkowej w obrębie tego etapu.

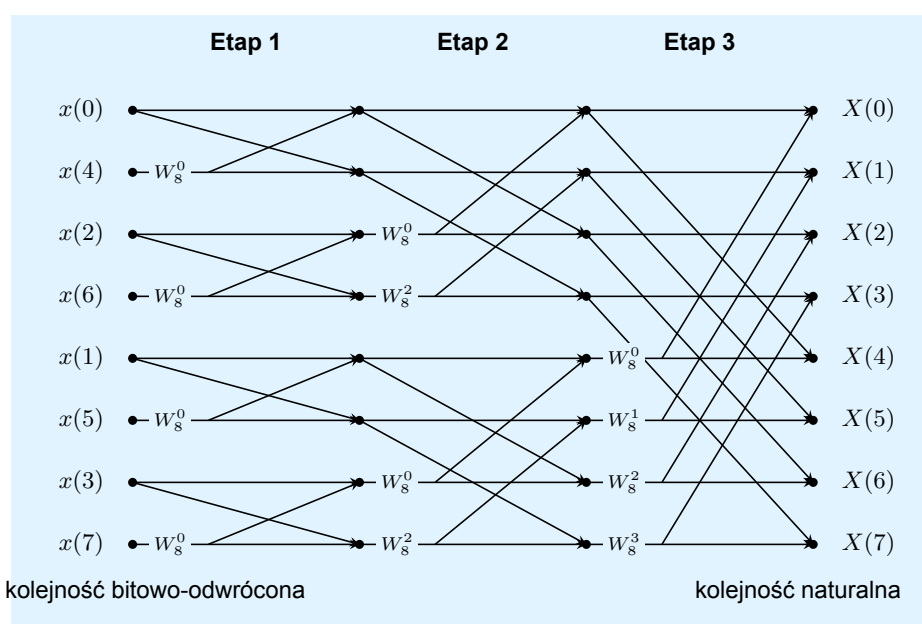
Istotną cechą tak zapisanej operacji jest to, że obie wartości wynikowe mogą zostać zapisane w tych samych komórkach pamięci, z których pobrano argumenty. Dzięki temu cały algorytm można zrealizować bez rezerwowania dodatkowej tablicy na wyniki pośrednie, co pozwala na zaoszczędzenie zasobów sprzętowych [8]. Graficzną reprezentację przepływu sygnałów w operacji motylkowej przedstawiono na rys. 3.1.

Zależności pomiędzy kolejnymi etapami algorytmu wygodnie jest przedstawić w postaci grafu przepływu sygnałów. Na rys. 3.2 zobrazowano pełny przebieg obliczeń dla sygnału o dłu-



Rysunek 3.1: Operacja motylkowa (ang. *butterfly*) algorytmu FFT o podstawie 2, opracowanie własne na podstawie [6]

gości $N = 8$, a więc dla $\log_2 8 = 3$ etapów, z których każdy składa się z $N/2 = 4$ operacji motylkowych.



Rysunek 3.2: Graf przepływu sygnałów algorytmu FFT o podstawie 2 z decymacją w czasie dla $N = 8$; każde skrzyżowanie linii odpowiada operacji motylkowej przedstawionej na rys. 3.1, opracowanie własne na podstawie [6, 9]

Graf ten dobrze ilustruje trzy własności algorytmu, istotne z punktu widzenia jego późniejszej implementacji. Po pierwsze, liczba etapów rośnie logarytmicznie wraz z długością przetwarzanego sygnału, a na każdym z nich wykonywanych jest $N/2$ operacji motylkowych, co daje łącznie $\frac{N}{2} \log_2 N$ mnożeń zespolonych i stanowi bezpośrednie źródło złożoności $O(N \log_2 N)$. Po drugie, próbki wejściowe podawane są w kolejności bitowo-odwróconej, dzięki czemu prążki widma otrzymuje się w kolejności naturalnej. Po trzeciej, odległość w pamięci pomiędzy argumentami pojedynczej operacji motylkowej podwaja się z każdym kolejnym etapem — w pierwszym etapie łączone są próbki bezpośrednio sąsiadujące, natomiast w ostatnim oddalone od siebie o $N/2$ pozycji. Ta ostatnia cecha ma bezpośredni wpływ na skuteczność wykorzystania pamięci podręcznej procesora, co szerzej omówiono w Sekcja 3.4.

Operacja bit-reversal jest iteracyjnym sposobem na odpowiednie uporządkowanie tablicy wejściowej przed właściwym przetwarzaniem algorytmu FFT. Sam algorytm można jednak zapisać również z wykorzystaniem rekurencji. Oba warianty, zarówno iteracyjny jak i rekurencyjny są

spotykane praktycznie [9]. Rekurencja dobrze oddaje ideę podziału na coraz mniejsze podproblemy, natomiast w implementacjach nastawionych na wydajność (oraz w tych wykonywanych w systemach o ograniczonej pamięci) częściej stosuje się wariant iteracyjny, w którym wystarczy pętla po kolejnych etapach [9].

Przedstawione powyżej koncepcje stosowane w algorytmie FFT składają się łącznie na kompletny algorytm. Jego minimalną postać, w wariancie radix-2 z decymacją w czasie, przedstawiono na alg. 1 [7].

Algorytm 1 Algorytm FFT o podstawie 2 z decymacją w czasie, na podstawie [6, 7]

Dane wejściowe: wektor próbek x o długości N , gdzie $N = 2^p$

Wynik: wektor x zawierający kolejne prążki widma

```

1: funkcja FFT( $x, N$ )
2:   Operacja BitReversal( $x, N$ )
3:   for  $s = 1, 2, \dots, \log_2 N$  do                                     ▷ kolejne etapy
4:      $m \leftarrow 2^s$                                                  ▷ długość widma składanego w tym etapie
5:     for  $i = 0, m, 2m, \dots, N - m$  do                               ▷ kolejne bloki
6:       for  $j = 0, 1, \dots, m/2 - 1$  do                               ▷ kolejne motylki
7:         wykonaj operację motylkową (3.7)
           na próbkach  $x(i + j)$  oraz  $x(i + j + m/2)$ 
           z czynnikiem obrotu  $W_m^j$ 
8:       end for
9:     end for
10:  end for
11: end funkcja

```

3.2. Opis procesu kompilacji programu komputerowego

3.2.1. Procesory języka programowania

W najnowszych rozwiązaniach dotyczących systemów komputerowych użytkownicy mają szeroki wybór języków programowania, które służą do opisywania idei projektowych człowieka w taki sposób, aby były one możliwe do wykonania przez maszynę. Sprzęt komputerowy nie jest jednak w stanie operować tak wysokopoziomowymi abstrakcjami, jak zmienne, pętle, instrukcje warunkowe itd., które są dostępne w językach programowania. W ich miejsce musi on mieć przygotowane binarne instrukcje maszynowe, które pozbawione są wysokopoziomowej logiki, tak łatwej do zrozumienia dla człowieka. Można zatem powiedzieć, że zanim jakkolwiek program będzie mógł zostać uruchomiony, musi najpierw zostać przetłumaczony na formę, w której będzie mógł być wykonany przez jednostkę centralną komputera. Oprogramowanie systemowe realizujące ten proces translacji określa się ogólnie mianem **procesorów języka** (ang. *language processors*) [10].

Jednym z najpopularniejszych procesorów języka jest **kompilator**, który można w ogólności zdefiniować jako program komputerowy, którego zadaniem jest przetłumaczenie kodu napisanego w jednym języku (języku źródłowym) na równoważny program w innym języku (języku docelowym). Najczęściej sprowadza się to do translacji z wysokopoziomowego języka programowania na język maszynowy, który może być bezpośrednio wykonany przez procesor. Podejście to jest powszechnie stosowane dla języków takich jak **C, C++ czy Java**. Wymagają one uprzedniej kompilacji, ale w zamian oferują programiście ogromną kontrolę nad zarządzaniem pamięcią oraz zasobami sprzętowymi. Dzięki wysokiej wydajności wygenerowanego kodu języki te stanowią standard w wytwarzaniu systemów wbudowanych.

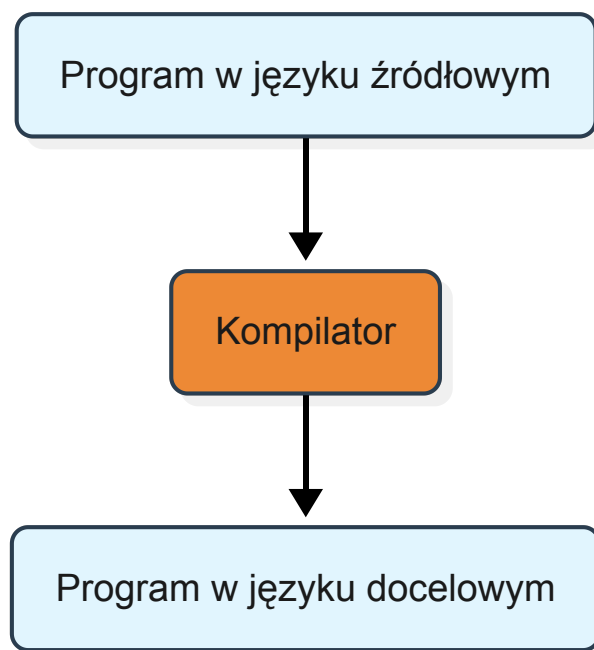
Zupełnie odmiennym mechanizmem charakteryzuje się inny program procesora języka, nazywany **interpreterem**. Rozwiązanie to zamiast generować docelowy plik wykonywalny, analizuje i realizuje instrukcje kodu źródłowego na bieżąco, biorąc pod uwagę aktualnie dostarczone dane wejściowe. Narzut obliczeniowy oraz pamięciowy związany z ciągłą przetwarzaniem instrukcji sprawia, że rozwiązania tego typu rzadko znajdują zastosowanie w systemach wbudowanych. Z tego względu dalsza część niniejszej pracy skupia się wyłącznie na architekturze i mechanizmach działania kompilatorów.

Dalsza analiza teoretyczna w niniejszej pracy zostanie oparta na kompilatorach języka C, ze względu na szerokie zastosowanie tego języka w systemach wbudowanych.

3.2.2. Szczegółowy opis procesu kompilacji

W zastosowaniach niezwiązanych z systemami wbudowanymi wielu użytkowników traktuje kompilator jako wysoką abstrakcję, stosując prosty model „czarnej skrzynki”, co obrazuje rys. 3.3. Zgodnie z tym podejściem narzędzie to zajmuje się wyłącznie tłumaczeniem, a programistę interesuje jedynie to, aby z dostarczonego kodu źródłowego otrzymać działający program docelowy. Przy takim zastosowaniu twórca oprogramowania nie musi poświęcać szczególnej uwagi sposobowi działania kompilatora, traktując go jedynie jako niezbędne narzędzie, pozwalające uruchomić jego program na konkretnym sprzęcie.

W systemach wbudowanych takie uproszczone podejście jest jednak często niewystarczające, ponieważ w takich zastosowaniach powszechnym zjawiskiem są silnie ograniczone zasoby sprzętowe, a nierzadko również ścisły rygor czasowy. Z tego względu programy tworzone przez inżyniera muszą być nie tylko poprawne pod kątem logicznym, ale również niezwykle rozsądnie gospodarować dostępnymi zasobami. Aby świadomie polepszać osiągi i kontrolować ten



Rysunek 3.3: Model "czarnej skrzynki" kompilatora

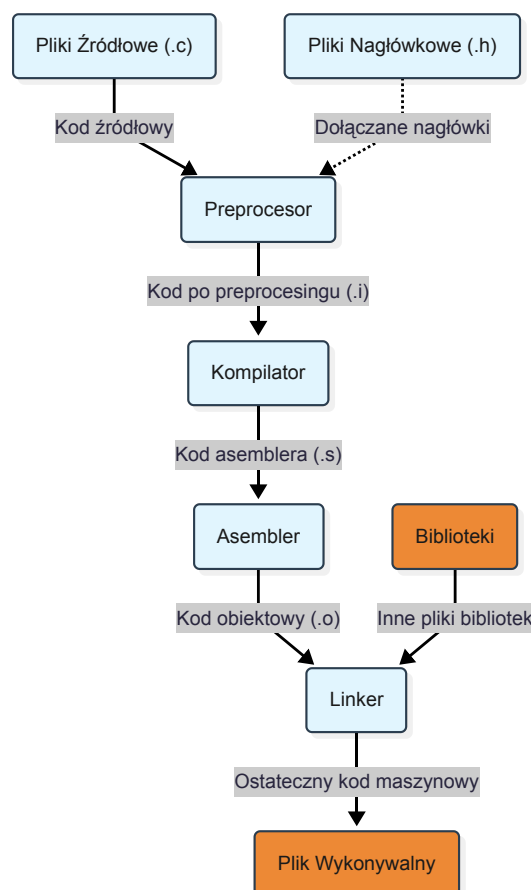
proces, konieczne jest porzucenie modelu czarnej skrzynki i zejście kilka poziomów abstrakcji niżej – począwszy od zrozumienia, jaki zestaw narzędzi jest w ogóle potrzebny do przetworzenia języka.

Na potrzeby niniejszej pracy zostanie przytoczony sposób postępowania opisany w książce [10], przedstawiony na schemacie rys. 3.4. Należy zaznaczyć, że zaprezentowany na rysunku pakiet oprogramowania, powszechnie określany jest przez użytkowników mianem kompilatora, choć w rzeczywistości zawiera w sobie zestaw wielu współpracujących programów komputerowych, w tym ten właściwy kompilator. Jak się okazuje, do utworzenia wykonywalnego programu docelowego wymagane jest najczęściej użycie kilku dodatkowych narzędzi, ułożonych w taki sposób, że wynik przetwarzania jednego jest wejściem dla kolejnego. W klasycznym podejściu proces tworzenia pliku wykonywalnego składa się z następujących faz [11]:

Preprocesor odpowiada za przygotowanie kodu źródłowego przed jego właściwą analizą. Jego początkowym zadaniem jest zebranie programu, który nierzadko bywa podzielony na oddzielne moduły plikowe. Następnie program ten wykonuje odpowiednie dyrektywy oraz rozwija zdefiniowane przez programistę skróty (makra) do postaci pełnoprawnych instrukcji języka źródłowego.

Kompilator przyjmuje zmodyfikowany przez preprocesor kod i dokonuje jego tłumaczenia. Zazwyczaj wynikiem jego pracy nie jest bezpośredni kod maszynowy, lecz program zapisany w języku asemblera, powiązany z docelową architekturą, na której uruchamiany ma zostać przygotowany program. Generowanie na tym etapie kodu asemblerowego wynika z faktu, iż jako język czytelny dla człowieka jest on znacznie łatwiejszy do wyprodukowania oraz ewentualnego debugowania.

Asembler zajmuje się przetworzeniem kodu asemblerowego na postać zrozumiałą dla maszyny. Wynikiem jego działania jest relokowalny kod maszynowy, zapisany w postaci tak zwanych plików obiektowych. Dodatkowo na tym etapie dla każdego z plików generowana jest tablica symboli, opisująca obiekty posiadające powiązania zewnętrzne [11].



Rysunek 3.4: Klasyczny przebieg systemu przetwarzania języka programowania [10]

Linker łączy wygenerowane pliki obiektowe oraz niezbędne biblioteki w jeden spójny plik wykonywalny. Jego kluczowym zadaniem jest rozwiązanie adresów pamięci dla odwołań zewnętrznych pomiędzy poszczególnymi modułami programu, do czego wykorzystuje wspomniane wcześniej tablice symboli. W procesie tym linker dołącza również kod niezbędnych funkcji pochodzących ze standardowych lub zewnętrznych bibliotek [11].

3.2.3. Wewnętrzna struktura kompilatora

Kompilator wewnętrznie dzieli się na dwie części: analizę (front end), która rozkłada program źródłowy na składowe elementy i na ich podstawie buduje reprezentację pośrednią oraz tablicę symboli, oraz syntezę (back end), która na tej podstawie konstruuje program docelowy [10].

Proces ten przebiega jako sekwencja faz, z których każda przekształca jedną reprezentację programu w inną, a tablica symboli jest wykorzystywana przez wszystkie z nich [10]:

- **Analiza leksykalna** - grupuje znaki programu źródłowego w leksemy i zamienia je na tokeny.
- **Analiza składniowa** - na podstawie tokenów buduje drzewo składniowe odzwierciedlające strukturę gramatyczną programu.
- **Analiza semantyczna** - sprawdza zgodność programu z definicją języka, w tym poprawność typów.
- **Generowanie kodu pośredniego** - tworzy niskopoziomową reprezentację programu, np. w postaci kodu trójadresowego.

- **Optymalizacja kodu** - opcjonalnie usprawnia kod pośredni, tak aby wynikowy kod docelowy był lepszy (np. szybszy lub mniejszy).
- **Generowanie kodu** - tłumaczy reprezentację pośrednią na kod maszyny docelowej, przydzielając m.in. rejestry zmiennym.

W praktycznych implementacjach kilka faz bywa łączonych w jeden przebieg (ang. *pass*) [10].

3.3. Przegląd najpopularniejszych kompilatorów języka C

Na rynku dostępnych jest wiele implementacji kompilatorów języka C, zarówno komercyjnych, jak i rozwijanych w modelu otwartoźródłowym. W praktyce jednak w środowisku systemów opartych na jądrze Linux najczęściej spotyka się dwa rozwiązania: **GNU Compiler Collection (GCC)**, powstały w ramach projektu GNU [12], oraz **Clang**, będący częścią projektu **LLVM**. Samo jądro systemu Linux przez długi okres było kompilowane wyłącznie narzędziami GNU, jednak w ostatnim czasie Clang i LLVM stały się dla nich dostępną alternatywą – kompilowane z ich wykorzystaniem jądra systemu Linux wykorzystują między innymi systemy Android czy ChromeOS [13].

Warto zaznaczyć, że również w systemach wbudowanych, niewykorzystujących systemów operacyjnych, narzędzia GNU czy Clang wraz z LLVM są często spotykane. Oferują one wsparcie dla wielu architektur procesorów, pozwalając na uruchomienie kompilatora na sprzęcie o jednej architekturze (nazywanej również terminem **maszyny hosta**) w celu wygenerowania kodu maszynowego przeznaczonego dla zupełnie innej, docelowej architektury (nazywanej **maszyną targetu**). Proces ten określany jest mianem **kompilacji skrośnej** (ang. *cross-compilation*) [10]. Obydwa wspomniane kompilatory posiadają zestaw narzędzi przeznaczonych do różnych architektur spotykanych w systemach wbudowanych, takich jak **ARM, RISC-V, AVR**.

3.3.1. Kompilator GCC

GCC powstał jako część projektu GNU [12] i jest tradycyjnym, domyślnym kompilatorem większości dystrybucji systemu Linux [4]. Jego architektura rozwiązań bazuje na klasycznym modelu kompilatora, który opisuje Sekcja 3.2.3. GCC dzieli proces tłumaczenia na część analizującą (front end) i część generującą kod wynikowy (back end). Część środkowa, odpowiedzialna za optymalizację, przekształca program w dwóch własnych, uproszczonych reprezentacjach – **GENERIC** oraz **GIMPLE** – na których wykonywane są kolejne przekształcenia mające na celu poprawienie kodu [14].

3.3.2. Kompilator Clang

Clang pełni rolę części analizującej (front end) dla części syntezy obsługiwanej przez **LLVM**. LLVM zapewnia zestaw narzędzi kompilatorowych, zaprojektowanego tak, aby umożliwić analizę i poprawianie kodu na każdym etapie jego translacji i uruchomienia [15]. Podobnie jak GCC, LLVM przekształca program do własnej, uproszczonej reprezentacji pośredniej, jednak w odróżnieniu od GCC reprezentacja ta została zaprojektowana w taki sposób, aby nie musieć zależeć od konkretnego języka programowania [15]. Dzięki takiej budowie LLVM można podzielić na trzy niezależne części: część analizującą (Clang), wspólną dla wszystkich architektur warstwę optymalizacji oraz część generującą kod maszynowy dla konkretnego procesora. Kod projektu LLVM, w tym Clang, jest udostępniany na licencji Apache License [16].

Oba kompilatory, mimo różnic w budowie wewnętrznej, są ze sobą często porównywane w literaturze pod kątem jakości i wydajności generowanego kodu, w tym w zastosowaniach wbudowanych systemów wbudowanych [17]. Dysponują one bowiem wbudowanymi mechanizmami zaawansowanej optymalizacji kodu. Podstawowym narzędziem do konfiguracji optymalizacji programów w języku C są **flagi kompilacji**, czyli ściśle określony zbiór poleceń określających pożądany stopień optymalizacji programu. Polecenia te używane są właśnie przy procesie translacji kodu źródłowego na kod wykonywalny. Podstawowe mechanizmy optymalizacji kodu, wraz z flagami kompilacji zostaną przedstawione w Sekcja 3.4

3.4. Zaawansowane techniki optymalizacji kodu i flagi kompilacji

Optymalizacja kodu w czasie procesu kompilacji jest to dążenie do takiego przekształcenia programu wynikowego, aby osiągał lepsze wyniki w pewnych metrykach, przy zachowaniu poprawności jego założonego działania [10]. Metrykami tymi są na przykład czas wykonania programu, zużycie zasobów pamięci, wielkość kodu programu, czy w przypadku bardzo dużych projektów czas kompilacji [10]. Warunek zachowania poprawności programu jest jednak nadrzędny — przekształcenie, które zmienia wynik obliczeń, nie jest optymalizacją, lecz błędem [10]. Kompilator nie analizuje przy tym programu na tyle głęboko, aby zastąpić zastosowany przez programistę algorytm innym, na przykład o mniejszej złożoności obliczeniowej; stosuje jedynie niskopoziomowe przekształcenia, polegające na przykład na przekształceniach algebraicznych lub na wykorzystaniu dostępnej architektury jednostki centralnej [10].

Termin "optymalizacja" jest w tym kontekście w pewnym stopniu mylący. Program kompilatora nie jest w stanie zagwarantować, że kod wygenerowany przez niego będzie najszybszym możliwym rozwiązaniem danego zadania [10]. W praktyce chodzi o zestaw przekształceń, które w większości przypadków poprawiają założone metryki dla wielu programów, a nie o rozwiązanie optymalne w sensie matematycznym.

Nie istnieje również jedno kryterium tego, co znaczy „lepszy” kod. Skrócenie czasu wykonania odbywa się zwykle kosztem zwiększenia rozmiaru programu, a przekształcenia poprawiające wydajność utrudniają jednocześnie pracę z debuggerem. W systemach wbudowanych kompromis ten nabiera szczególnego znaczenia, ponieważ ograniczona bywa zarówno moc obliczeniowa, jak i dostępna pamięć. Z tego względu kompilatory udostępniają nie jeden, lecz kilka poziomów optymalizacji, ukierunkowanych na różne cele — ich zestawienie przedstawiono w tab. 3.2.

Na przestrzeni lat zmieniły się także same kryteria tego, co w kodzie warto ograniczać. Przykładowo dawniej, gdy procesory sygnałowe wykonywały operacje mnożenia w większej ilości cykli zegara, opłacało się minimalizować ich liczbę nawet kosztem zwiększenia liczby pozostałych operacji, na przykład dodawania, które wymagało mniej cykli. Obecnie jednak procesory sygnałowe potrzebują na każdą z przytoczonych operacji w zasadzie tyle samo czasu, a przesyłanie danych pomiędzy rejestrami a pamięcią trwa, w większości przypadków, tyle samo co operacje arytmetyczne [6]. Pomimo tego, że w dzisiejszych czasach nie ma konieczności na zwracania uwagi na typ wykonywanych operacji, wciąż pozostaje dążenie do zmniejszania ilości ich wykonania.

3.4.1. Przekształcenia optymalizujące

Metody usuwania nadmiarowości

Programowanie w językach wysokiego poziomu ma zarówno wiele aspektów pozytywnych, takich jak używanie wysokopoziomowych abstrakcji, jak i negatywnych, takich jak wykonywanie zbędnych obliczeń. Właśnie z tej przyczyny spora część operacji w programach jest niepotrzebna i nie wynika z braku wiedzy inżyniera oprogramowania, a z narzutu języka. Takie operacje jak odwołania do elementów tablic czy pól struktur rozwijane są bowiem na etapie kompilacji do szeregu operacji arytmetycznych na komórkach pamięci. Operacje te bywają wspólne dla wielu odwołań, a programista nie ma możliwości ich samodzielnego wyeliminowania [10]. W dalszej części prezentowanej sekcji opisano kilka metod na eliminację niepotrzebnych operacji w czasie trwania programu.

Z **wspólnym podwyrażeniem** (ang. *common subexpression*) mamy do czynienia, gdy wynik operacji jest już znany (na przykład ta sama operacja z tymi samymi składnikami została już przeprowadzona), a jej zmienne się nie zmieniły. Kompilator ma możliwość przed wykonaniem programu uznać, że operacja da ten sam wynik co poprzednio i użyć wartości obliczonej za pierwszym razem, zamiast powtarzać obliczenia [10]. Pokrewną techniką jest **propagacja kopii** (ang. *copy propagation*), czyli zastąpienie zmiennej wartością, którą wcześniej jej przypisano [10].

Jeżeli kompilator zdoła ustalić, że wartość danej zmiennej jest stała, czyli nie zmienia się w czasie programu, może wyznaczyć ją już w czasie kompilacji i wstawić w miejsce odwołań do niej. Przekształcenie to nosi nazwę **zwijania stałych** (ang. *constant folding*) [10].

Przekształcenia pętli programów

Pętle, a w szczególności pętle zagnieżdżone, są dość newralgicznym punktem programów komputerowych, ze względu na możliwie długi czas ich wykonywania. Z tego względu opłaca się zmniejszać liczbę instrukcji wykonywanych w ich wnętrzu, nawet za cenę zwiększenia ilości kodu poza pętlą [10]. Na tej zasadzie opiera się **przenoszenie kodu** (ang. *code motion*), polegające na wyniesieniu przed pętlę tych obliczeń, których wynik nie zmienia się pomiędzy kolejnymi iteracjami.

Kolejnym przekształceniem jest **redukcja siły operacji** (ang. *strength reduction*), polegająca na zastąpieniu kosztownych instrukcji ich szybszymi odpowiednikami. Historycznie technikę tę wykorzystywano dość powszechnie do zamiany mnożenia na proste dodawanie wewnątrz pętli [10, 6]. Współczesne procesory posiadają rozbudowane układy mnożące, które są na tyle wydajne, że z tego konkretnego zastosowania się już korzysta. Redukcja siły wciąż pozostaje przydatna w przypadku powolnych operacji, na przykład dzielenia i modulo, które kompilator może zastępować szybkimi operacjami bitowymi.

Kolejną operacją jest **rozwijanie pętli** (ang. *loop unrolling*), polegające na umieszczeniu kilku kolejnych iteracji w jednym, powiększonej instrukcji pętli. Zabieg ten ogranicza narzut związany z obsługą instrukcji pętli, a jednocześnie zwiększa liczbę operacji wykonywanych w pojedynczym przebiegu, co daje kompilatorowi większe możliwości wykrycia operacji nadających się do równoległego wykonania [10]. Odbywa się to jednak kosztem zwiększenia rozmiaru kodu programu wynikowego.

Wywołania procedur i przydział rejestrów

Optymalizacja nie musi zawsze dotyczyć samych obliczeń. Może również polegać na redukcji kosztów ich obsługi. Jedną z technik realizującą takie przekształcenia jest **wciąganie procedur**

(ang. *procedure inlining*), czyli zastąpienie wywołania procedury jej ciałem [10]. Eliminuje ono narzut związany ze skokiem do innej części programu.

Kolejną taką techniką jest odpowiedni **przydział rejestrów** (ang. *register allocation*). Rejestry są najszybszą pamięcią dostępną w procesorze, jednak ich liczba jest ograniczona, przez co nie wszystkie wartości mogą być w nich przechowywane jednocześnie; pozostałe muszą zostać umieszczone w pamięci operacyjnej. Instrukcje operujące na rejestrach wymagają mniej cykli procesora do wykonania, niż te, które odwołują się do pamięci [10]. Zadanie kompilatora polega więc na takim doborze wartości przechowywanych w rejestrach, aby ograniczyć liczbę odwołań do pamięci — problem ten rozwiązuje się między innymi metodą kolorowania grafu [10].

3.4.2. Hierarchia pamięci i pamięć podręczna

Czas wykonania programu zależy nie tylko od liczby wykonanych instrukcji, ale również od czasu dostępu do danych, potrzebnych do ich wykonania. W związku z tym, że pamięć o dużym rozmiarze i bardzo krótkim czasie dostępu jest praktycznie niespotykana (lub bardzo kosztowna), komputery projektuje się z pamięcią w postaci hierarchii — od nielicznych, szybkich rejestrów procesora, przez pamięć podręczną, aż po wolniejszą pamięć operacyjną [10].

Skuteczność opisanej hierarchii wynika z naturalnego sposobu działania komputerów — programy rzadko odwołują się do pamięci w sposób losowy. Zamiast tego mają tendencję do używania tych samych lub sąsiadujących ze sobą danych. Właśnie dlatego sprzęt pobiera informacje do pamięci podręcznej od razu całymi fragmentami, zwanymi liniami [10]. Odczyt danych już obecnych w pamięci podręcznej, nazywany **trafieniem** (ang. *cache hit*), trwa zaledwie kilka cykli procesora, podczas gdy **chybienie** (ang. *cache miss*) wymaga sięgnięcia do wolniejszej pamięci operacyjnej i trwa nawet kilkadziesiąt razy dłużej [10]. Z tego powodu sposób, w jaki algorytm odwołuje się do kolejnych komórek tablicy, wpływa na czas jego wykonania niezależnie od liczby wykonywanych operacji arytmetycznych.

3.4.3. Podstawowe flagi optymalizacyjne

Opisane wcześniej przekształcenia nie są stosowane przez kompilator samoczynnie. Decyzja, które z nich zostaną wykonane, należy do twórcy oprogramowania. Do realizacji tych technik służą odpowiednie flagi kompilacji. Najczęściej wykorzystuje się w tym celu zbiorcze poziomy optymalizacji, oznaczane jako `-O0`, `-O1`, `-O2`, `-O3`, `-Os`, `-Oz`, `-Og` oraz `-Ofast`. W tab. 3.2 zestawiono ogólne założenia stojące za każdym z tych poziomów, w postaci opisanej w dokumentacjach kompilatorów clang i gcc.

Warto podkreślić, że pod każdym z nich kryje się lista pojedynczych flag, odpowiadających konkretnym przekształceniom optymalizującym program wykonywalny. Flagi pozwalają wybrać zarówno najważniejsze przekształcenia opisane w Sekcja 3.4, jak i inne techniki opisane szerzej w literaturze [10].

Dla przykładu poziom `-O1` włącza w kompilatorze GCC kilkadziesiąt takich flag, między innymi `-ftree-dce` odpowiadającą za eliminację martwego kodu, `-ftree-copy-prop` realizującą propagację kopii czy `-fmove-loop-invariants` przenoszącą niezmienniki poza pętlę [18]. Kolejne poziomy dokładają do tej listy następne przekształcenia. Pełne wykazy flag włączanych przez poszczególne poziomy dostępne są w dokumentacjach kompilatorów [18, 19]. Dokładny zestaw przekształceń włączanych na danym poziomie można również sprawdzić, kompilując program z opcją `-Q --help=optimizers` [18]. Poszczególne flagi można również włączać i wyłączać pojedynczo, niezależnie od wybranego poziomu.

Flagi związane z architekturą procesora

Osobną grupę stanowią flagi określające docelową architekturę sprzętową. Mają one szczególne znaczenie przy kompilacji skrośnej, ponieważ kompilator uruchamiany jest wówczas na maszynie o innej architekturze niż docelowa i nie ma jak ustalić możliwości procesora, na którym program będzie wykonywany.

Współczesne procesory udostępniają między innymi możliwości przetwarzania typu **SIMD** (ang. *Single Instruction, Multiple Data*), pozwalające wykonać jedną operację równocześnie na wielu wartościach umieszczonych w pojedynczym, szerokim rejestrze [20]. Rozwiązanie to sprawdza się zwłaszcza tam, gdzie te same, proste obliczenia powtarzane są wielokrotnie na dużych zbiorach danych, a więc również w algorytmach cyfrowego przetwarzania sygnałów [20]. W procesorach ARM moduł sprzętowy realizujący przetwarzanie SIMD nosi nazwę **NEON** i został szerzej omówiony w Sekcja 4.1.1.

Architekturę wskazuje się opcjami `-march` oraz `-mcpu` [20], a do uruchomienia jednostki NEON konieczne jest dodatkowo podanie opcji `-mfpu=neon`, sygnalizującej dostępność tego mechanizmu [20]. Pominięcie opcji `-mcpu` sprawia, że kompilator przyjmuje wbudowany rdzeń domyślny, przez co wygenerowany kod może działać wolniej [20].

Za samo przekształcenie kodu w tę postać, nazywane **wektoryzacją** (ang. *vectorization*), odpowiada w kompilatorze GCC flaga `-ftree-vectorize`, włączana domyślnie na poziomie `-O2` [18]. Bez poprawnie wskazanej architektury docelowej nie zostanie ona jednak wykonana, mimo wybrania wysokiego poziomu optymalizacji.

Flagi wpływające na dokładność obliczeń

Ostatnią grupę stanowią flagi zmieniające sposób wykonywania działań zmiennoprzecinkowych. Najbardziej znaną z nich jest `-ffast-math`, włączana automatycznie przez poziom `-Ofast` i przez żaden inny [18]. Pozwala ona kompilatorowi między innymi na zmianę kolejności działań zmiennoprzecinkowych, co narusza wymagania standardów języków C i C++ i może prowadzić do otrzymania innego wyniku obliczeń [18].

Flagi tego typu mogą przyspieszyć wykonanie programu, jednak ich zastosowanie wymaga świadomej decyzji. W algorytmach, w których kolejność wykonywania działań wpływa na dokładność wyniku, uzyskane wartości mogą znacząco różnić się od poprawnych wartości [18].

Tabela 3.2: Poziomy optymalizacji w kompilatorach GCC i Clang, na podstawie oficjalnych dokumentacji [18, 19]

Flaga	GCC	Clang
-00	Ustawienie domyślne. Skraca czas kompilacji i ułatwia debugowanie, ponieważ zachowanie programu odpowiada bezpośrednio kodowi źródłowemu.	Brak optymalizacji. Najszybsza kompilacja i najlepiej debugowalny kod wynikowy.
-01 (-O)	Kompilator ogranicza rozmiar kodu i czas jego wykonania, unikając przekształceń wymagających dużego nakładu obliczeniowego. Poziom zalecany dla obszernego kodu generowanego maszynowo, jako kompromis między czasem kompilacji a zużyciem pamięci.	Poziom pośredni między -00 a -02.
-02	Włącza niemal wszystkie przekształcenia niewymagające kompromisu między rozmiarem a szybkością kodu. Wydłuża czas kompilacji względem -01.	Umiarkowany poziom optymalizacji, włączający większość dostępnych przekształceń.
-03	Włącza wszystkie przekształcenia z -02 oraz dodatkowe, dotyczące głównie pętli – między innymi zamianę kolejności pętli zagnieżdżonych, ich dzielenie oraz częściowe rozwijanie.	Odpowiednik -02 rozszerzony o przekształcenia wymagające dłuższego czasu kompilacji lub zwiększające rozmiar kodu w zamian za szybsze wykonanie.
-Os	Bazuje na -02, wyłączając przekształcenia zwykle zwiększające rozmiar kodu, m.in. wyrównanie funkcji i pętli. Dodatkowo włącza wciąganie procedur oraz przekształcenia ukierunkowane na redukcję rozmiaru.	Odpowiednik -02 z dodatkowymi przekształceniami zmniejszającymi rozmiar kodu wynikowego.
-0z	Agresywna redukcja rozmiaru kodu, zbliżona do -0s. Może zwiększyć liczbę wykonywanych instrukcji, jeśli zajmują one mniej bajtów w pamięci.	Odpowiednik -0s, lecz z dalszą redukcją rozmiaru kodu wynikowego.
-0g	Bazuje na -01, wyłączając przekształcenia najbardziej utrudniające debugowanie – między innymi przenoszenie niezmienników poza pętlę oraz eliminację martwych zapisów.	Zbliżony do -01. Część przekształceń jest wyłączona na rzecz lepszej widoczności zmiennych podczas debugowania.
-Ofast	Włącza wszystkie przekształcenia z -03 oraz dodatkowe, mogące naruszać zgodność ze standardem języka (m.in. -ffast-math).	Włącza wszystkie przekształcenia z -03 wraz z dodatkowymi, agresywnymi optymalizacjami naruszającymi zgodność ze standardem języka. Od wersji 19 Clang oznacza tę flagę jako przestarzałą, ze względu na nieprzewidywalny wpływ na zachowanie poprawnego kodu.

4. PROJEKT SYSTEMU TESTOWEGO I METODYKA PRZEPROWADZONYCH BADAŃ

Stosowany na potrzeby niniejszej pracy system pomiarowy składa się z dwóch warstw: fizycznej platformy sprzętowej oraz oprogramowania na nim uruchomianego. Warstwy te są ze sobą ściśle powiązane, dobór sprzętu determinuje dostępne mechanizmy przyspieszania obliczeń, takie jak wielordzeniowość czy jednostka SIMD, natomiast to, czy te mechanizmy zostaną faktycznie wykorzystane, zależy od zaimplementowanego algorytmu oraz od sposobu, w jaki zostanie skompilowany.

Warstwę sprzętową stanowi płytki Raspberry Pi 4 Model B, oparta na czterech rdzeniach ARM Cortex-A72. Warstwę programową tworzą natomiast cztery niezbędne elementy. Pierwszym z nich jest system operacyjny Linux, pod którego nadzorem uruchamiany jest program testowy. Następne elementy są ściśle powiązane ze sobą a należą do nich zaimplementowane warianty algorytmu FFT, program pomiarowy, odpowiedzialny za wykonanie i zmierzenie pojedynczej transformaty oraz skrypt powłoki, automatyzujący przeprowadzenie całej serii pomiarów w powtarzalnych warunkach. Rozdzielenie tych ról pozwala zmieniać jeden element testu, bez ingerencji w pozostałe, co jest warunkiem koniecznym rzetelnego porównania.

Prezentowany rozdział ma na celu przedstawienie wszystkich elementów składających się na przeprowadzone badanie. Pierwsza część opisuje platformę sprzętową, następnie prezentowane są zaimplementowane warianty algorytmu FFT wraz z uzasadnieniem ich doboru. Kolejna część opisuje architekturę systemu pomiarowego i kroki podjęte w celu ograniczenia szumu pomiarowego. Rozdział zamyka metodyka przeprowadzonych pomiarów, wraz z stosowanymi metrykami pomiarowymi.

4.1. Opis platformy sprzętowej

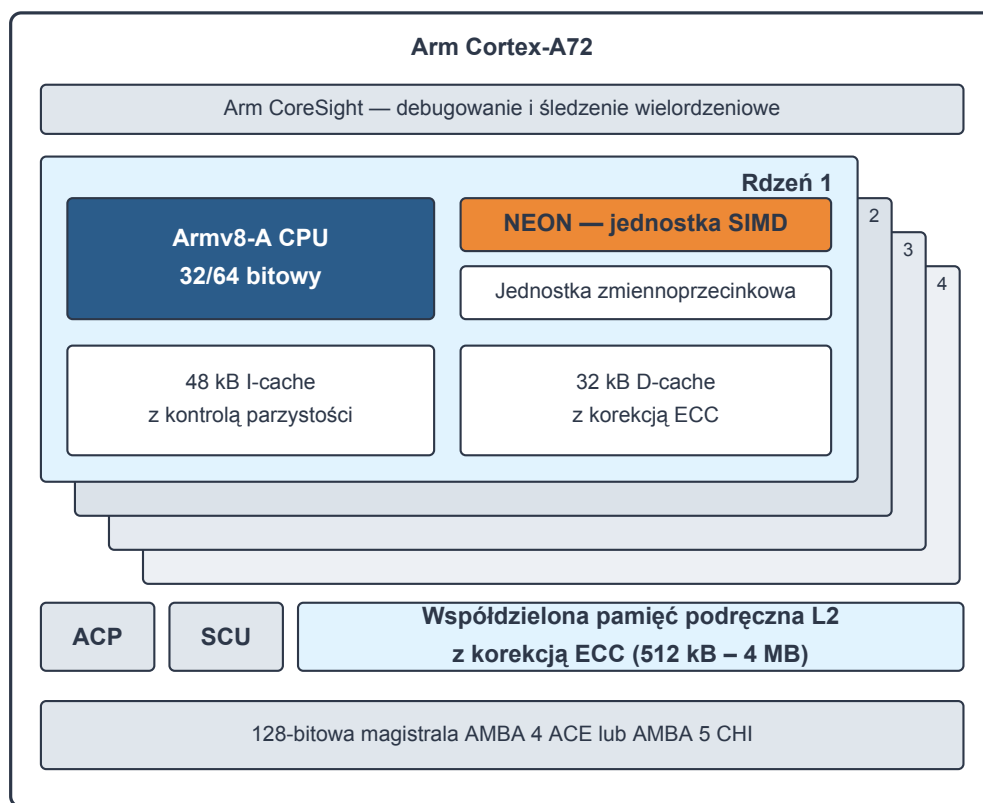
Platformą sprzętową, na której zrealizowano opisywany system testowy, jest Raspberry Pi 4 Model B [21]. Na płycie znajduje się układ SoC (system na chipie, ang. *System on Chip*) Broadcom BCM2711, zawierający cztery rdzenie ARM Cortex-A72 taktowane zegarem do 1,8 GHz [21]. W celu zapewnienia odpowiednich temperatur dla pomiarów zastosowano również dedykowaną obudowę na płytce z wentylatorami i radiator do rozpraszania ciepła z układu SoC. Podstawowe parametry techniczne przedstawiono poniżej:

- **SoC:** Broadcom BCM2711 [21]
- **Rdzenie CPU:** 4× ARM Cortex-A72, architektura ARMv8-A [22]
- **Taktowanie:** 1,8 GHz [21]
- **Pamięć podręczna L1:** 48 kB instrukcji (z kontrolą parzystości) + 32 kB danych (z korekcją ECC) na rdzeń [22]
- **Pamięć podręczna L2:** współdzielona, z korekcją ECC [22]
- **Pamięć operacyjna:** 1 GB [21]
- **Jednostka SIMD:** NEON (Advanced SIMD), rejestry 128-bitowe [20]

Wybór tej platformy wynika z jej powszechnej dostępności oraz z faktu, że reprezentuje typowy, niedrogi system wbudowany oparty na architekturze ARMv8-A [21]. Rdzeń Cortex-A72 posiada jednostkę zarządzania pamięcią (MMU), co umożliwia uruchomienie pełnego systemu Linux, a nie jedynie jego okrojonej wersji przeznaczonej dla mikrokontrolerów [22]. Płytkę posiada dodatkowo wszystkie elementy potrzebne do praktycznego korzystania z systemu - kartę microSD jako nośnik systemu, złącza sieciowe oraz porty USB.

Uproszczony diagram blokowy rdzenia (z zaznaczoną jednostką SIMD NEON) przedstawiono na rys. 4.1 [22].

Na rysunku widoczne są cztery identyczne rdzenie, z których każdy zawiera własną jednostkę centralną (CPU), jednostkę zmiennoprzecinkową (FPU) oraz pamięć podręczną L1, które występują na rysunku wraz z oznaczeniami mechanizmów ochrony danych (kontrola parzystości i korekcja ECC). Poza rdzeniami znajdują się bloki ACP i SCU odpowiadające za spójność danych między pamięciami podręcznymi poszczególnych rdzeni oraz dostęp zewnętrznych urządzeń. Jest też do dyspozycji pamięć podręczna L2, współdzielona między rdzenie. Jednostka SoC komunikuje się ze pozostałymi układami peryferyjnymi na płycie za pomocą 128-bitowej magistrali. Jednostka NEON jest sprzętowym blokiem SIMD wbudowanym w każdy rdzeń; jej działanie opisano szczegółowo w Sekcji 4.1.1.



Rysunek 4.1: Uproszczony schemat rdzenia Cortex-A72 z zaznaczoną jednostką NEON. Opracowanie własne na podstawie [22].

4.1.1. Jednostka NEON

NEON to sprzętowa jednostka realizująca rozszerzenie SIMD (*Single Instruction, Multiple Data* - pojedyncza instrukcja, wiele danych), dostępna w rdzeniach ARM, w tym w Cortex-A72 [20]. Przetwarzanie SIMD polega na wykonywaniu tej samej operacji jednocześnie na kilku zmien-

nych naraz, zamiast przetwarzać je pojedynczo w kolejnych cyklach. Jednostka ta operuje na dedykowanych rejestrach 128-bitowych, mieszczących w jednej instrukcji do szesnastu liczb 8-bitowych, ośmiu 16-bitowych, czterech 32-bitowych lub dwóch 64-bitowych. Przetwarzanie jest możliwe zarówno na typach całkowitych, jak i zmiennoprzecinkowych [20].

W prowadzonych badaniach jednostka NEON używana była zarówno samoczynnie przez kompilator, poprzez automatyczną wektoryzację włączaną flagami kompilatora, jak i świadomie przez użycie funkcji w `radix2-v3-neon`, opisanym w Sekcja 4.2.3.

4.2. Zaimplementowane warianty algorytmu FFT

Badania przeprowadzono na 11 implementacjach FFT. Proponowany zestaw opisano w tab. 4.1. Należy przy tym zaznaczyć, że badanie nie miało być przeglądem wszystkich znanych odmian algorytmu. Jego celem było sprawdzenie, w jaki sposób przekształcenia wewnątrz kodu oraz flagi kompilacji wpływają na wybrane metryki wydajnościowe transformaty. Prezentowany zestaw wybrano tak, aby dało się rozdzielić dwa źródła różnic w metrykach wykonania: schemat algorytmu i sposób zapisania tego schematu w kodzie.

Tabela 4.1: Zestawienie zaimplementowanych wariantów FFT

Wariant	Algorytm	Cecha wyróżniająca
<code>radix2-v0</code>	radix-2	wyznaczanie współczynników obrotu na bieżąco
<code>radix2-v1</code>	radix-2	współczynniki obrotu z tablicy
<code>radix2-v2-soa</code>	radix-2	<code>restrict</code> , układ danych SoA
<code>radix2-v3-neon</code>	radix-2	jawne funkcje wewnętrzne NEON
<code>radix4-v0</code>	radix-4	współczynniki obrotu liczone w pętli
<code>radix4-v1</code>	radix-4	współczynniki obrotu z tablicy
<code>splitradix</code>	split-radix	mieszany podział radix-2/radix-4, niestandardowy dostęp do pamięci
<code>stockham</code>	Stockham	kopiowanie tablicy, bez permutacji wstępnej
<code>radix2-par</code>	radix-2	podział po blokach etapu
<code>radix2-par-v2</code>	radix-2	równomierny podział pracy w etapie
<code>radix4-par</code>	radix-4	równomierny podział pracy w etapie

Pierwsze kryterium podziału to schemat obliczeń. Proponowane algorytmy to radix-2, radix-4, split-radix i Stockham. Różnią się one liczbą operacji zmiennoprzecinkowych i wzorcem dostępu do pamięci. Pełny kod źródłowy wszystkich wariantów znajduje się w załączniku do niniejszej pracy, a uzasadnienie doboru poszczególnych implementacji omówiono w Sekcja 4.2.3.

Drugim kryterium jest sposób implementacji rozwiązania w kodzie. Dla tego samego algorytmu przetwarzania przygotowano kilka implementacji różniących się wyłącznie technikami programistycznymi. Sprawdzono wpływ układu danych wejściowych, źródła współczynników obrotu, korzyści płynących z używania w kodzie programu typów danych pod instrukcje SIMD oraz sposobu podziału pracy między wątki. Dwa z zaimplementowanych wariantów ilustrują również, jak złe decyzje implementacyjne mogą zmieniać parametry wykonania, w tym nawet dokładność algorytmu. Zachowano w nich celowo nieefektywne rozwiązanie, żeby zmierzyć koszt pojedynczej decyzji implementacyjnej przy z pozoru niezmienionym algorytmie.

4.2.1. Wspólne założenia implementacyjne

Obliczenia prowadzone są w pojedynczej precyzji. Wybór ten wynika z docelowej platformy — rejestr wektorowy NEON o szerokości 128 bitów mieści cztery liczby 32-bitowe i tylko dwie 64-bitowe [20], więc typ `float` daje dwukrotnie większy potencjał wektoryzacji.

Dane wejściowe przechowywane są, w zależności od algorytmu, w jednym z dwóch układów. Rozróżnienie to ma znaczenie, ponieważ w pracy operuje się na strukturach liczb zespolonych, złożonych z części rzeczywistej i urojonej. Różnica między nimi polega na rozdzieleniu tablic struktur na osobne tablice dla każdego jej pola. Zabieg ten mnoży ułatwić pracę kompilatora, ponieważ eliminuje ryzyko nakładania się obszarów pamięci sygnalizowane analizie aliasowania wskaźników [20]. W pracy przyjęto dla tych dwóch układów danych nazewnictwo powszechne w literaturze: **układ złożony** (skrót **AoS**, ang. *Array of Structures*) oraz **układ rozdzielony** (skrót **SoA**, ang. *Structure of Arrays*).

Współczynniki obrotu, dla wszystkich algorytmów, poza jednym `radix2`, liczone są raz, a ich tablica przekazywana jest do funkcji transformaty jako argument. Tablica występuje w dwóch formach używanych zamiennie przez różne warianty. Do wyliczenia współczynnika obrotu przyjęto konwencję z ujemnym wykładnikiem, przedstawioną na eq. (3.4), stosowaną konsekwentnie we wszystkich implementacjach.

Wszystkie warianty, poza algorytmem Stockhama, działają w miejscu, co oznacza, że wynik transformaty zapisywany jest w tym samym buforze, w którym znajdowały się dane wejściowe [6]. Nie potrzebna jest alokacja nowej pamięci na wyniki pośrednie. Stosowane algorytmy wymagają wstępnej permutacji danych — odwrócenia bitowego dla radiksu 2 i `split-radix` oraz odwrócenia cyfr w systemie czwórkowym dla radiksu 4. Rozmiar transformaty jest zawsze potęgą dwójki; warianty `radix-4` dodatkowo wymagają potęgi czwórki.

Dodatkowo algorytmy `Radix2` i `Radix 4` można implementować na dwa możliwe sposoby, różniące się tym, co podlega podziałowi. Warianty te stosują **decymację w dziedzinie czasu** (DIT, ang. *Decimation in Time*) lub **decymację w dziedzinie częstotliwości** (DIF, ang. *Decimation in Frequency*) [6].

W schemacie DIT dzieli się próbki sygnału na parzyste i nieparzyste, co wymaga wstępnego przestawienia próbek. Wynik w takim przypadku otrzymuje się w porządku naturalnym. W schemacie DIF przestawieniu podlegają zamiast tego prążki widma - sygnał wejściowy pozostaje w porządku naturalnym, a permutacja przenoszona jest na wyjście [6]. Wszystkie zaimplementowane warianty `radix2`, `radix4` i `split-radix` wykorzystują schemat DIT.

4.2.2. Użyte biblioteki

Zestaw bibliotek zewnętrznych jest celowo ubogi, żeby mierzony program pozostał w całości pod kontrolą kompilatora badanego w pracy.

Biblioteka **pthread** (*POSIX Threads*), będąca częścią standardu POSIX [23], używana jest do wszystkich wariantów wielowątkowych. Wykorzystano z niej tworzenie i przyłączanie wątków oraz barierę `pthread_barrier_t`, która synchronizuje wątki między kolejnymi etapami transformaty. Implementacje wariantów wielowątkowych oparto na wiedzy zaczerpniętej z [23].

Biblioteka matematyczna **libm** dostarcza funkcje `cosf` i `sinf`. Używana jest przy generowaniu tablicy współczynników obrotu oraz wewnątrz pętli obliczeniowych dwóch wariantów algorytmu, co opisano w Sekcji 4.2.3.

Biblioteka **FFTW** [24] pełni rolę punktu odniesienia. Zainstalowana została jako gotowy,

skompilowany pakiet dystrybucji Debian (`libfftw3-dev`), a nie zbudowana ze źródeł na potrzeby tej pracy. Nie jest więc przedmiotem badania wpływu flag optymalizacji. Warto zaznaczyć, że biblioteka dobiera własną strategię obliczeń w czasie działania [24]. W pracy biblioteka posłużyła do oceny poprawności wyników oraz jako wartość referencyjna dla parametrów wydajnościowych osiąganych przez dojrzałą i sprawdzoną implementację.

Należy wymienić również plik nagłówkowy `arm_neon.h`. Udostępnia on funkcje i typy zmiennych dla jednostki NEON. Nie jest to biblioteka w klasycznym tego słowa znaczeniu, lecz interfejs do instrukcji procesora.

4.2.3. Badane wersje pojedynczo wątkowe

Pierwszym z prezentowanych algorytmów jest `radix2-v0` — klasyczny radix-2 DIT w układzie AoS, w którym współczynniki obrotu powstają na bieżąco w każdej pętli [9]. W każdym etapie wyliczana jest jedna wartość twiddle factor, a kolejne otrzymuje się przez mnożenie, wykorzystując poprzednią wartość. Rozwiązanie to jest tanie obliczeniowo, ale kumuluje błąd zaokrąglenia w kolejnych mnożeniach. Kod źródłowy algorytmu przedstawiono w listing 4.1

```

1 static complex_t *Radix2_v0(complex_t *x, uint32_t n, ...){
2     bit_reverse_aos(x, n);
3
4     for (uint32_t step = 2; step <= n; step <<= 1)
5     {
6         uint32_t half = step >> 1;
7         float theta = -2.0f * PI_F / (float)step;
8         complex_t w_step = {cosf(theta), sinf(theta)};
9
10        for (uint32_t i = 0; i < n; i += step)
11        {
12            complex_t w = {1.0f, 0.0f};
13            for (uint32_t j = 0; j < half; j++)
14            {
15                uint32_t top = i + j;
16                uint32_t bot = top + half;
17                complex_t t = complex_mul(x[bot], w.real, w.imag);
18
19                x[bot].real = x[top].real - t.real;
20                x[bot].imag = x[top].imag - t.imag;
21                x[top].real += t.real;
22                x[top].imag += t.imag;
23
24                w = complex_mul(w, w_step.real, w_step.imag);
25            }
26        }
27    }
28    return x;
29 }
```

Listing 4.1: Kompletna implementacja `radix2-v0`

Wariant `radix2-v1` różni się od poprzedniego wyłącznie źródłem współczynników: pobiera je z przygotowanej wcześniej tablicy, zamiast wyliczać je na bieżąco. Algorytm, układ danych i struktura pętli pozostają identyczne. Na listing 4.2 przedstawiono istotną modyfikację względem `v0`, która poprawia dokładność numeryczną.

```

1 const complex_t *w = tw->st_aos + tw_stage_offset(half);
```

```

2 /* współczynnik pobierany bezpośrednio z tablicy, modyfikacja w stosunku do żponiego
   wyliczenia z v0/
3 complex_t t = complex_mul(x[bot], w[j].real, w[j].imag);

```

Listing 4.2: Modyfikacja radix2-v1, względem v0. Odczyt współczynnika obrotu z tablicy

Wariant radix2-v2-soa przenosi ten sam algorytm do układu SoA i oznacza wszystkie wskaźniki modyfikatorem restrict. Obie zmiany służą temu samemu celowi — usunięciu przeszkód, które blokują automatyczną wektoryzację [20]. Rozdzielenie części rzeczywistej i urojonej daje w wewnętrznej pętli ciągły dostęp do pamięci, a modyfikator restrict informuje kompilator, że ten wskaźnik będzie jedynym odniesieniem do danej komórki pamięci [20]. Obie zmiany, względem wersji radix2-v1 widoczne są na listing 4.3.

```

1 float *restrict re = x->re;
2 float *restrict im = x->im;
3
4 const float *restrict wr = tw->st_re + tw_stage_offset(half);
5 const float *restrict wi = tw->st_im + tw_stage_offset(half);
6
7 float *restrict top_re = re + i;
8 float *restrict bot_re = re + i + half;
9 /* analogicznie top_im, bot_im */

```

Listing 4.3: Rozdzielone tablice i modyfikator restrict, zastosowany w wariancie radix2-v2-soa

Wariant radix2-v3-neon rozszerza poprzedni o jawne funkcje wewnętrzne NEON [20]. Pojedyncza pętla algorytmu przetwarza cztery próbki naraz, a mnożenie zespolone realizowane jest instrukcjami złożonego mnożenia z akumulacją, widocznymi na listing 4.4 [20]. Pozostała, niepełna część pętli obsługiwana jest kodem skalarnym. Wariant dodano, aby sprawdzić, o ile ręczne użycie niskopoziomowych instrukcji SIMD poprawia mierzone metryki względem wersji polegającej na automatycznej wektoryzacji włączanej flagami kompilatora.

```

1 float32x4_t br = vld1q_f32(bot_re + j);
2 float32x4_t bi = vld1q_f32(bot_im + j);
3 float32x4_t vwr = vld1q_f32(wr + j);
4 float32x4_t vwi = vld1q_f32(wi + j);
5
6 float32x4_t tr = vmulq_f32(br, vwr);
7 tr = vfmsq_f32(tr, bi, vwi); /* tr = br*wr - bi*wi */
8 float32x4_t ti = vmulq_f32(br, vwi);
9 ti = vfmaq_f32(ti, bi, vwr); /* ti = br*wi + bi*wr */

```

Listing 4.4: Zastosowanie instrukcji SIMD w wariancie radix2-v3-neon

Warianty radix4-v0 i radix4-v1 różnią się, podobnie jak te same wersje dla radix2, źródłem współczynników obrotu. Teoretyczny opis algorytmu radix4 oraz idee jego implementacji opisano w [25]. Wariant radix4-v0 wymaga jednak wyliczenia dużej ilości współczynników trygonometrycznych i innej permutacji bitowej niż radix2. Kod źródłowy pojedynczego przebiegu algorytmu przedstawiono w listing 4.5.

```

1 for (uint32_t step = 4; step <= n; step <= 2) {
2     uint32_t quarter = step >> 2;
3     float theta = -2.0f * PI_F / (float)step;
4
5     for (uint32_t i = 0; i < n; i += step) {
6         for (uint32_t j = 0; j < quarter; j++) {

```

```

7         float w1_r = cosf(theta * (float)j);
8         float w1_i = sinf(theta * (float)j);
9         float w2_r = cosf(theta * 2.0f * (float)j);
10        float w2_i = sinf(theta * 2.0f * (float)j);
11        float w3_r = cosf(theta * 3.0f * (float)j);
12        float w3_i = sinf(theta * 3.0f * (float)j);
13
14        uint32_t i0 = i + j;
15        uint32_t i1 = i0 + quarter;
16        uint32_t i2 = i1 + quarter;
17        uint32_t i3 = i2 + quarter;
18
19        complex_t x0 = x[i0];
20        complex_t t1 = complex_mul(x[i1], w1_r, w1_i);
21        complex_t t2 = complex_mul(x[i2], w2_r, w2_i);
22        complex_t t3 = complex_mul(x[i3], w3_r, w3_i);
23
24        float a_r = x0.real + t2.real, a_i = x0.imag + t2.imag;
25        float b_r = x0.real - t2.real, b_i = x0.imag - t2.imag;
26        float c_r = t1.real + t3.real, c_i = t1.imag + t3.imag;
27        float d_r = t1.real - t3.real, d_i = t1.imag - t3.imag;
28
29        x[i0].real = a_r + c_r; x[i0].imag = a_i + c_i;
30        x[i1].real = b_r + d_i; x[i1].imag = b_i - d_r;
31        x[i2].real = a_r - c_r; x[i2].imag = a_i - c_i;
32        x[i3].real = b_r - d_i; x[i3].imag = b_i + d_r;
33    }

```

Listing 4.5: Przebieg algorytmu w radix4-v0

Wariant radix4-v1 stosuje ten sam zabieg co radix2-v1 — współczynniki obrotu pobierane są z przygotowanej wcześniej tablicy, zamiast wyliczane na bieżąco, przy identycznej strukturze motylka.

Wersje algorytmu radix4 uwzględniono w pracy, ponieważ przy tej samej liczbie próbek wymagają mniej etapów obliczeń i krótszych pętli niż radix2, co powinno przekładać się na mniejszą liczbę wykonywanych instrukcji; wymagają jednak innego rozkładu danych wejściowych, co może sprawić, że niekoniecznie będą one szybsze, pomimo mniejszej liczby instrukcji.

Wariant splitradix łączy podział radix2 i radix4, dzięki czemu wykonuje mniej mnożeń zespolonych niż każdy z nich osobno [25]. W pojedynczym przebiegu algorytmu dwie z czterech próbek są korygowane współczynnikami obrotu przed połączeniem, a pozostałe dwie łączone są bezpośrednio [25]. Teoretyczny opis algorytmu oraz wyprowadzenie tej struktury przedstawiono w [25]. Fragment etapu L-kształtnego przedstawiono w listing 4.6.

```

1         float t1_r = h.real * w1_r - h.imag * w1_i;
2         float t1_i = h.real * w1_i + h.imag * w1_r;
3         float t3_r = iq.real * w3_r - iq.imag * w3_i;
4         float t3_i = iq.real * w3_i + iq.imag * w3_r;
5
6         float a_r = t1_r + t3_r;
7         float a_i = t1_i + t3_i;
8         float b_r = t1_i - t3_i;
9         float b_i = -(t1_r - t3_r);
10
11        x[ig].real = g.real + a_r;
12        x[ig].imag = g.imag + a_i;

```

```

13         x[ih].real = g.real - a_r;
14         x[ih].imag = g.imag - a_i;
15         x[ig4].real = g4.real + b_r;
16         x[ig4].imag = g4.imag + b_i;
17         x[ii].real = g4.real - b_r;
18         x[ii].imag = g4.imag - b_i;

```

Listing 4.6: Fragment etapu L-kształtnego wariantu `splitradix`

Wariant `splitradix` uwzględniono w pracy, ponieważ mniejsza liczba mnożeń zespolonych powinna przekładać się na mniejszą liczbę wykonywanych instrukcji. Wymaga on jednak bardziej nieregularnej struktury pętli niż `radix2` i `radix4` i trudniejszego dostępu do pamięci.

Wariant `stockham` ma inne założenie niż prezentowane algorytmy z rodziny Radix 2 i 4. Działa on bowiem z dwoma tablicami zamiast z jedną, co nazywane jest działaniem (ang. *out-of-place*). Zamiast przestawiać elementy tablicy wejścia, w każdym etapie zapisuje wynik do bufora [25]. Dzięki temu nie jest potrzebna wstępna permutacja bitowa, a wynik końcowy otrzymywany jest w porządku naturalnym. Fragment odpowiedzialny za zamianę buforów przedstawiono w listing 4.7. Teoretyczny opis algorytmu dostępny jest w pozycji [25].

```

1  const complex_t *a = src + j * m;
2  const complex_t *b = src + j * m + l * m;
3  complex_t *o0 = dst + 2 * j * m;
4  complex_t *o1 = dst + 2 * j * m + m;
5
6  o0[k].real = ar + br;
7  o1[k].real = dr * wr - di * wi;
8
9  complex_t *t = src; src = dst; dst = t;

```

Listing 4.7: Zapis do drugiego bufora w wariacie `stockham`

Wariant `stockham` uwzględniono w pracy, ponieważ eliminuje sekwencyjną fazę permutacji bitowej; wymaga jednak dwukrotnie większego zapotrzebowania na pamięć niż warianty działające w miejscu.

4.2.4. Badane wersje wielowątkowe

Wszystkie warianty zaimplementowano na podobnych założeniach. Wątki są tworzone raz, przed pierwszym etapem, i przyłączane dopiero po ostatnim, a synchronizację między etapami zapewnia bariera wywoływana na końcu każdego z nich — jest to niezbędne, ponieważ kolejny etap FFT korzysta z wyników poprzedniego [23]. Permutacja wstępna pozostaje we wszystkich wariantach sekwencyjna, co wprowadza ograniczenie minimalnego czasu przetwarzania.

Wariant `radix2-par` dzieli pracę po blokach etapu: wątek o numerze t obsługuje bloki o indeksach $t, t+p, t+2p$ i tak dalej, gdzie p oznacza liczbę wątków. Liczba bloków maleje dwukrotnie w każdym kolejnym etapie, więc w etapach końcowych część wątków nie ma przydzielonej pracy — w ostatnim etapie istnieje tylko jeden blok, więc obliczenia wykonuje wyłącznie jeden wątek. Kod źródłowy funkcji wątku przedstawiono w listing 4.8.

```

1  static void *worker(void *arg)
2  {
3      thread_data_t *d = (thread_data_t *)arg;
4      uint32_t n = d->n;
5
6      for (uint32_t step = 2; step <= n; step <<= 1) {

```

```

7     uint32_t half = step >> 1;
8     const complex_t *w = d->tw->st_aos + tw_stage_offset(half);
9
10    for (uint32_t i = d->thread_id * step; i < n; i += step * d->num_threads) {
11        for (uint32_t j = 0; j < half; j++) {
12            uint32_t top = i + j;
13            uint32_t bot = top + half;
14            complex_t t = complex_mul(d->x[bot], w[j].real, w[j].imag);
15
16            d->x[bot].real = d->x[top].real - t.real;
17            d->x[bot].imag = d->x[top].imag - t.imag;
18            d->x[top].real += t.real;
19            d->x[top].imag += t.imag;
20        }
21    }
22    pthread_barrier_wait(d->barrier);
23 }
24 return NULL;
25 }

```

Listing 4.8: Funkcja wątku w wariancie radix2-par

Synchronizacja między wątkami opiera się na barierze utworzonej raz, przed pierwszym etapem, oraz zniszczonej po przyłączeniu wszystkich wątków. Fragment odpowiedzialny za administrację wątkami przedstawiono w listing 4.9.

```

1 pthread_barrier_init(&barrier, NULL, nthreads);
2 for (uint32_t i = 0; i < nthreads; i++) {
3     data[i] = (thread_data_t){ x, tw, n, i, nthreads, &barrier };
4     pthread_create(&threads[i], NULL, worker, &data[i]);
5 }
6 for (uint32_t i = 0; i < nthreads; i++)
7     pthread_join(threads[i], NULL);
8 pthread_barrier_destroy(&barrier);

```

Listing 4.9: Administracja wątków w wariancie radix2-par

Wariant radix2-par-v2 ma inne podejście do podziału pracy między wątki. Zamiast dzielić dane wejściowe na bloki, wariant radix2-par-v2 traktuje wszystkie operacje wykonywane w danym etapie jako jeden ciągły ciąg o stałej długości $N/2$ i dzieli go na równe części, po jednej dla każdego wątku. Indeksy próbek odtwarzane są z numeru elementu tego ciągu operacjami przesunięcia bitowego i maskowania, co jest możliwe dzięki temu, że długości etapów są potęgami dwójki. Podział jest więc wyliczany raz i pozostaje zrównoważony we wszystkich etapach. Zmiana podziału pracy między wątki została zaczerpnięta z sposobu implementacji fft na procesorach graficznych, co zostało zaprezentowane w [26]. Zmianę względem radix2-par przedstawiono w listing 4.10.

```

1 static void range_of(uint32_t total, uint32_t id, uint32_t nthreads,
2                     uint32_t *first, uint32_t *last)
3 {
4     uint32_t base = total / nthreads;
5     uint32_t rest = total % nthreads;
6     *first = id * base + (id < rest ? id : rest);
7     *last = *first + base + (id < rest ? 1u : 0u);
8 }
9

```

```

10 /* odtworzenie indeksow z numeru motylka b, zamiast petli po blokach */
11 uint32_t j = b & mask;
12 uint32_t top = ((b >> log_half) << (log_half + 1)) + j;
13 uint32_t bot = top + half;

```

Listing 4.10: Podział płaskiej przestrzeni motylków i odtworzenie indeksów w wariacie radix2-par-v2 — modyfikacja względem radix2-par

Algorytmy radix2-par-radix2-par-v2 wybrano do porównania z uwagi na chęć zbadania, czy przy niezmienionym algorytmie sam sposób przydziału pracy między wątki może decydować o skalowalności.

Wariant radix4-par przenosi rozwiązanie z radix2-par-v2 na schemat radix-4. Przetwórczy etapów przetwarzania ma rozmiar $N/4$, a każdy z motylków przetwarza cztery próbki i wymaga trzech współczynników obrotu pobieranych z pełnej tablicy. Odtworzenie indeksów przedstawiono w listing 4.11.

```

1 uint32_t j = b & mask;
2 uint32_t i = (b >> log_quarter) << (log_quarter + 2);
3
4 uint32_t k1 = (j * stride) & msk;
5 uint32_t k2 = (2u * j * stride) & msk;
6 uint32_t k3 = (3u * j * stride) & msk;

```

Listing 4.11: Odtworzenie indeksów z numeru motylka w wariacie radix4-par

Wariant ten wybrano do badań, ponieważ zdecydowano się sprawdzić, czy mniejsza liczba etapów — a więc i mniejsza liczba barier — rekompensuje większy koszt pojedynczego przetwarzania.

4.3. Założenia projektowe i architektura aplikacji

Użyty system pomiarowy, od strony programowej, składa się z systemu operacyjnego Linux oraz programu użytkownika napisanego w języku C, który steruje przebiegiem badań. Program ten uruchamia kolejne warianty, mierzy ich czas wykonania i zbiera dane liczników sprzętowych. Ponieważ program pomiarowy działa jako zwykły proces pod kontrolą systemu operacyjnego, pomiar jest nieuchronnie obciążony szumem systemowym. Przez pojęcie szumu rozumie się w tym wypadku przełączaniem kontekstu, obsługą przerw, aktywnością innych procesów czy wyłączenia związane z przerwami. Z tej zależności wynika konieczność przeprowadzenia konkretnych działań, których celem było ograniczenie szumu. Działania te opisano w Sekcji 4.3.1.

Należy zwrócić przy tym uwagę, że w realnych zastosowaniach systemu wbudowanego prawdopodobnie nie byłoby możliwości przeprowadzenia wszystkich działań opisanych omawianej sekcji. Docelowe urządzenie w większości zastosowań pracuje z aktywnymi usługami systemowymi i bez kontroli nad taktowaniem procesora. Na potrzeby niniejszej pracy zdecydowano się częściowo wyciszyć system, aby uzyskać wyniki powtarzalne i możliwe do jak najbardziej jednoznacznej interpretacji.

4.3.1. Użyty system operacyjny i jego konfiguracja

W przeprowadzonych badaniach wykorzystano system operacyjny Linux, a dokładnie Raspberry Pi OS, opartego na dystrybucji Debian 13, z jądrem Linux w wersji 6.18.39+rpt-rpi-v8 oraz biblioteką standardową `glibc` w wersji 2.41.

Do kompilacji przygotowanych wariantów użyto dwóch kompilatorów:

- **GCC** w wersji 14.2.0 (pakiet dystrybucji Debian),
- **Clang** w wersji 19.1.7 (pakiet dystrybucji Debian),

Obie wersje kompilatorów pochodzą z oficjalnych repozytoriów pakietów Debiana, co jest standardową praktyką, ponieważ zapewnia spójność z resztą oprogramowania systemowego i powtarzalność środowiska badawczego.

Przed przystąpieniem do pomiarów podjęto szereg działań mających na celu ograniczenie zmienności wyników, niewynikających z stosowanych programów lub konfiguracji procesu kompilacji. Przed pomiarami wykonano następujące działania:

- zasilanie zgodne ze specyfikacją i aktywne chłodzenie, aby procesor nie obniżał częstotliwości taktowania z powodu spadku napięcia lub przegrzania — zjawisko to określa się jako ograniczanie wydajności (ang. *throttling*);
- instalacja Raspberry Pi OS Lite 64-bit bez środowiska graficznego - obowiązkowe wsparcie NEON i mniej procesów w tle;
- kompilacja bezpośrednio na platformie docelowej, bez kompilacji krzyżowej;
- ustawienie stałej częstotliwości taktowania procesora (*performance*);
- kontrola throttlingu i temperatury rdzenia przed każdą serią i w jej trakcie, z odrzucaniem pomiarów wykonanych w stanie throttlingu;
- wyłączenie zbędnych usług systemowych i zadań cyklicznych, w tym łączności bezprzewodowej;
- przypięcie procesu pomiarowego do wybranych rdzeni procesora z podniesionym priorytetem szeregowania.

Środowisko weryfikowano również przed każdą serią skryptem sprawdzającym parametry środowiska. Dane te były zapisywane do plików logujących.

4.3.2. Architektura programu testowego

Program testowy składa się z trzech elementów: zaimplementowanego algorytmu, programu realizującego pomiar oraz skryptu powłoki, który ma za zadanie zautomatyzować przeprowadzanie wielu pomiarów w powtarzalnych warunkach. Do powiązania wdrożonego algorytmu z programem pomiarowym wprowadzono kilka funkcji pomocniczych, służących na przykład do pomiaru czasu, zapisywaniu danych pomiarowych do plików, etc.

Każdy z zaimplementowanych wariantów FFT używa jedną wspólną strukturę opisową. Finalny program pomiarowy jest kompilowany razem z jednym konkretnym plikiem algorytmu i wspólnym kodem infrastruktury, tworząc jeden plik wykonywalny na wariant. Dodatkowo, wyłącznie na potrzeby analizy wyników, plik algorytmu jest kompilowany osobno do pojedynczego pliku obiektowego, co pozwala wyodrębnić asembler samego algorytmu, bez domieszki kodu wspólnego.

Program pomiarowy udostępnia trzy tryby pracy: pomiar czasu, walidację numeryczną względem referencyjnej transformaty w podwójnej precyzji oraz zapis widma do pliku. W trybie pomiaru dane wejściowe są reinicjalizowane przed każdym powtórzeniem. Rejestrowane są między innymi **czas ściany** (ang. *wall-clock time*, czyli rzeczywisty czas upływający między początkiem a

końcem pomiaru), **czas procesora** (ang. *CPU time*, czyli czas, przez który procesor faktycznie wykonywał instrukcje programu, z pominięciem oczekiwania na zasoby systemowe). Przed zapisaniem wyników pomiaru wykonywane są nierejestrowane przebiegi algorytmu (tak zwane przebiegi rozgrzewające), a przy każdym powtórzeniu zapisywana jest temperatura rdzenia i szczytowe zużycie pamięci.

Skrypt powłoki służy do generowania pełnej macierzy testów, rozumianej jako kombinacje wybranych algorytmów FFT z warunkami testowymi, na które składają się kompilator, zestaw flag kompilacji, rozmiar danych wejściowych oraz liczba wątków, stosowana dla wariantów wielowątkowych. Przed faktycznym rozpoczęciem testu skrypt sprawdza konfigurację środowiska, opisana w Sekcja 4.3.1.

Rzeczywisty przebieg skryptu testowego dzieli się na cztery fazy: budowanie, walidację numeryczną (wariant, który nie przechodzi, nie jest uwzględniany w wynikach), pomiar czasu w kilku rundach oraz pomiar liczników sprzętowych. Pojedyncza runda pomiarowa obejmuje kilka powtórzeń pomiaru dla danej kombinacji warunków testowych, a cały pomiar czasu składa się z kilku takich rund, dzięki czemu każda komórka macierzy testów jest ostatecznie reprezentowana przez kilkadziesiąt powtórzeń. Kolejność komórek macierzy jest losowana niezależnie w każdej rundzie, co stanowi obronę przed rosnącą temperaturą fizycznego sprzętu.

Wynikiem działania skryptu testowego jest zapis surowych, danych z zapisanych metryk do plików pomiarowych, stanowiących materiał do analizy. Przedstawiona powyżej procedura pomiarowa została przedstawiona na schemacie rys. 4.2.

4.3.3. Programowe narzędzia pomiarowe

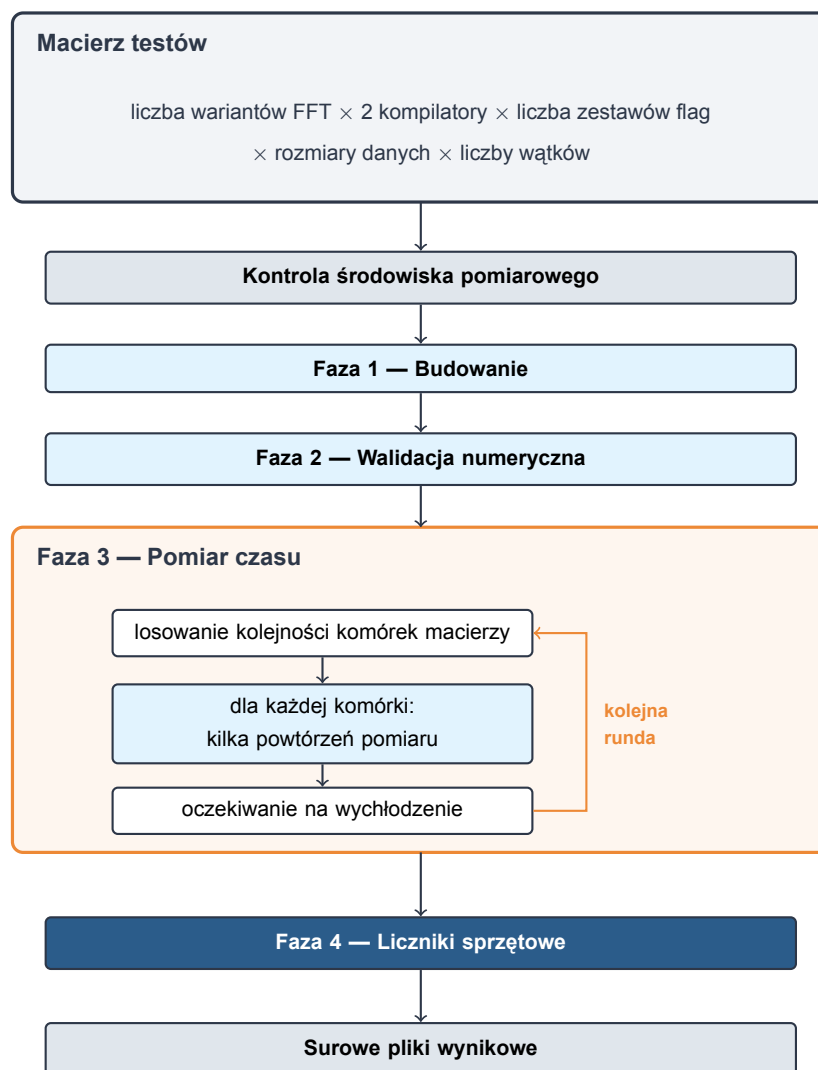
Do pomiaru czasu wykorzystano zegar monotoniczny systemu operacyjnego, niepodlegający korekcie czasu systemowego. Liczniki sprzętowe procesora odczytywano popularnym narzędziem Linuxa, o nazwie `perf`. Narzędzie to udostępnia dostęp do jednostki monitorowania wydajności. Jednostka ta zlicza sprzętowo wybrane zdarzenia zachodzące podczas wykonywania programu, bez modyfikowania mierzonego kodu. Rdzeń Cortex-A72 udostępnia ograniczoną liczbę programalnych liczników jednocześnie, dlatego zamiast długiej listy zdarzeń zastosowano dwa krótsze zestawy. Pierwszy z nich obejmuje ogólny obraz wykonania: liczbę cykli procesora, liczbę wykonanych instrukcji, odwołania i chybień pamięci podręcznej oraz błędne predykcje skoków. Drugi zestaw rozróżnia chybień według konkretnego poziomu pamięci podręcznej.

Zużycie pamięci i temperaturę rdzenia odczytywano z interfejsów systemowych. Do analizy kodu wynikowego użyto narzędzi `objdump` oraz `size`, a stopień wektoryzacji oceniano dodatkowo na podstawie raportów generowanych przez same kompilatory.

4.3.4. Weryfikacja poprawności obliczeń z biblioteka zewnętrzna

Poprawność każdej implementacji weryfikowano przez porównanie wyniku z wartością referencyjną, wyznaczoną biblioteką FFTW w podwójnej precyzji [24]. Zastosowanie wyższej precyzji w wzorcu pozwala odróżnić błąd wynikający z pojedynczej precyzji obliczeń od błędu implementacji.

Weryfikacja odbywa się w osobnej fazie, przed pomiarami czasu i liczników sprzętowych. Wariant, którego błąd przekracza przyjęty próg, jest odrzucany i nie bierze udziału w dalszych pomiarach. Wyniki porównania są zapisywane do pliku wraz z informacją o werdykcie.



Rysunek 4.2: Przebieg pojedynczej serii pomiarowej

4.4. Metodyka przeprowadzonych pomiarów

Badanie prowadzone jest wzdłuż trzech osi porównań. Pierwsza to wpływ flag kompilacji: ten sam wariant kodu budowany jest z różnymi zestawami flag, a punktem odniesienia jest poziom -O0, przy którym kompilator nie stosuje przekształceń optymalizacyjnych. Druga to wpływ technik programistycznych: te same flagi stosowane są do różnych wariantów implementacyjnych. Trzecia to wpływ samego kompilatora - ten sam kod budowany jest niezależnie kompilatorami GCC i Clang, przy zachowaniu odpowiadających sobie zestawów flag, co pozwala sprawdzić, na ile obserwowane efekty są właściwością konkretnej implementacji kompilatora, a na ile wynikają z samych przekształceń optymalizacyjnych.

Warunkiem poprawnego porównania jest to, aby między zestawianymi pomiarami zmieniła się wyłącznie badana zmienna. Dlatego wszystkie warianty korzystają z tego samego sygnału wejściowego, tej samej tablicy współczynników obrotu i tego samego kodu pomiarowego, a porównania prowadzone są wyłącznie w obrębie jednej serii, ponieważ między seriami mogą zmienić się warunki termiczne platformy.

Przyjęto następujące parametry pomiaru:

- kompilatory: GCC oraz Clang, z odpowiadającymi zestawami flag,
- rozmiary danych wejściowych: 1024, 16384 oraz 1048576 próbek dla wariantów sekwencyjnych, a 4096, 65536 oraz 1048576 próbek dla wariantów wielowątkowych,
- liczby wątków: 1, 2, 4 oraz 8 (dla wariantów wielowątkowych),
- sygnał wejściowy: multitonowy ten sam dla każdego algorytmu,
- liczba rund pomiarowych: 6, po 5 powtórzeń w rundzie, co daje 30 powtórzeń na każdą komórkę macierzy,
- liczba przebiegów rozgrzewających: 3 przed każdą serią powtórzeń,
- liczba zestawów flag: 11, od -O0 po zestawy łączące -O3 z flagami architektury i wektoryzacji.

Rozmiary transformaty dobrano tak, aby rozpiąć zakres od zbioru danych mieszczącego się w pamięci podręcznej pierwszego poziomu, przez rozmiary mieszczące się w pamięci drugiego poziomu, aż po zbiory wyraźnie ją przekraczające.

We wszystkich pomiarach stosowano ten sam sygnał wejściowy - wielotonowy, o częstotliwości próbkowania 6000 Hz i częstotliwości podstawowej 1000 Hz. Parametry te są stałymi wspólnymi dla wszystkich wariantów. Sygnał generowany jest deterministycznie, więc jest identyczny w każdym uruchomieniu programu. Pełny wykaz badanych zestawów flag przedstawiono w listing 4.12.

```
1 00 -O0
2 01 -O1
3 02 -O2
4 03 -O3
5 0s -Os
6 03-unroll -O3 -funroll-loops
7 03-ffast -O3 -ffast-math
8 03-mcpu -O3 -mcpu=cortex-a72
9 03-ffast-mcpu -O3 -ffast-math -mcpu=cortex-a72
10 03-ffast-mcpu-alias -O3 -ffast-math -mcpu=cortex-a72 \
```

```

11 --param vect-max-version-for-alias-checks=30
12 03-lto -O3 -flto

```

Listing 4.12: Zestawy flag kompilacji badane w pracy

Etykiety w lewej kolumnie służą do identyfikacji zestawów w wynikach. Kryteria odrzucania pomiarów przyjęto przed rozpoczęciem badań. Pomiar wykonany w stanie ograniczenia zegara jest odrzucany, wariant niepoprawny numerycznie nie jest mierzony, a seria, w trakcie której wystąpił throttling jest powtarzana.

W przypadkach, w których pomiary czasu i liczników sprzętowych nie wyjaśniają zaobserwowanej różnicy, stosowane jest porównanie kodu wynikowego. Asembler samego algorytmu, uzyskany z osobno skompilowanego pliku, jest normalizowany przez usunięcie adresów i przesunięcie, a następnie zestawiany z innym, porównywanym algorytmem.

4.4.1. Metryki używane w badaniach

W pracy stosowano następujące metryki:

Metryki czasowe

Podstawową metryką jest czas wykonania pojedynczej transformaty. Ze względu na prawoskośny rozkład czasów jako wartość reprezentatywną przyjęto medianę $t_{1/2}$, a nie średnią arytmetyczną. Rozrzut opisuje rozstęp międzykwartylowy, wyrażany również względem mediany, przedstawiony w eq. (4.1).

$$IQR = Q_3 - Q_1, \quad IQR_{\%} = \frac{Q_3 - Q_1}{t_{1/2}} \cdot 100\% \quad (4.1)$$

gdzie Q_1 i Q_3 to odpowiednio pierwszy i trzeci kwartył. Wartość $IQR_{\%}$ pełni jednocześnie rolę wskaźnika jakości środowiska pomiarowego i wyznacza rozdzielczość całego badania - różnice mniejsze niż zmierzony rozrzut są nieodróżnialne od szumu,

Stosunek czasu procesora do czasu ściany, opisany w eq. (4.2), pozwala ocenić wykorzystanie równoległości.

$$R = \frac{t_{CPU}}{t_{wall}} \quad (4.2)$$

Liczniki sprzętowe

Zliczana jest liczba cykli procesora C oraz wykonanych instrukcji I . Ich iloraz, przedstawiony w eq. (4.3), określa średnią liczbę instrukcji wykonanych w jednym cyklu.

$$IPC = \frac{I}{C} \quad (4.3)$$

Metryka ta rozdziela dwie różne przyczyny wolniejszego działania: większą liczbę instrukcji do wykonania od gorszego wykorzystania procesora. Dla pamięci podręcznej wyznaczany jest współczynnik chybień, osobno dla każdego poziomu, zgodnie z eq. (4.4).

$$m = \frac{r}{a} \cdot 100\% \quad (4.4)$$

gdzie a oznacza liczbę odwołań do danego poziomu, a r liczbę chybień.

Metryki wektoryzacji

Na podstawie asemblera samego algorytmu zliczane są instrukcje wektorowe n_{SIMD} oraz skalarne n_{skal} operujące na liczbach zmiennoprzecinkowych. Udział instrukcji wektorowych wyznaczono zgodnie z eq. (4.5).

$$v = \frac{n_{\text{SIMD}}}{n_{\text{SIMD}} + n_{\text{skal}}} \cdot 100\% \quad (4.5)$$

Metryki pamięciowe i rozmiar kodu

Rejestrowane jest szczytowe zużycie pamięci procesu oraz rozmiar sekcji kodu wynikowego samego algorytmu.

Metryki dokładności

Poprawność wyniku oceniana jest błędem średniokwadratowym względem transformaty referencyjnej, wyznaczanym zgodnie z eq. (4.6).

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{k=0}^{N-1} |X_k - X_k^{\text{ref}}|^2} \quad (4.6)$$

Drugą stosowaną miarą jest stosunek sygnału do szumu, opisany w eq. (4.7).

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \frac{\sum_k |X_k^{\text{ref}}|^2}{\sum_k |X_k - X_k^{\text{ref}}|^2} \quad (4.7)$$

gdzie X_k oznacza k -ty prążek widma badanego wariantu, a X_k^{ref} — odpowiadający mu prążek transformaty referencyjnej. Dodatkowo rejestrowany jest błąd maksymalny pojedynczego prążka.

5. ANALIZA WYNIKÓW PRZEPROWADZONYCH BADAŃ

Program kompilatora jest niezbędnym narzędziem pracy dla każdego inżyniera wytwarzającego oprogramowanie, który zwraca uwagę na zużycie zasobów maszyny wykonującej jego program. W wielu przypadkach narzędzie to jest traktowane jako czarna skrzynka - kod źródłowy jest wejściem, na wyjściu otrzymujemy plik wykonywalny, a to, co dzieje się pomiędzy, pozostaje poza uwagą programisty. Przy próbie optymalizacji programu zwykle to logika i przebieg algorytmu jest modyfikowana jako pierwsza, pomijając wpływ, jaki na wydajność ma proces tłumaczenia kodu na instrukcje maszynowe.

Celem przeprowadzonych badań było sprawdzenie, na ile możliwa była poprawa wydajności implementacji FFT z użyciem różnych zabiegów. Jednym z nich była weryfikacja, ile dawała sama zmiana flag optymalizacji kompilatora przy niezmiennym kodzie algorytmu. Weryfikowano również, czy jawne użycie instrukcji wykorzystania sprzętu fizycznego, nawet w ograniczonej skali takiej jak zmiana układu danych wejściowych czy ręczne użycie instrukcji wektorowych, mogło dać realny wpływ na optymalizację.

Wyniki i wnioski z przeprowadzonych badań mogą mieć aspekt praktyczny, zwłaszcza w systemach wbudowanych, które często mają ograniczone zasoby. W ich przypadku większa wydajność, bez droższego sprzętu fizycznego, oznacza niższy koszt i pobór energii, co może realnie poprawić działanie, stabilność i rentowność całego systemu.

Celem niniejszego rozdziału jest przedstawienie uzyskanych wyników oraz próba ich wyjaśnienia poprzez analizę przyczyn stojących za zmianą poszczególnych parametrów optymalizacyjnych.

5.1. Powtarzalność pomiarów i próg istotności różnic

Przed omawianiem wyników należy zaznaczyć, że zgodnie z metodyką przedstawioną w Sekcja 4.4, przeprowadzone badania dały bardzo obszerny zbiór wyników, który w całości załączono jako załącznik do niniejszej pracy. Zestaw ten zawiera 100 tabel, zbliżonych formą do prezentowanych w rozdziale. Tabele przedstawione poniżej wybrano spośród nich celowo, tak aby zilustrować najbardziej interesujące mechanizmy zaobserwowane w trakcie badań.

Jakość środowiska pomiarowego dla wersji sekwencyjnej, z danymi wejściowymi o 1048576 próbkach ilustruje tab. 5.1. Rozstęp względny większości wariantów nie przekraczał 3%. W załączonych wynikach tabela dla wersji współbieżnej pokazała bardzo zbliżony wzorzec. Rozstęp względny między pomiarami nie przekraczał tam 4%, co potwierdziło, że wysoka powtarzalność pomiarów nie była cechą specyficzną dla wariantów sekwencyjnych, lecz efektem zastosowanej metodyki.

Na tej podstawie można pokusić się o stwierdzenie, że różnice w czasie wykonania przekraczające 5% wynikały z rzeczywistego wpływu flag kompilacji lub decyzji implementacyjnych, a nie z szumu pomiarowego. Należy brać jednak pod uwagę, że w innych konfiguracjach macierzy testów, na przykład przy mniejszych rozmiarach danych wejściowych rozstęp względny bywał wyższy (ale wciąż nieprzekraczający 7%), co należy uwzględnić przy interpretacji różnic bliskich granicy.

Tabela 5.1: Rozstęp międzykwartylowy bezwzględny i względny dla GCC i 1048576 próbek, wersja sekwencyjna. Pogrubiono najmniejszą wartość w każdym wierszu.

Algorytm	00	01	02	03	0s	unroll	ffast	mcpu	ff+mcpu	+alias	lto
<i>Rozstęp międzykwartylowy IQR [ms]</i>											
radix2_v0	1,197	3,271	2,567	2,570	3,411	2,688	3,129	3,684	1,575	2,685	1,051
radix2_v1	15,923	1,872	0,540	1,279	1,603	1,593	1,415	1,197	1,634	1,720	1,336
radix2_v2_soa	5,861	2,353	3,665	2,685	8,621	3,282	3,947	2,987	3,155	2,241	2,272
radix2_v3_neon	4,969	5,419	4,200	4,928	5,496	6,389	3,962	1,507	2,235	3,438	6,759
radix4	5,112	3,821	4,789	6,146	4,149	3,889	4,990	3,929	3,847	2,535	3,556
radix4_naive	4,731	1,170	1,175	1,252	1,365	1,374	1,333	1,441	0,343	0,224	1,298
splitradix	11,586	6,939	5,438	10,603	12,877	4,317	7,192	7,439	4,638	3,148	11,708
stockham	4,809	0,706	0,432	0,729	1,280	0,223	0,826	0,952	0,240	0,493	0,172
<i>Rozstęp względny IQR/mediana [%]</i>											
radix2_v0	0,14	1,54	1,24	1,24	1,63	1,30	1,52	1,78	0,76	1,29	0,50
radix2_v1	2,49	0,88	0,26	0,61	0,76	0,77	0,68	0,58	0,78	0,82	0,63
radix2_v2_soa	1,24	1,11	1,75	1,29	4,15	1,55	1,82	1,43	1,46	1,04	1,07
radix2_v3_neon	1,18	2,55	1,99	2,34	2,56	3,06	1,86	0,71	1,05	1,60	3,18
radix4	0,79	1,26	1,71	2,20	1,51	1,47	1,62	1,45	1,28	0,84	1,25
radix4_naive	0,75	0,46	0,46	0,42	0,54	0,56	0,53	0,55	0,14	0,09	0,50
splitradix	1,55	1,09	0,91	1,78	2,14	0,72	1,14	1,34	0,77	0,52	1,95
stockham	1,55	0,41	0,25	0,80	0,76	0,24	0,91	1,04	0,26	0,54	0,19

5.2. Analiza wpływu optymalizacji programu sekwencyjnego

5.2.1. Wpływ poziomów optymalizacji kompilatora

Wyniki pomiarów czasu zestawiono w tab. 5.2. Największy i zarazem jedyny konsekwentny efekt dało odejście od poziomu -00: samo przejście na -01 skracało czas wykonania o **62-88%** pod GCC i o **46-87%** pod Clang, co odpowiada przyspieszeniu od 2,6 do 8,2 razy (tab. 5.3). Najsilniej reagowały warianty radix-2, najsłabiej splitradix i radix4_naive. Dalsze podnoszenie poziomu optymalizacji przyniosło już wyraźnie mniej - najlepszy zestaw flag skracał czas względem -01 o kolejne 0,5-29%, przy medianie około **12%**. Nie było przy tym zestawu, który byłby najlepszy dla wszystkich algorytmów: w szesnastu wierszach tabeli najkrótszy czas osiągało osiem różnych zestawów flag, a -03 nie wygrał w żadnym z nich. Wyjątkiem od ogólnego obrazu był radix4_naive, dla którego pod GCC poziomy -02 i -03 okazały się o około **40%** wolniejsze od -01. Wiersz fftw pozostał płaski w zakresie 0,3470-0,3764 ms, ponieważ biblioteka pochodziła z gotowego pakietu i nie podlegała badanym ustawieniom kompilacji.

Zróznicowanie reakcji poszczególnych algorytmów na przejście z -00 wynikać mogło z tego, że poziom ten, dla ułatwienia debugowania kodu, znacząco ogranicza przydział rejestrów procesora dla zmiennych, w taki sposób, aby można je było prosto podejrzeć ze stosu [10]. Z tego powodu, przy użyciu wyższego poziomu optymalizacji zyskały najbardziej warianty o zwartej pętli wewnętrznej i niewielkiej liczbie zmiennych żywych - pojedynczy etap algorytmu, taki jak radix-2, który operuje na kilku wartościach, które po włączeniu -01 mogą być trzymane bezpośrednio w rejestrach. Warianty splitradix i radix4_naive, o bardziej rozbudowanej strukturze algorytmu, spędzały poza właściwymi obliczeniami większą część czasu, którego przydział rejestrów nie skraca. Osobnego wyjaśnienia wymaga marginalny wpływ -03, poziom ten dodaje względem -02 przede wszystkim automatyczną wektoryzację, a ta powiodła się jedynie dla dwóch z ośmiu algorytmów pod GCC i jednego pod Clang, co widać w tab. 5.6. W pozostałych przypadkach kod wynikowy był niemal identyczny jak przy -02, a różnice w czasie wykonania mieściły się poniżej progu istotności przyjętego w Sekcja 5.1.

Tabela 5.2: Mediana czasu wykonania transformaty dla GCC i Clang, 16384 próbek, wersja sekwencyjna. Wartości w milisekundach. Pogrubiono najmniejszą wartość w każdym wierszu.

Algorytm	Kompilator	00	01	02	03	0s	unroll	ffast	mcpu	ff+mcpu	+alias	lto
radix2_v0	GCC	8,6189	1,0518	0,9270	0,9421	0,9592	0,8966	0,9259	0,9315	0,9156	0,9143	0,9405
	Clang	6,4565	0,9253	0,9261	0,9018	0,9199	0,9060	0,8185	0,7955	0,8049	0,8040	0,9320
radix2_v1	GCC	6,4426	0,9004	0,7562	0,7563	0,9076	0,7775	0,8442	0,8871	0,8472	0,8453	0,7858
	Clang	5,0313	0,6843	0,7063	0,7172	0,6778	0,7136	0,7277	0,7377	0,7551	0,7569	0,7188
radix2_v2_soa	GCC	4,2421	0,9567	0,8427	0,8594	0,9564	0,8606	0,8561	0,8834	0,8826	0,7111	0,9681
	Clang	4,1100	0,8651	0,6758	0,6579	0,8528	0,6484	0,6248	0,6239	0,6290	0,6153	0,7038
radix2_v3_neon	GCC	3,6592	0,5794	0,5624	0,5643	0,6118	0,5635	0,5656	0,5721	0,5840	0,6496	0,5872
	Clang	4,5329	0,6017	0,5995	0,6188	0,5986	0,6420	0,6603	0,6267	0,6265	0,6362	0,6335
radix4	GCC	4,6056	0,9789	0,9057	0,8940	0,9521	0,8850	0,9829	0,9307	0,9901	0,9858	0,9310
	Clang	3,7740	0,9248	0,9086	0,8852	0,8792	0,8759	0,9005	0,8518	0,8478	0,8480	0,8920
radix4_naive	GCC	6,0956	2,1677	3,0243	3,0293	2,2145	3,0125	2,0307	3,0414	1,9943	1,9947	2,1756
	Clang	5,1931	2,7980	2,7485	2,7394	2,7238	2,6721	2,1736	2,7164	2,1778	2,1781	2,6662
splitradix	GCC	3,1690	1,1991	1,0808	1,0852	1,1302	1,1023	1,1422	1,0561	1,1259	1,1261	1,1097
	Clang	3,2003	1,0451	1,0273	1,0389	1,0633	1,0200	0,9967	0,9883	1,0310	1,0090	1,0474
stockham	GCC	3,5101	0,6360	0,6060	0,6218	0,5544	0,6218	0,6022	0,6189	0,6149	0,6203	0,6245
	Clang	3,5632	0,4995	0,4456	0,4475	0,4702	0,4395	0,4433	0,4742	0,4478	0,4548	0,4296
fftw	GCC	0,3596	0,3764	0,3513	0,3704	0,3649	0,3514	0,3701	0,3730	0,3498	0,3679	0,3631
	Clang	0,3516	0,3535	0,3470	0,3625	0,3502	0,3570	0,3671	0,3659	0,3630	0,3622	0,3589

5.2.2. Wpływ celowych modyfikacji kodu programu, w tym wybór algorytmu

Wpływ celowych modyfikacji kodu przy niezmiennym schemacie dla algorytmu radix-2 zestawiono w tab. 5.2. Zastąpienie przeliczanych na bieżąco współczynników obrotu tablicą przekazaną jako argument, w przypadku (*radix2_v1*), skróciło czas wobec *radix2_v0* o 18% pod GCC i 24% pod Clang, a jednocześnie podniosło stosunek sygnału do szumu, co omówiono w Sekcja 5.2.4. Zmiana układu danych na strukturę tablic (*radix2_v2_soa*) dała wynik zależny od kompilatora: pod Clang skróciła czas do 0,6758 ms, czyli poniżej wariantu *radix2_v1*, natomiast pod GCC zatrzymała się na 0,8427 ms i pozostała wolniejsza od niego. Przyczyną było to, że sam układ danych nie przyspiesza obliczeń - umożliwia jedynie wektoryzację, którą w badanych warunkach przeprowadził skuteczniej kompilator Clang. Tłumaczyło to również zachowanie tego wariantu przy poziomach -01 i -0s, przy których czas wzrósł do 0,8651 i 0,8528 ms, a więc do wartości bliskiej *radix2_v0*: oba poziomy nie włączały wektoryzacji, więc korzyść z przebudowy układu danych zanikła. Najszybszy w tej grupie był wariant z jawnym użyciem instrukcji NEON (*radix2_v3_neon*) - 0,5624 ms pod GCC, czyli o 39% mniej niż *radix2_v0* i o 26% mniej niż *radix2_v1*. Warto jednak zauważyć, że pod Clang jego przewaga nad *radix2_v2_soa* wynosiła już tylko 11%, ponieważ kompilator ten wektoryzował układ SoA samodzielnie. Zestawienie wyników dla rodziny algorytmów radix-2 pokazało, że same modyfikacje kodu, przy niezmiennym schemacie obliczeń, skróciły czas wykonania o **39%**. Pozostałe badane warianty, takie jak *radix4*, *splitradix* oraz *stockham*, zaimplementowano wyłącznie w postaci bazowej, bez przebudowy układu danych, bez jawnego użycia instrukcji wektorowych czy innych technik optymalizacji wewnątrz programu. Korzyść uzyskana dla radix-2 może budzić nadzieję, że zastosowanie wobec nich analogicznych technik przyniosłoby wymierne efekty, a zmierzone tu czasy wykonania należy traktować jako punkt wyjścia, a nie jako granicę możliwości tych schematów. Weryfikacja tego przypuszczenia wykracza poza zakres niniejszej pracy i stanowi naturalny kierunek dalszych badań.

Skala zmian uzyskanych za pomocą jawnych technik optymalizacji w programie pozo-

stała mniejsza niż skala wynikająca z wyboru samego schematu obliczeń. W obrębie algorytmów radix-2, z pominięciem programów skompilowanych z flagą -O0, różnica między najlepszym a najgorszym wariantem nie przekraczała 65%. Dla algorytmów niestosujących schematu radix-2 uzyskany za to zysk przekroczył 24%, względem najszybszego wariantu. Zysk ten osiągnięto dla algorytmu stockham, którego najmniejszy czas wykonania wyniósł 0,4296 ms pod Clang.

Należy się również pochylić nad rozmiarami kodu wynikowego, zestawionymi w tab. 5.4. W jedenastu z szesnastu wierszy najmniejszą sekcję .text dawał poziom -O0s, przy czym różnice bywały znaczne: dla stockham pod Clang było to 216 bajtów wobec 1080 przy -O2, czyli pięciokrotnie mniej. Rozrost kodu towarzyszył wektoryzacji i rozwijaniu pętli, nie zawsze przynosząc korzyść w czasie - dla stockham pod GCC zestaw unroll powiększył sekcję z 320 do 1544 bajtów, pozostawiając czas na poziomie 0,6218 ms wobec 0,6060 ms przy -O2. Zdarzało się też przeciwnie: dla radix4_naive pod Clang zestaw ffast powiększył kod z 560 do 2208 bajtów i skrócił czas o 21%.

Tabela 5.3: Przyspieszenie względem -O0 dla GCC i 16384 próbek, wersja sekwencyjna. Wartości bezwymiarowe, poziom -O0 przyjęto za 1,00. Pogrubiono największą wartość w każdym wierszu.

Algorytm	O0	O1	O2	O3	O4	unroll	ffast	mcpu	ff+mcpu	+alias	lto
radix2_v0	1,00	8,19	9,30	9,15	8,99	9,61	9,31	9,25	9,41	9,43	9,16
radix2_v1	1,00	7,15	8,52	8,52	7,10	8,29	7,63	7,26	7,60	7,62	8,20
radix2_v2_soa	1,00	4,43	5,03	4,94	4,44	4,93	4,96	4,80	4,81	5,97	4,38
radix2_v3_neon	1,00	6,32	6,51	6,48	5,98	6,49	6,47	6,40	6,27	5,63	6,23
radix4	1,00	4,70	5,08	5,15	4,84	5,20	4,69	4,95	4,65	4,67	4,95
radix4_naive	1,00	2,81	2,02	2,01	2,75	2,02	3,00	2,00	3,06	3,06	2,80
splitradix	1,00	2,64	2,93	2,92	2,80	2,88	2,77	3,00	2,81	2,81	2,86
stockham	1,00	5,52	5,79	5,65	6,33	5,65	5,83	5,67	5,71	5,66	5,62

5.2.3. Porównanie kompilatorów GCC i Clang

Zestawienie kompilatorów GCC i Clang prowadziło do czytelnego podziału ról. Pod względem rozmiaru kodu przewagę miał GCC: choć mniejszą sekcję .text generował w 45 z 88 przypadków, a Clang w 42 (tab. 5.4), o wyniku decydowała skala różnic w pozostałych komórkach - średni rozmiar wynosił 518 B dla GCC i 724 B dla Clang, czyli o 40% więcej. Największe rozbieżności dotyczyły wariantów używających wektoryzacji: dla radix2_v1 przy -O2 było to 224 B wobec 840 B, a dla stockham 320 B wobec 1080 B.

Pod względem czasu wykonania relacja była znacząco inna. Clang osiągał krótszy czas w 63 z 80 konfiguracji przy 1024 i przy 16384 próbkach danych wejściowych, skracając go średnio o około **8,5%** (tab. 5.2). Przewaga ta zanikała jednak przy 1048576 próbkach wektora wejściowego, gdzie mediana skrócenia spadała do 0,5%, a liczba wygranych do 44 z 80. Wynikać to mogło z faktu, że przy rozmiarze tego rzędu o czasie wykonania decydował dostęp do pamięci, a nie jakość wygenerowanego kodu.

Źródłem przewagi Clang była skuteczniejsza wektoryzacja automatyczna. W tab. 5.6 instrukcje SIMD występowały w 38 z 80 konfiguracji dla Clang wobec 30 dla GCC, a różnica dotyczyła przede wszystkim wariantów, w których GCC nie wektoryzował w ogóle: dla radix2_v2_soa generował zero instrukcji wektorowych w dziewięciu z dziesięciu zestawów flag, podczas gdy Clang emitował ich osiem lub dziewięć w ośmiu zestawach. Clang zachowywał przy tym wektoryzację niezależnie od zestawu flag, natomiast GCC tracił ją po dodaniu -ffast-math lub -mcpu do poziomu -O2. Zagadnienie to omówiono szerzej w Sekcja 5.2.5.

Wybór między kompilatorami sprowadza się zatem do kompromisu między rozmiarem

kodu a czasem wykonania. W systemach wbudowanych wybór ten może być szczególnie istotny, ponieważ często pojemność pamięci programu bywa ograniczeniem twardszym niż wydajność.

Tabela 5.4: Rozmiar sekcji kodu wynikowego algorytmu dla GCC i Clang, wersja sekwencyjna. Rozmiar sekcji `.text` w bajtach. Pogrubiono najmniejszą wartość w każdym wierszu.

Algorytm	Kompilator	00	01	02	03	0s	unroll	ffast	mcpu	ff+mcpu	+alias	lto
radix2_v0	GCC	732	344	348	344	276	580	344	336	336	336	344
	Clang	628	292	292	280	276	280	272	280	280	280	280
radix2_v1	GCC	704	260	224	228	236	392	260	244	244	244	228
	Clang	604	236	840	852	224	852	848	864	860	860	852
radix2_v2_soa	GCC	784	272	268	272	260	460	272	264	264	1032	272
	Clang	652	288	712	740	264	740	736	772	768	768	740
radix2_v3_neon	GCC	1448	404	384	564	360	832	564	560	560	1068	564
	Clang	1396	444	1048	1536	412	1536	1532	1528	1528	1528	1536
radix4	GCC	1216	476	452	456	428	456	472	440	468	468	456
	Clang	1068	416	416	384	416	384	384	396	396	396	384
radix4_naive	GCC	1188	536	512	512	500	512	572	524	572	572	512
	Clang	1080	560	560	548	548	548	2208	552	2232	2232	548
splitradix	GCC	1444	580	528	528	532	528	540	532	544	544	528
	Clang	1392	464	464	464	448	464	464	476	476	476	464
stockham	GCC	764	320	320	684	208	1544	620	668	668	668	684
	Clang	716	268	1080	852	216	852	840	1092	868	868	852

5.2.4. Wpływ kompilatora na błędy numeryczne wybranych algorytmów

Dokładność wyników dla 16384 próbek zestawiono w tab. 5.5. Zakres, w jakim kompilator wpływał na wynik, był wąski - pomijając zestawy z `-ffast-math`, rozpiętość SNR w obrębie wiersza nie przekraczała 1,31 dB, a same te zestawy zmieniały go najwyżej o 0,45 dB, przy czym dla `splitradix` i `radix4_naive` nieznacznie go podnosiły.

Wyjątkiem był `radix2_v0`, który osiągał **91,52 dB** wobec 134,89-138,48 dB dla pozostałych wariantów. Różnica wynikała ze sposobu wyznaczania współczynników obrotu w tej implementacji, opisanego w Sekcja 4.2.3. Kolejne współczynniki powstają przez mnożenie poprzedniej wartości przez stały czynnik, więc błąd zaokrąglenia każdego kroku wchodzi do kroku następnego i narasta wraz z liczbą etapów. Potwierdziło to porównanie rozmiarów - SNR spadał ze 113,41 dB przy 1024 próbkach do 75,14 dB przy 1048576, podczas gdy pozostałe warianty traciły w tym zakresie najwyżej 4 dB.

Wynik ten pokazał, że kompilator jest narzędziem tłumaczącym kod, a nie mechanizmem korygującym jego wady. Żaden poziom optymalizacji ani zestaw flag nie zmniejszył błędu wynikającego z konstrukcji algorytmu; skuteczna okazała się dopiero zmiana implementacji - `radix2_v1` przy tym samym schemacie obliczeń osiągał 136,71 dB. Obserwacja ta jest istotna dla codziennej praktyki w pracy z oprogramowaniem. Kompilator nie poprawi błędów merytorycznych w kodzie programu komputerowego i nie zwiększy jego jakości.

Tabela 5.5: Błąd średniokwadratowy i stosunek sygnału do szumu dla GCC i 16384 próbek, wersja sekwencyjna. Pogrubiono najmniejszy błąd i największy SNR w każdym wierszu.

Algorytm	00	01	02	03	0s	unroll	ffast	mcpu	ff+mcpu	+alias	lto
<i>Błąd średniokwadratowy RMSE</i>											
radix2_v0 ($\cdot 10^{-3}$)	1,714	1,714	2,595	2,595	2,595	2,595	2,594	2,595	2,594	2,594	2,595
radix2_v1 ($\cdot 10^{-5}$)	1,377	1,377	1,429	1,429	1,429	1,429	1,444	1,429	1,444	1,444	1,429
radix2_v2_soa ($\cdot 10^{-5}$)	1,377	1,377	1,429	1,429	1,429	1,429	1,444	1,429	1,444	1,444	1,429
radix2_v3_neon ($\cdot 10^{-5}$)	1,445	1,445	1,467	1,467	1,467	1,467	1,542	1,467	1,542	1,542	1,467
radix4 ($\cdot 10^{-5}$)	1,365	1,365	1,174	1,174	1,174	1,174	1,236	1,174	1,236	1,236	1,174
radix4_naive ($\cdot 10^{-5}$)	1,760	1,760	1,610	1,610	1,610	1,610	1,571	1,610	1,571	1,571	1,610
splitradix ($\cdot 10^{-5}$)	1,276	1,276	1,280	1,280	1,280	1,280	1,165	1,280	1,165	1,165	1,280
stockham ($\cdot 10^{-5}$)	1,342	1,342	1,387	1,387	1,387	1,387	1,429	1,387	1,429	1,429	1,387
<i>Stosunek sygnału do szumu SNR [dB]</i>											
radix2_v0	95,12	95,12	91,52	91,52	91,52	91,52	91,53	91,52	91,53	91,53	91,52
radix2_v1	137,03	137,03	136,71	136,71	136,71	136,71	136,61	136,71	136,61	136,61	136,71
radix2_v2_soa	137,03	137,03	136,71	136,71	136,71	136,71	136,61	136,71	136,61	136,61	136,71
radix2_v3_neon	136,61	136,61	136,48	136,48	136,48	136,48	136,04	136,48	136,04	136,04	136,48
radix4	137,10	137,10	138,41	138,41	138,41	138,41	137,96	138,41	137,96	137,96	138,41
radix4_naive	134,89	134,89	135,67	135,67	135,67	135,67	135,88	135,67	135,88	135,88	135,67
splitradix	137,69	137,69	137,66	137,66	137,66	137,66	138,48	137,66	138,48	138,48	137,66
stockham	137,25	137,25	136,96	136,96	136,96	136,96	136,70	136,96	136,70	136,70	136,96

5.2.5. Wpływ wybranych flag na wektoryzację

Liczbę instrukcji zmiennoprzecinkowych w kodzie wynikowym zestawiono w tab. 5.6; są to statyczne wystąpienia instrukcji, a nie liczba ich wykonań. Automatyczna wektoryzacja przez kompilator nie okazała się zjawiskiem powszechnym - pomijając -00, wystąpiła tylko w 30 z 80 konfiguracji pod GCC i w 38 pod Clang, a poziomy -01 oraz -0s nie włączały jej w ogóle.

Decydowała o niej nie poziom optymalizacji, lecz pojedyncza flaga. Pod GCC wariant radix2_v2_soa został zwektoryzowany wyłącznie w zestawie +alias, znoszącym założenie o możliwym nakładaniu się buforów, i właśnie tam osiągnął najlepszy czas - 0,7111 ms wobec 0,8427 ms przy -02 (tab. 5.2). Zależność działała też odwrotnie: dla radix2_v1 pod GCC dodanie -ffast-math lub -mcpu usuwało wektoryzację obecną przy -02, wydłużając czas z 0,7562 do 0,8442 ms.

Tłumaczyło to znikomy zysk z użycia flagi -03. W użytej wersji GCC wektoryzacja działała już przy -02, a -03 zmieniał jedynie model kosztu na mniej zachowawczy, więc skutek widać było tylko tam, gdzie decyzja o opłacalności była graniczna, na przykład dla algorytmu stockham liczba instrukcji SIMD rosła wtedy z zera do siedmiu, a w pozostałych ich liczba pozostawała bez zmian.

Tabela 5.6: Liczba statycznych instrukcji zmiennoprzecinkowych w kodzie wynikowym dla GCC i Clang, wersja sekwencyjna. Format komórki: liczba instrukcji wektorowych (SIMD), w nawiasie liczba instrukcji skalarnych. Pogrubiono komórki, w których wystąpiła wektoryzacja.

Algorytm	Komp.	00	01	02	03	0s	unroll	ffast	mcpu	ff+mcpu	+alias	lto
radix2_v0	GCC	0 (11)	0 (17)	0 (13)	0 (13)	0 (13)	0 (36)	0 (13)	0 (13)	0 (13)	0 (13)	0 (13)
	Clang	0 (9)	0 (13)	0 (13)	0 (13)	0 (13)	0 (13)	0 (13)	0 (13)	0 (13)	0 (13)	0 (13)
radix2_v1	GCC	0 (10)	0 (10)	5 (1)	5 (1)	0 (8)	15 (3)	0 (8)	0 (8)	0 (8)	0 (8)	5 (1)
	Clang	0 (8)	0 (8)	9 (8)	9 (8)	0 (8)	9 (8)	8 (8)	9 (8)	8 (8)	8 (8)	9 (8)
radix2_v2_soa	GCC	0 (10)	0 (10)	0 (8)	0 (8)	0 (8)	0 (24)	0 (8)	0 (8)	0 (8)	16 (16)	0 (8)
	Clang	0 (8)	0 (8)	9 (8)	9 (8)	0 (8)	9 (8)	8 (8)	9 (8)	8 (8)	8 (8)	9 (8)
radix2_v3_neon	GCC	9 (10)	8 (10)	8 (8)	8 (24)	8 (8)	32 (24)	8 (24)	8 (24)	8 (24)	16 (32)	8 (24)
	Clang	9 (8)	8 (8)	17 (8)	17 (32)	8 (8)	17 (32)	16 (32)	17 (32)	16 (32)	16 (32)	17 (32)
radix4	GCC	0 (22)	0 (34)	0 (28)	0 (28)	0 (28)	0 (28)	0 (28)	0 (28)	0 (28)	0 (28)	0 (28)
	Clang	0 (20)	0 (28)	0 (28)	0 (28)	0 (28)	0 (28)	0 (28)	0 (28)	0 (28)	0 (28)	0 (28)
radix4_naive	GCC	0 (33)	0 (40)	0 (34)	0 (34)	0 (34)	0 (34)	0 (34)	0 (34)	0 (34)	0 (34)	0 (34)
	Clang	0 (31)	0 (34)	0 (34)	0 (34)	0 (34)	0 (34)	31 (34)	0 (34)	31 (34)	31 (34)	0 (34)
splitradix	GCC	0 (29)	0 (28)	2 (20)	2 (20)	0 (24)	2 (20)	2 (23)	2 (20)	2 (23)	2 (23)	2 (20)
	Clang	0 (25)	0 (24)	0 (24)	0 (24)	0 (24)	0 (24)	0 (24)	0 (24)	0 (24)	0 (24)	0 (24)
stockham	GCC	0 (10)	0 (10)	0 (8)	7 (16)	0 (8)	49 (64)	9 (12)	7 (16)	7 (16)	7 (16)	7 (16)
	Clang	0 (8)	0 (8)	14 (0)	14 (0)	5 (0)	14 (0)	14 (0)	24 (0)	14 (0)	14 (0)	14 (0)

5.2.6. Wpływ rozmiaru danych wejściowych

Rozmiary danych wejściowych dobrano tak, aby ich wielkość pozwoliła na sprawdzenie dostępu do kolejnych poziomów pamięci podręcznej badanej platformy sprzętowej. Przy 1024 próbkach oba bufor zajmowały 16 kB i mieściły się w pamięci L1D o pojemności 32 kB; przy 16384 próbkach było to 256 kB, a więc powyżej L1D, lecz w granicach 1 MB pamięci L2; przy 1048576 próbkach — 16 MB, czyli szesnastokrotnie ponad pojemność L2.

Skutek widać w tab. 5.9 i tab. 5.8. Współczynnik chybień L2 rósł z wartości nieprzekraczających 0,7% przy 16384 próbkach do **23–35%** przy 1048576. Ponieważ miara ta odnosi się do odwołań, które chybiły już w L1, oznaczało to, że co trzecie z nich kończyło się sięgnięciem do pamięci operacyjnej. Przełożyło się to bezpośrednio na stosunek czasu wykonania do wielkości danych wejściowych. Miara ta uwzględnia już wzrost liczby etapów z 14 do 20, zatem pojedyncza operacja motylkowa kosztowała przy większym rozmiarze danych ponad dwukrotnie więcej czasu, mimo że wykonywała dokładnie tyle samo działań arytmetycznych. Obserwowane wydłużenie wynikało więc z czasu dostępu do danych.

Skala chybień w pamięci podręcznej zależała od wzorca dostępu do danych. Najgorzej wypadł `splitradix`: 19,10% chybień L1D i koszt jednostkowy **28,50 ns** wobec 4,71 ns przy 16384 próbkach. Dla porównania `stockham`, przechodzący przez tablicę liniowo, notował 1,47% chybień i **8,13 ns**.

Zachowanie `splitradix` pokazało, że o czasie wykonania decydowała nie liczba chybień, lecz ich koszt. Wariant ten notował wysokie chybień L1D niezależnie od rozmiaru danych - 15,52% przy 16384 próbkach i 19,10% przy 1048576, wobec 1,42% i 1,08% dla `radix2_v0`. Wynikać to mogło z samego schematu obliczeń, opartego na podciągach o indeksach $4k+1$ i $4k+3$, a więc na odwołaniach z krokiem co cztery. Przy 16384 próbkach obsługiwała je jednak pamięć L2 i strata wobec `radix2_v0` wynosiła 17%; przy 1048576, gdy chybień L2 sięgały 26,75%, te same odwołania kierowane były do pamięci operacyjnej, a strata zdecydowanie rosła.

Jednocześnie zanikały różnice między wariantami tego samego schematu. Cztery im-

plementacje radix-2 dzieliło **65%** przy 16384 próbkach i zaledwie **2,1%** przy 1048576 (tab. 5.2, tab. 5.7). Powyżej pojemności L2 o wydajności decydowała zatem organizacja dostępu do pamięci, a nie jakość wygenerowanego kodu. Wniosek ten może być niezwykle istotny przy organizacji systemów, których zadaniem jest przetwarzanie bardzo dużych zbiorów danych.

Odczytując tab. 5.8, należy zachować ostrożność wobec kolumny –00. Chybień L1D wynosiły tam 0,14–2,69%, czyli kilkukrotnie mniej niż na wyższych poziomach optymalizacji, co nie świadczyło o lepszej organizacji dostępu do pamięci. Kod programu bez optymalizacji wykonywał wiele nadmiarowych odwołań do pamięci trafiających do L1D, które powiększały mianownik współczynnika badanego stosunku.

Tabela 5.7: Mediana czasu wykonania transformaty dla GCC i 1048576 próbek, wersja sekwencyjna. Wartości w milisekundach. Pogrubiono najmniejszą wartość w każdym wierszu.

Algorytm	00	01	02	03	0s	unroll	ffast	mcpu	ff+mcpu	+alias	lto
radix2_v0	834,30	212,17	206,54	206,60	209,55	207,53	206,52	207,13	207,44	207,72	208,17
radix2_v1	638,30	211,61	208,53	208,09	212,02	207,28	209,20	207,36	209,96	210,51	212,75
radix2_v2_soa	473,97	211,86	209,70	208,33	207,83	212,03	216,72	208,50	216,78	215,87	212,16
radix2_v3_neon	419,65	212,35	210,82	210,87	214,28	208,78	212,63	211,66	213,72	214,83	212,40
radix4	647,67	304,00	280,17	279,04	275,54	263,93	307,58	271,54	301,06	301,80	283,62
radix4_naive	632,95	254,73	257,96	254,87	254,23	246,92	251,88	256,21	246,13	245,51	258,40
splitradix	745,81	634,16	597,59	596,89	602,57	601,41	630,62	553,78	605,62	600,52	600,61
stockham	310,10	171,62	170,54	91,42	167,89	92,67	90,69	91,51	91,50	91,78	91,27
fftw	52,34	52,36	52,73	52,69	52,42	52,52	52,41	53,04	52,45	52,56	52,40

Tabela 5.8: Współczynnik chybień pamięci podręcznej L1D i L2 dla GCC i 1048576 próbek, wersja sekwencyjna. Pogrubiono najmniejszą wartość w każdym wierszu z pominięciem –00.

Algorytm	00	01	02	03	0s	unroll	ffast	mcpu	ff+mcpu	+alias	lto
<i>Chybień L1D [%]</i>											
radix2_v0	0,14	1,06	1,08	1,06	1,07	1,07	1,08	1,08	1,07	1,09	1,08
radix2_v1	0,26	1,12	1,94	2,01	1,10	2,00	1,11	1,14	1,12	1,14	2,17
radix2_v2_soa	0,52	2,09	2,15	1,98	2,13	1,87	1,90	1,98	1,84	4,52	2,53
radix2_v3_neon	0,85	4,76	5,43	4,87	4,94	5,03	5,03	5,08	5,21	5,09	6,03
radix4	1,33	10,83	10,91	11,24	11,32	10,81	11,06	11,82	10,87	10,85	11,78
radix4_naive	0,43	1,82	2,37	1,74	2,55	2,12	1,72	1,63	1,82	1,60	1,82
splitradix	2,69	18,06	19,10	18,97	18,93	18,97	18,70	18,86	18,88	19,04	19,07
stockham	0,76	3,24	3,18	1,51	5,63	1,57	1,47	1,44	1,57	1,39	1,41
<i>Chybień L2 [%]</i>											
radix2_v0	33,71	34,74	34,89	35,25	34,94	34,90	34,70	34,86	34,87	34,85	34,90
radix2_v1	31,46	31,38	31,15	30,77	31,53	31,03	31,34	30,99	31,49	31,29	30,76
radix2_v2_soa	26,39	30,00	29,04	29,29	29,25	30,86	30,95	30,38	31,90	30,56	26,82
radix2_v3_neon	24,48	31,44	29,53	31,23	31,08	30,34	31,02	30,72	30,62	30,54	28,10
radix4	27,55	25,42	27,98	27,35	26,01	27,73	28,52	25,63	27,83	28,77	26,59
radix4_naive	23,01	25,40	24,23	26,53	23,82	24,28	26,15	23,16	24,95	26,41	25,85
splitradix	27,73	28,77	26,75	25,19	25,56	26,01	26,94	26,17	25,72	25,48	28,34
stockham	28,89	32,79	33,09	26,19	32,65	26,24	26,20	26,29	26,03	26,24	26,34

Tabela 5.9: Współczynnik chybień pamięci podręcznej L1D i L2 dla GCC i 16384 próbek, wersja sekwencyjna. Pogrubiono najmniejszą wartość w każdym wierszu z pominięciem -00.

Algorytm	00	01	02	03	0s	unroll	ffast	mcpu	ff+mcpu	+alias	lto
<i>Chybienia L1D [%]</i>											
radix2_v0	0,19	1,44	1,42	1,46	1,42	1,48	1,43	1,47	1,45	1,38	1,47
radix2_v1	0,30	1,45	2,68	2,71	1,36	2,63	1,35	1,32	1,45	1,43	2,35
radix2_v2_soa	0,84	2,17	2,83	1,97	2,70	2,65	2,28	2,15	2,57	4,33	2,73
radix2_v3_neon	0,79	4,99	4,88	5,27	5,68	4,93	5,20	5,45	5,19	4,73	5,63
radix4	0,83	6,59	7,51	6,95	6,87	6,80	6,96	7,29	7,25	6,29	6,94
radix4_naive	0,24	1,33	1,44	1,96	1,48	1,20	1,49	1,05	1,54	1,57	1,64
splitradix	2,07	14,12	15,52	15,46	14,19	15,02	14,85	15,49	14,39	15,48	15,72
stockham	0,73	2,75	2,91	1,62	4,73	1,56	1,86	1,42	1,64	1,58	1,46
<i>Chybienia L2 [%]</i>											
radix2_v0	0,07	0,23	0,22	0,00	0,16	0,14	0,04	0,07	0,24	0,26	0,22
radix2_v1	0,55	0,35	0,31	0,25	0,54	0,61	0,58	0,31	0,19	0,64	0,29
radix2_v2_soa	0,23	0,42	0,13	0,42	0,59	0,28	0,27	0,31	0,17	0,30	0,32
radix2_v3_neon	0,22	0,47	0,12	0,27	0,04	0,11	0,17	0,27	0,17	0,25	0,37
radix4	0,18	0,16	0,17	0,27	0,18	0,12	0,11	0,20	0,08	0,05	0,47
radix4_naive	0,17	0,10	0,10	0,11	0,30	0,15	0,12	0,17	0,12	0,16	0,28
splitradix	0,17	0,09	0,21	0,02	0,11	0,07	0,11	0,28	0,14	0,13	0,21
stockham	0,39	0,14	0,41	0,45	0,43	0,32	0,32	0,46	0,48	0,34	0,67

5.3. Analiza wpływu optymalizacji programu wielowątkowego

Znaczna część wniosków z analizy wariantów sekwencyjnych pozostała prawidłowa również dla wersji wielowątkowych. Mechanizmy działania poszczególnych flag opisano już przy wariantach sekwencyjnych i pozostały one bardzo zbliżone. Mechanizm ten mógł występować, ponieważ kompilator przetwarzał identyczny kod obliczeniowy niezależnie od liczby wątków. Dalsza analiza skupia się więc na zjawiskach specyficznych dla programów zrównoleglonych.

Bardzo podobne wnioski uzyskano przy porównaniu kompilatorów. Clang ponownie uzyskał nieco krótsze czasy wykonania niż GCC w większości konfiguracji, jednak przewaga ta miała tę samą naturę co w części sekwencyjnej. Jednocześnie GCC uzyskał mniejsze kody programu, podobnie jak w porównaniu wersji sekwencyjnych. Z tego powodu tabele omawiane w prezentowanej sekcji dotyczą GCC, a odpowiadające im wyniki dla Clanga znajdują się w załączniku do niniejszej pracy. Wyniki te nie zmieniły formułowanych wniosków. Wybór GCC jako podstawy analizy wynika z jego roli. Jest on domyślnym kompilatorem dla użytej dystrybucji systemu.

Tabela 5.10: Mediana czasu wykonania transformaty dla GCC i 1048576 próbek, wersja współbieżna. Wartości w milisekundach. Pogrubiono najmniejszą wartość w każdym wierszu.

Algorytm (wątki)	00	01	02	03	0s	unroll	ffast	mcpu	ff+mcpu	+alias	lto
radix2_par (1)	637,42	211,43	209,69	211,15	212,29	212,82	209,59	211,09	209,59	211,45	210,77
radix2_par (2)	415,02	197,09	196,34	196,99	198,30	196,15	199,48	197,21	195,69	196,75	197,49
radix2_par (4)	288,64	182,77	182,70	181,09	183,14	182,01	181,97	182,29	182,42	182,42	181,37
radix2_par (8)	312,27	204,87	203,18	202,12	204,69	201,27	202,64	204,96	204,12	203,77	202,07
radix2_par_v2 (1)	709,30	213,34	210,20	210,46	211,23	209,57	208,53	210,26	208,95	210,87	210,78
radix2_par_v2 (2)	397,30	202,39	204,92	203,84	205,25	203,54	203,71	204,52	204,42	204,86	204,91
radix2_par_v2 (4)	245,13	179,38	178,99	178,37	181,66	179,40	179,22	179,37	180,69	181,30	179,91
radix2_par_v2 (8)	268,34	188,42	187,55	186,44	189,79	185,36	187,31	188,90	188,74	188,35	188,76
radix4_par (1)	654,86	301,52	279,47	283,32	294,55	270,43	298,29	279,77	307,36	306,00	283,99
radix4_par (2)	383,40	217,00	211,01	215,11	218,61	195,09	217,98	216,45	221,98	223,15	210,99
radix4_par (4)	276,50	182,69	182,53	183,74	187,90	166,68	184,60	186,89	188,05	189,99	180,58
radix4_par (8)	286,94	190,93	189,87	192,42	193,49	177,01	194,49	190,89	200,64	198,21	187,17

5.3.1. Wpływ flag kompilacji na warianty równoległe

Wyniki z przeprowadzonych badań nad wariantami równoległymi, zaprezentowane w tab. 5.11, pokazały, że korzyść ze zrównoleglenia nie zmieniała się drastycznie w zależności od poziomu optymalizacji powyżej -O0. Dla 1048576 próbek i czterech wątków `radix2_par_v2` osiągał od 1,16 do 1,19 we wszystkich jedenastu zestawach flag, przy 2,89 dla -O0. Ten sam układ powtarzał się w pozostałych algorytmach i przy mniejszych rozmiarach danych, gdzie przy 4096 próbkach wartości spadały poniżej jedności - wersja czterowątkowa była wtedy wolniejsza od jednowątkowej. Wynik ten nie był jednak zadziwiający, ponieważ przy tak niewielkich danych wejściowych czas wykonania był krótki, a udział czasu koniecznego do tworzenia i synchronizacji wątków stanowił znaczną część całości.

Bezwzględne czasy wykonania z tab. 5.10 ilustrują przyczynę omawianej różnicy. Dla `radix2_par` przy -O2 czas przetwarzania spadał z 209,69 ms dla jednego wątku do 182,70 ms dla czterech. Przy -O0 ten sam algorytm potrzebował 637,42 ms na jednym wątku i 288,64 ms na czterech. W obu przypadkach rozdzielenie pracy między cztery rdzenie skracało czas o podobną wartość, ale wersja bez optymalizacji była ponad trzykrotnie wolniejsza, więc ta sama oszczędność dawała wyższe przyspieszenia. Warianty ośmiowątkowe były wolniejsze od czterowątkowych w każdej konfiguracji, co wynikało prawdopodobnie z faktu, że platforma sprzętowa jest wyposażona w cztery rdzenie procesora.

Brak zauważalnego wpływu poziomu optymalizacji na zysk ze zrównoleglenia skłonił do zbadania różnic w kodzie wynikowym sekcji `.text` obu wariantów. Porównanie plików obiektowych analogicznego wariantu sekwencyjnego `radix2_v1` i jego odpowiednika współbieżnego `radix2_par`, skompilowanych przez Clang z -O3, wykazało bardzo zbliżony schemat obliczeń - tę samą sekwencję instrukcji zmiennoprzecinkowych oraz ten sam blok zwektoryzowany, oparty na instrukcjach `ld2` i `st2`. Wariant równoległy zawierał dodatkowo wywołania biblioteki `pthread`, odpowiadające za utworzenie wątków, ich synchronizację barierą po każdym etapie transformaty oraz oczekiwanie na ich zakończenie. Przyjmowały one postać instrukcji skoku `b1` do symboli zewnętrznych, rozwiązywanych dopiero na etapie linkowania, a ich treść znajdowała się w prekompilowanej bibliotece systemowej. Koszt ich wykonania pozostawał zatem niezależny od badanych flag kompilacji.

5.3.2. Skalowanie z liczbą wątków i wpływ rozmiaru danych

Z przeprowadzonych badań można zauważyć, że przyspieszenie względem wariantu jednowątkowego osiągało maksimum przy czterech wątkach i malało dla większej ich liczby. Dla 1048576 próbek i poziomu -O2 wariant `radix2_par_v2` uzyskał 1,03 na dwóch wątkach, 1,17 na czterech i 1,12 na ośmiu, co zaprezentowano w tab. 5.11. Rozmiar danych wejściowych zmieniał wyniki silniej niż liczba wątków: ten sam wariant na czterech wątkach osiągał 0,51 przy 4096 próbkach, 1,63 przy 65536 oraz 1,17 przy 1048576.

Stopień wykorzystania rdzeni opisuje stosunek czasu procesora do czasu ściany (oznaczony jako R), zestawiony w tab. 5.12. Dla `radix2_par_v2` przy -O2 wynosił on 3,27 na czterech wątkach i 3,02 na ośmiu, mimo dwukrotnie większej liczby wątków. Zwiększenie ich liczby ponad liczbę rdzeni fizycznych nie podniosło zatem w bardzo znaczący sposób tej metryki. Liczba wykonanych instrukcji pozostawała niemal stała, w obrębie tych samych flag, we wszystkich konfiguracjach wątkowych, co wskazywało, że podział pracy nie zmieniał objętości obliczeń, a jedynie ich organizację.

Tabela 5.11: Przyspieszenie względem jednego wątku dla GCC i trzech rozmiarów danych, wersja współbieżna. Wartości w krotnościach. Pogrubiono największą wartość w każdym wierszu z pominięciem -00.

Algorytm (wątki)	00	01	02	03	0s	unroll	ffast	mcpu	ff+mcpu	+alias	lto
<i>4096 próbek</i>											
radix2_par (2)	1,30	0,59	0,58	0,58	0,63	0,59	0,61	0,63	0,61	0,61	0,57
radix2_par (4)	1,63	0,47	0,45	0,44	0,51	0,47	0,48	0,48	0,49	0,49	0,44
radix2_par (8)	0,94	0,22	0,20	0,19	0,23	0,20	0,21	0,22	0,21	0,21	0,19
radix2_par_v2 (2)	1,56	0,75	0,64	0,63	0,74	0,62	0,73	0,74	0,74	0,74	0,63
radix2_par_v2 (4)	2,17	0,61	0,51	0,50	0,62	0,49	0,58	0,62	0,60	0,60	0,50
radix2_par_v2 (8)	1,16	0,26	0,20	0,20	0,27	0,20	0,25	0,26	0,26	0,26	0,21
radix4_par (2)	1,36	0,73	0,71	0,73	0,73	0,71	0,74	0,73	0,73	0,75	0,70
radix4_par (4)	1,61	0,54	0,54	0,57	0,55	0,54	0,57	0,58	0,60	0,57	0,54
radix4_par (8)	0,88	0,25	0,24	0,25	0,26	0,24	0,26	0,26	0,27	0,26	0,25
<i>65536 próbek</i>											
radix2_par (2)	1,57	1,20	1,30	1,29	1,30	1,34	1,36	1,36	1,33	1,36	1,26
radix2_par (4)	2,40	1,51	1,57	1,51	1,60	1,54	1,54	1,52	1,60	1,64	1,48
radix2_par (8)	1,90	1,00	1,01	1,05	1,06	1,04	1,03	1,12	1,08	1,09	1,04
radix2_par_v2 (2)	1,83	1,44	1,38	1,41	1,47	1,36	1,46	1,49	1,49	1,46	1,38
radix2_par_v2 (4)	3,20	1,94	1,63	1,69	1,90	1,67	1,89	2,01	1,91	1,92	1,66
radix2_par_v2 (8)	2,54	1,25	1,10	1,12	1,27	1,11	1,24	1,34	1,28	1,29	1,11
radix4_par (2)	1,73	1,25	1,22	1,25	1,20	1,28	1,25	1,23	1,27	1,23	1,22
radix4_par (4)	2,48	1,28	1,24	1,26	1,21	1,30	1,25	1,21	1,26	1,28	1,23
radix4_par (8)	1,98	1,12	1,06	1,09	1,04	1,08	1,08	1,05	1,08	1,09	1,06
<i>1048576 próbek</i>											
radix2_par (2)	1,54	1,07	1,07	1,07	1,07	1,08	1,05	1,07	1,07	1,07	1,07
radix2_par (4)	2,21	1,16	1,15	1,17	1,16	1,17	1,15	1,16	1,15	1,16	1,16
radix2_par (8)	2,04	1,03	1,03	1,04	1,04	1,06	1,03	1,03	1,03	1,04	1,04
radix2_par_v2 (2)	1,79	1,05	1,03	1,03	1,03	1,03	1,02	1,03	1,02	1,03	1,03
radix2_par_v2 (4)	2,89	1,19	1,17	1,18	1,16	1,17	1,16	1,17	1,16	1,16	1,17
radix2_par_v2 (8)	2,64	1,13	1,12	1,13	1,11	1,13	1,11	1,11	1,11	1,12	1,12
radix4_par (2)	1,71	1,39	1,32	1,32	1,35	1,39	1,37	1,29	1,38	1,37	1,35
radix4_par (4)	2,37	1,65	1,53	1,54	1,57	1,62	1,62	1,50	1,63	1,61	1,57
radix4_par (8)	2,28	1,58	1,47	1,47	1,52	1,53	1,53	1,47	1,53	1,54	1,52

5.3.3. Wpływ sposobu podziału pracy i schematu algorytmu

Przy zwiększaniu rozmiarów danych wejściowych coraz lepsze przyspieszenie uzyskiwał wariant `radix4_par`. Dla 1048576 próbek i poziomu -02 osiągał **1,53** na czterech wątkach, wobec 1,15 i 1,17 dla wariantów `radix-2` (tab. 5.11). Przy 4096 próbkach relacja ta zanikała - wszystkie trzy implementacje pozostawały poniżej jedności.

Zysk ze stosowania `radix4_par` można wiązać z mniejszą liczbą etapów transformaty, a tym samym z mniejszą liczbą synchronizacji wykorzystujących barierę. Potwierdziły to czasy bezwzględne z tab. 5.10: na jednym wątku wariant ten pozostawał najwolniejszy (279,47 ms wobec 209,69 ms dla `radix2_par` przy -02), natomiast na czterech wątkach różnica względem wariantów `radix-2` niemal zanikała. Schemat podziału pracy między wątki pozostał przy tym identyczny we wszystkich trzech implementacjach, co pozwala przypisać obserwowane różnice właściwościom samego algorytmu.

Tabela 5.12: Wykonane instrukcje, liczba instrukcji na cykl oraz stosunek czasu procesora do czasu ściany dla GCC i 1048576 próbek, wersja współbieżna. Pogrubiono najmniejszą liczbę instrukcji oraz największe IPC i R w każdym wierszu z pominięciem -00.

Algorytm (wątki)	00	01	02	03	0s	unroll	ffast	mcpu	ff+mcpu	+alias	lto
<i>Wykonane instrukcje [mln]</i>											
radix2_par (1)	1345,3	332,3	224,3	224,2	315,5	226,3	308,2	308,6	308,7	308,7	224,2
radix2_par (2)	1443,1	396,6	224,6	224,6	327,4	226,7	308,5	309,0	309,0	309,1	224,7
radix2_par (4)	1443,6	397,3	225,2	225,2	328,1	227,4	309,2	309,8	309,6	309,7	225,3
radix2_par (8)	1445,1	398,7	226,8	226,7	329,5	228,7	310,7	311,2	311,2	311,2	226,8
radix2_par_v2 (1)	1425,1	398,3	260,0	260,0	388,8	249,4	375,2	375,7	375,8	375,8	261,0
radix2_par_v2 (2)	1425,4	398,8	260,4	260,3	389,3	250,0	375,7	376,3	376,3	376,2	261,5
radix2_par_v2 (4)	1425,7	399,4	261,0	261,0	390,1	250,8	376,3	376,8	377,0	377,1	262,1
radix2_par_v2 (8)	1427,3	400,9	262,3	262,3	391,3	251,9	377,6	378,1	378,2	378,2	263,4
radix4_par (1)	1080,4	307,3	284,1	284,0	291,4	257,9	294,6	297,7	308,2	308,2	284,1
radix4_par (2)	1080,6	307,4	284,4	284,4	291,8	258,2	294,9	298,0	308,5	308,5	284,4
radix4_par (4)	1080,9	307,9	284,9	284,9	292,2	258,6	295,3	298,5	309,0	309,2	284,8
radix4_par (8)	1082,0	308,8	285,8	285,9	293,2	259,5	296,4	299,5	310,0	310,0	285,8
<i>Liczba instrukcji na cykl (IPC)</i>											
radix2_par (1)	1,171	0,846	0,578	0,583	0,792	0,566	0,783	0,799	0,785	0,766	0,575
radix2_par (2)	1,111	0,645	0,363	0,368	0,534	0,370	0,501	0,500	0,500	0,488	0,361
radix2_par (4)	1,067	0,394	0,221	0,230	0,313	0,223	0,291	0,307	0,344	0,307	0,226
radix2_par (8)	1,014	0,381	0,219	0,218	0,306	0,216	0,286	0,288	0,296	0,303	0,216
radix2_par_v2 (1)	1,107	1,042	0,677	0,649	1,011	0,621	0,981	0,991	0,988	0,953	0,672
radix2_par_v2 (2)	1,108	0,628	0,410	0,415	0,616	0,389	0,593	0,576	0,591	0,583	0,400
radix2_par_v2 (4)	1,086	0,346	0,250	0,246	0,362	0,170	0,358	0,357	0,362	0,288	0,246
radix2_par_v2 (8)	1,086	0,399	0,250	0,258	0,384	0,252	0,370	0,369	0,372	0,366	0,256
radix4_par (1)	0,982	0,566	0,554	0,551	0,543	0,523	0,534	0,584	0,550	0,551	0,551
radix4_par (2)	0,956	0,479	0,439	0,444	0,442	0,428	0,446	0,453	0,466	0,453	0,441
radix4_par (4)	0,889	0,333	0,311	0,277	0,304	0,281	0,318	0,316	0,323	0,327	0,314
radix4_par (8)	0,860	0,338	0,312	0,282	0,319	0,288	0,309	0,320	0,319	0,321	0,315
<i>Stosunek czasu procesora do czasu ściany R</i>											
radix2_par (1)	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
radix2_par (2)	1,73	1,71	1,72	1,72	1,71	1,73	1,71	1,71	1,73	1,72	1,71
radix2_par (4)	2,62	3,05	3,06	3,07	3,05	3,07	3,07	3,06	3,06	3,04	3,07
radix2_par (8)	2,53	2,87	2,87	2,88	2,86	2,89	2,88	2,87	2,84	2,84	2,91
radix2_par_v2 (1)	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
radix2_par_v2 (2)	1,79	1,71	1,71	1,72	1,72	1,73	1,72	1,72	1,71	1,71	1,71
radix2_par_v2 (4)	2,94	3,26	3,27	3,28	3,24	3,27	3,27	3,27	3,25	3,25	3,27
radix2_par_v2 (8)	2,71	3,00	3,02	3,01	3,00	3,05	3,01	3,04	3,02	3,04	2,99
radix4_par (1)	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00	1,00
radix4_par (2)	1,65	1,65	1,66	1,66	1,65	1,70	1,66	1,65	1,66	1,65	1,66
radix4_par (4)	2,45	2,78	2,79	2,81	2,80	2,92	2,80	2,77	2,79	2,78	2,78
radix4_par (8)	2,39	2,70	2,67	2,73	2,72	2,82	2,70	2,67	2,70	2,72	2,70

6. PODSUMOWANIE I WNIOSKI

6.1. Wnioski końcowe

Przedstawione poniżej wnioski opierają się na pomiarach wykonanych na systemie Linux ogólnego przeznaczenia, który z założenia nie daje możliwości pracy w twardym systemie wbudowanym czasu rzeczywistego. Stosowana konfiguracja systemu oraz metodyka pomiarowa pozwoliły uzyskać rozstęp międzykwartylowy względny poniżej 4%, co może sugerować możliwość interpretacji różnic na poziomie 5-7%. Warto jednak zaznaczyć, że uzyskany rozstęp nie pozwala określić badanego systemu jako takiego, który mógłby pracować pod rygorem czasu rzeczywistego.

Najważniejsze wnioski z przeprowadzonych badań prezentują się następująco:

1. Największy pojedynczy zysk dało przejście z `-O0` na `-O1`. Skala tego zysku zależała od rozmiaru danych - wyniosła 5,11 przy $N = 16384$ i tylko 2,18 przy $N = 1048576$. Różnica ta może sugerować, że nawet skrajna różnica w jakości kodu przestawała mieć znaczenie, gdy o czasie wykonania decydował dostęp do pamięci. Warto przy tym wniosku nadmienić, że kompilatory GCC i Clang przyjmują poziom `-O0` jako domyślną opcję.
2. Ręczna wektoryzacja z użyciem instrukcji NEON dała wyraźną przewagę nad pozostałymi wariantami `radix-2` dla rozmiarów do $N = 16384$, ale przy $N = 1048576$ zysk zanikł całkowicie. Powyżej pojemności pamięci L2 cztery implementacje `radix-2` dzieliło już tylko 2,1% wobec 65% przy $N = 16384$, co oznaczało, że o wydajności decydowała wtedy organizacja dostępu do pamięci, a nie jakość wygenerowanego kodu. Wyniki te pozwoliły uznać jawne użycie instrukcji specyficznych dla architektury za skuteczną metodę optymalizacji, wymagającą jednak wiedzy wykraczającej poza znajomość samej listy instrukcji. Układ danych konieczny do efektywnego wykorzystania rejestrów wektorowych pogarszał bowiem lokalność dostępu: wariant NEON notował 5,43% chybień L1D wobec 1,08% dla `radix2_v0`.
3. Kompilatory GCC i Clang nie tłumaczyły kodu w języku C na jednakowy program wykonalny. W wykonanych badaniach Clang generował kod, który miał niższą medianę czasów wykonania przy rozmiarach danych wejściowych mieszczących się w pamięci podręcznej, kosztem istotnie większej wielkości programu. GCC tracił wektoryzację włączoną wcześniej przez `-O2` po dodaniu pewnej grupy innych flag - dla `radix2_v1` usuwało ją `-ffast-math` lub `-mcpu`, co wydłużało czas z 0,7562 do 0,8442 ms. Clang zachowywał ją we wszystkich zestawach. Sama automatyczna wektoryzacja okazała się zjawiskiem rzadkim: wystąpiła w 30 z 80 konfiguracji pod GCC i w 38 pod Clang, a poziomy `-O1` oraz `-Os` nie włączały jej w ogóle. Decydowała o niej pojedyncza flaga, a nie poziom optymalizacji.
4. Kompilator nie naprawiał samodzielnie błędów numerycznych tkwiących w algorytmie. Obserwacja ta była szczególnie widoczna przy zestawieniu dokładności dla 16384 próbek. Dla jednej implementacji kompilowanej na jedenaście sposobów wartości SNR rozchodziły się najwyżej o 1,31 dB, a zestawy zawierające `-ffast-math` obniżały wynik o nie więcej niż 0,45 dB. Porównanie między wariantami algorytmu pokazało zupełnie inną tendencję. Siedem wariantów osiągnęło 134,89–138,48 dB, natomiast `radix2_v0` tylko 91,52 dB, czyli o 44 dB mniej, co wskazywało na znaczące niedokładności wyników. Żaden poziom optymalizacji ani zestaw flag tej różnicy nie zmniejszył, poprawa nastąpiła dopiero po zmianie implementacji.

5. Powyżej poziomu -01 zyski były niesystematyczne. Najlepszy zestaw flag skracał czas względem -01 o kolejne 0,5-29%, przy medianie około 12%, jednak w szesnastu badanych konfiguracjach najkrótszy czas osiągało osiem różnych zestawów, a -03 nie wygrał w żadnej z nich. Podniesienie poziomu optymalizacji potrafiło przy tym zaszkodzić - dla `radix4_naive` pod GCC poziomy -02 i -03 okazały się o około 40% wolniejsze od -01. Uniwersalnie najlepszy zestaw flag zatem nie istniał: pewne było jedynie, że -00 dawał najgorszy wynik, a powyżej tego poziomu o wyniku decydowały konkretna implementacja i rozmiar przetwarzanych danych. Dobór flag był więc zadaniem pomiarowym, a uzyskany wynik należy potwierdzić analizą kodu wynikowego oraz metryk optymalizacji.
6. Rozmiar przetwarzanych danych warunkował dobór schematu algorytmu. Kryterium oparte na liczbie operacji arytmetycznych zachowywało ważność wyłącznie wtedy, gdy zbiór roboczy mieścił się w pamięci podręcznej pierwszego poziomu - przy $N = 1024$ oba bufor zajmowały 16 kB wobec 32 kB pojemności L1D i najszybszy był `splitradix`, schemat o najmniejszej liczbie mnożeń. Przy $N = 1048576$ ten sam schemat wypadł najgorzej, a zdecydowanie najszybszy okazał się `stockham`, przechodzący przez tablicę liniowo - koszt pojedynczej operacji motylkowej wyniósł dla niego 8,13 ns wobec 28,50 ns dla `splitradix`. Jak wynioskowano z analizy plików assemblera, zysk zaobserwowany dla wariantu Stockhama wynikał prawdopodobnie ze sposobu dostępu do pamięci, a nie, jakby sugerowała intuicja, z liczby wykonywanych działań. Opisany wariant, jako jedyny z badanych, nie wymagał wstępnej permutacji bitowej, a więc dodatkowego przebiegu po całej szesnastomegabajtowej tablicy. Działając na dwóch buforach, w każdym etapie czytał i zapisywał kolejne komórki obu tablic, podczas gdy warianty działające w miejscu odwoływały się do znacznie oddalonych elementów. Skutkiem tego rozwiązania było prawie dwukrotnie większe zapotrzebowanie na pamięć, co tłumaczyło, dlaczego ten sam schemat nie okazał się najszybszy przy $N = 1024$.
7. Zrównoleglenie opłacało się dopiero wtedy, gdy praca przypadająca na jeden etap przekraczała koszt mechanizmu synchronizacyjnego (w przypadku niniejszej pracy bariery). Przy $N = 4096$, gdzie na jedną transformację przypadało dwanaście barier, żaden (wyłączając warianty z flagą -00) z wariantów nie osiągnął przyspieszenia. Przy $N = 65536$ przyspieszenie sięgnęło natomiast 1,94. W stosowanej platformie sprzętowej najniższe czasy wykonania uzyskiwano przy stosowaniu czterech wątków, zgodnie z liczbą rdzeni platformy. Sam podział pracy między wątki okazał się równie istotny jak wybór schematu: przy niezmiennym algorytmie radix-2 podział płaskiej przestrzeni motylków dawał 1,94 wobec 1,51 dla podziału po blokach, który zmniejszał się znacząco w końcowych etapach, gdy liczba bloków transformaty spadała poniżej liczby wątków. Schemat najlepszy dla wersji jednowątkowej nie musiał być najlepszy po zrównolegleniu - `radix4_par`, najslabszy przy $N = 65536$ z wynikiem 1,28, okazał się najlepszy przy $N = 1048576$ z wynikiem 1,65, dzięki o połowę mniejszej liczbie etapów, a więc i barier. Przy analizie zaimplementowanych algorytmów wielowątkowych zauważono, że dobór flag dawał mniejsze korzyści. Dla obu wariantów radix-2 rozrzut czasów między zestawami nie przekraczał 2,4%, a więc pozostawał poniżej progu istotności, a jedynym wyjątkiem był `radix4_par` z zestawem `unroll`, dla którego przewaga wyniosła 8,3%. Wniosek ten mógł być zgodny z przewidywaniami teoretycznymi, ponieważ stosowane flagi kompilacji nie miały bezpośredniej możliwości optymalizacji komunikacji międzywątkowej, czyli barier.

Uzyskane wyniki pozwoliły porównać metody na zwiększenie wydajności co do rzędu wielkości. Dobór schematu obliczeń dopasowanego do rozmiaru danych oraz odejście od poziomu -00 dały efekty tego samego rzędu - odpowiednio około 3,5 raza między najlepszym a najgorszym schematem przy $N = 1048576$ oraz od 2,18 do 5,11 raza przy zmianie poziomu optymalizacji. Warto zaznaczyć przy tym, że drugi z nich nie wymagał zmiany ani jednej linii kodu programu źródłowego. Wyraźnie mniej wniosło strojenie flag powyżej poziomu -01, wartość zwykle około 12%.

Kompilator można w takim razie traktować jako realne narzędzie, lecz nie zastępuje on kluczowych decyzji projektowych. Praktyczny wniosek z przeprowadzonych badań jest taki, że optymalizację należy zaczynać od doboru algorytmu i odpowiedniej organizacji danych, ponieważ o czasie działania programu nie decyduje wyłącznie liczba wykonywanych działań arytmetycznych, lecz w porównywalnym stopniu wzorzec dostępu do pamięci.

6.2. Kierunek dalszych badań

Największy potencjał do dalszych badań ma jawne użycie instrukcji specyficznych dla stosowanej architektury sprzętowej. W pracy zastosowano je tylko w jednym wariancie i przy jednym układzie danych, co ujawniło konflikt między wektoryzacją a lokalnością dostępu do pamięci. Wartościowym kierunkiem badań może być sprawdzenie połączenia instrukcji wektorowych ze schematem obliczeń o korzystnym wzorcu dostępu do pamięci, z potencjalnym sprawdzeniem działania proponowanego schematu w wariantach wielowątkowych.

Na uwagę zasługują również badania samego mechanizmu synchronizacji wariantów wielowątkowych, zwłaszcza w kontekście stosowania flag optymalizacji. Istotną informacją byłoby w tym przypadku wyznaczenie prógów opłacalności stosowania wariantów wielowątkowych. Przy okazji realizacji proponowanych badań można by sprawdzić opłacalność blokowania obliczeń pod rozmiar pamięci podręcznej, stosowane w bibliotece FFTW [24].

SPIS RYSUNKÓW

2.1	Warstwowy model systemu komputerowego	11
2.2	Zestawienie przebiegów funkcji przydatności odpowiedzi $y(t)$ w dziedzinie czasu [1].	15
3.1	Operacja motylkowa algorytmu FFT o podstawie 2	24
3.2	Graf przepływu sygnałów algorytmu FFT z decymacją w czasie dla $N = 8$	24
3.3	Model "czarnej skrzynki" kompilatora	27
3.4	Klasyczny przebieg systemu przetwarzania języka programowania [10]	28
4.1	Uproszczony schemat rdzenia Cortex-A72 z zaznaczoną jednostką NEON. Opracowanie własne na podstawie [22].	36
4.2	Przebieg pojedynczej serii pomiarowej	47

SPIS TABEL

3.1	Permutacja bitowo-odwrócona indeksów próbek dla $N = 16$	23
3.2	Poziomy optymalizacji w kompilatorach GCC i Clang, na podstawie oficjalnych dokumentacji [18, 19]	34
4.1	Zestawienie zaimplementowanych wariantów FFT	37
5.1	Rozstęp międzykwartylowy bezwzględny i względny dla GCC i 1048576 próbek, wersja sekwencyjna. Pogrubiono najmniejszą wartość w każdym wierszu.	52
5.2	Mediana czasu wykonania transformaty dla GCC i Clang, 16384 próbek, wersja sekwencyjna. Wartości w milisekundach. Pogrubiono najmniejszą wartość w każdym wierszu.	53
5.3	Przyspieszenie względem -00 dla GCC i 16384 próbek, wersja sekwencyjna. Wartości bezwymiarowe, poziom -00 przyjęto za 1,00. Pogrubiono największą wartość w każdym wierszu.	54
5.4	Rozmiar sekcji kodu wynikowego algorytmu dla GCC i Clang, wersja sekwencyjna. Rozmiar sekcji .text w bajtach. Pogrubiono najmniejszą wartość w każdym wierszu.	55
5.5	Błąd średniokwadratowy i stosunek sygnału do szumu dla GCC i 16384 próbek, wersja sekwencyjna. Pogrubiono najmniejszy błąd i największy SNR w każdym wierszu.	56
5.6	Liczba statycznych instrukcji zmiennoprzecinkowych w kodzie wynikowym dla GCC i Clang, wersja sekwencyjna. Format komórki: liczba instrukcji wektorowych (SIMD), w nawiasie liczba instrukcji skalarnych. Pogrubiono komórki, w których wystąpiła wektoryzacja.	57
5.7	Mediana czasu wykonania transformaty dla GCC i 1048576 próbek, wersja sekwencyjna. Wartości w milisekundach. Pogrubiono najmniejszą wartość w każdym wierszu.	58
5.8	Współczynnik chybień pamięci podręcznej L1D i L2 dla GCC i 1048576 próbek, wersja sekwencyjna. Pogrubiono najmniejszą wartość w każdym wierszu z pominięciem -00.	58
5.9	Współczynnik chybień pamięci podręcznej L1D i L2 dla GCC i 16384 próbek, wersja sekwencyjna. Pogrubiono najmniejszą wartość w każdym wierszu z pominięciem -00.	59
5.10	Mediana czasu wykonania transformaty dla GCC i 1048576 próbek, wersja współbieżna. Wartości w milisekundach. Pogrubiono najmniejszą wartość w każdym wierszu.	59
5.11	Przyspieszenie względem jednego wątku dla GCC i trzech rozmiarów danych, wersja współbieżna. Wartości w krotnościach. Pogrubiono największą wartość w każdym wierszu z pominięciem -00.	61
5.12	Wykonane instrukcje, liczba instrukcji na cykl oraz stosunek czasu procesora do czasu ściany dla GCC i 1048576 próbek, wersja współbieżna. Pogrubiono najmniejszą liczbę instrukcji oraz największe IPC i R w każdym wierszu z pominięciem -00.	62

BIBLIOGRAFIA

- [1] Andrew S. Tanenbaum and Herbert Bos. *Systemy operacyjne*. Helion, Gliwice, 2024. Przekład: Radosław Meryk.
- [2] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. J. Wiley & Sons, Hoboken, N.J., 8th ed. edition, 2008.
- [3] K. C Wang. *Embedded and Real-Time Operating Systems*. Springer Nature, Cham, 1 edition, 2017.
- [4] Robert Love. *Linux kernel development*. Developer's library : essential references for programming professionals Linux kernel development. Addison Wesley, Place of publication not identified, 3rd ed. edition, 2010.
- [5] Jack Dongarra and Francis Sullivan. Guest editors' introduction to the top 10 algorithms. *Computing in Science & Engineering*, 2(1):22–23, 2000.
- [6] Tomasz P. Zieliński. *Cyfrowe przetwarzanie sygnałów*. Wydawnictwa Komunikacji i Łączności, 2007.
- [7] Thomas H. Cormen, Thomas H. Cormen, Charles Eric Leiserson, Ronald L. Rivest, Clifford Stein, and M.I.T. Press Wydawca. *Introduction to algorithms / Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein*. MIT Press, Cambridge, MA, third edition. edition, 2009.
- [8] James W Cooley and John W Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of computation*, 19(90):297–301, 1965.
- [9] William H. Press and Cambridge University Press. *Numerical recipes : the art of scientific computing / William H. Press [et al.]*. Cambridge University Press, Cambridge [etc, 3rd ed. edition, 2007.
- [10] Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman, Marek Włodarz, and Wydawnictwo Naukowe PWN Wydawca. *Kompilatory : reguły, metody, narzędzia / Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman ; [przekład Marek Włodarz]*. [Wydawnictwo Naukowe PWN SA], Warszawa, wydanie ii. edition, 2019.
- [11] Peter Prinz and Tony Crawford. *Język C w pigułce / Prinz, Peter*. APN Promise, 2nd edition edition, 2016.
- [12] Richard Stallman et al. The gnu project, 1998.
- [13] The Linux Kernel Documentation. Building Linux with Clang/LLVM. <https://docs.kernel.org/kbuild/llvm.html>. Dostęp: 19.08.2026.
- [14] Diego Novillo. Gcc an architectural overview, current status, and future directions. In *Proceedings of the linux symposium*, volume 2, page 185, 2006.
- [15] Chris Lattner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004.*, pages 75–86. IEEE, 2004.

- [16] LLVM Project. LLVM developer policy. <https://llvm.org/docs/DeveloperPolicy.html>. Dostęp: 19.08.2026.
- [17] Mehrdad Poorhosseini, Wolfgang Nebel, and Kim Grüttner. A compiler comparison in the risc-v ecosystem. In *2020 International Conference on Omni-layer Intelligent Systems (COINS)*, pages 1–6. IEEE, 2020.
- [18] Free Software Foundation. Optimization levels – GNAT user’s guide for native platforms. <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>.
- [19] The Clang Team. clang – the Clang C, C++, and Objective-C compiler (Command Guide). <https://clang.llvm.org/docs/CommandGuide/clang.html>.
- [20] ARM Limited. *NEON Programmer’s Guide*, version 1.0 edition, 2013. ARM DEN0018A, ID071613.
- [21] Raspberry Pi Ltd. Raspberry pi 4 model b. <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>, 2019. Specyfikacja techniczna, dostęp: [DO UZUPEŁNIENIA: data].
- [22] Arm Limited. Arm® Cortex®-a72 MPCore processor — technical reference manual. Technical Report ARM 100095_0001_02_en, Arm Limited, 2015. Revision: r0p1.
- [23] David R. Butenhof. *Programming with POSIX threads*. Addison-Wesley professional computing series. Addison Wesley, Place of publication not identified, 1st edition edition, 1997.
- [24] Matteo Frigo and Steven G Johnson. The design and implementation of fftw3. *Proceedings of the IEEE*, 93(2):216–231, 2005.
- [25] Daisuke Takahashi. *Fast fourier transform*. Springer, 2019.
- [26] Naga K Govindaraju, Brandon Lloyd, Yuri Dotsenko, Burton Smith, and John Manferdelli. High performance discrete fourier transforms on graphics processors. *SC*, 8:1–12, 2008.